HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO MÔN HỌC
PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Tiểu luận Final

Họ tên: Phan Văn Khôi
Mã sinh viên: B23DCCN447
Lớp: D23CTPM01-B
Giảng viên PGS. TS. Trần Đình Quế

Hà Nội – 2026

Mục lục

1 Lịch sử phát triển của trí tuệ nhân tạo **1**

1.1 Từ câu hỏi triết học đến bài toán tính toán . . . 1
1.2 Dartmouth và sự ra đời của một lĩnh vực . . . 2
1.3 AI biểu tượng: sức mạnh và giới hạn . . . 3
1.4 Các thời kỳ suy giảm kỳ vọng . . . 4
1.5 Từ quy tắc viết tay sang học máy thống kê . . . 4
1.6 Mạng nơ-ron và học sâu trở lại . . . 5
1.7 Từ mô hình chuyên biệt đến mô hình nền tảng . . . 6
1.8 Những dòng nghiên cứu song song . . . 7
1.9 Dữ liệu, phép đo và khả năng tái lập . . . 7
1.10 Các bước chuyển trong thị giác và ngôn ngữ . . . 8
1.11 AI hiện đại như một hệ thống hoàn chỉnh . . . 9

2 Các kỹ thuật học máy cơ bản **11**

2.1 Bài toán, quy trình và nguyên tắc so sánh . . . 11
2.1.1 Các họ mô hình cơ bản . . . 12
2.1.2 Metric theo bản chất nhiệm vụ . . . 13
2.2 Customer comments: phân loại nhận xét mua sắm . . . 13
2.3 Diabetes large: phân loại nguy cơ tiểu đường . . . 19
2.4 House price: hồi quy giá bất động sản . . . 26
2.5 Tổng hợp Chương 2 . . . 36

3 Mạng nơ-ron tích chập **37**

3.1 Từ ảnh số đến đặc trưng không gian . . . 37
3.1.1 Huấn luyện và các kỹ thuật cải thiện . . . 38
3.1.2 Kích thước feature map và số tham số . . . 39
3.1.3 Công bằng trong benchmark và ý nghĩa thời gian . . . 40
3.2 MNIST: chữ số viết tay . . . 40

3.3 CIFAR-10: ảnh vật thể tự nhiên . . . 48
3.4 Phân tích lỗi và lựa chọn mô hình khi sử dụng . . . 57
3.5 Tổng hợp Chương 3 . . . 58

4 Mạng nơ-ron hồi tiếp **59**
4.1 Chuỗi thời gian và trạng thái ẩn . . . 59
4.1.1 Thiết kế dự báo và chống rò rỉ tương lai . . . 61
4.1.2 Ví dụ về cửa sổ, chuẩn hóa và phép đo . . . 62
4.1.3 Đọc biểu đồ dự báo thay vì chỉ đọc bảng . . . 62
4.2 Amazon Stock Price: dự báo giá đóng cửa . . . 62
4.3 Avocado Price: nhiều chuỗi giá theo thị trường . . . 69
4.4 Kiểm tra luồng deploy trên localhost . . . 75
4.5 Giới hạn của hai bài toán dự báo . . . 76
4.6 Tổng hợp Chương 4 và ý nghĩa triển khai . . . 77

5 Knowledge Graph, RAG và Agent **78**
5.1 Từ cơ sở tri thức đến hệ hỏi đáp có nguồn . . . 78
5.2 Biểu diễn graph và luồng RAG . . . 79
5.2.1 Schema, truy vấn và nguồn dữ liệu . . . 79
5.2.2 Tạo tài liệu, truy xuất và sinh câu trả lời . . . 79
5.3 Agent chọn công cụ và giới hạn của GraphRAG . . . 81
5.4 Amazon Stock Price: hỏi đáp lịch sử giao dịch . . . 82
5.5 diabetes_large: hỏi đáp thống kê theo nhóm . . . 86
5.6 Triển khai và giới hạn . . . 91

6 Agent học tăng cường và agent dựa trên luật **92**
6.1 Giới thiệu và mục tiêu . . . 92
6.2 Môi trường và hai agent . . . 93
6.2.1 Trạng thái, hành động và động lực môi trường . . . 93
6.2.2 Agent A: Q-learning dạng bảng . . . 93
6.2.3 Agent B: chính sách dựa trên luật . . . 94
6.2.4 Phân tích lợi nhuận kỳ vọng một bước . . . 94
6.3 Cài đặt và thiết lập thí nghiệm . . . 95
6.3.1 Kiến trúc chương trình . . . 95
6.3.2 Các thay đổi so với mã tham khảo . . . 95
6.3.3 Tham số và quy trình đánh giá . . . 96
6.3.4 Trích đoạn mã nguồn chính . . . 96

6.3.4.1 Mô hình thị trường và môi trường . . . 96
6.3.4.2 Agent Q-learning . . . 98
6.3.4.3 Agent dựa trên luật . . . 98
6.3.4.4 Vòng huấn luyện . . . 99
6.3.4.5 Đánh giá tách biệt và đánh giá chính xác . . . 100
6.4 Kết quả thí nghiệm . . . 102
6.4.1 Thí nghiệm A: baseline . . . 102
6.4.2 Thí nghiệm B: quá trình học . . . 103
6.4.3 Thí nghiệm C: tốc độ học . . . 105
6.4.4 Thí nghiệm D: mức khám phá . . . 107
6.4.5 Thí nghiệm E: so sánh công bằng trên nhiều seed . . . 108
6.5 Trả lời mười câu hỏi . . . 109
Câu 1. Mô hình hóa agent và môi trường . . . 110
Câu 2. Quyết định dựa trên học và dựa trên luật . . . 110
Câu 3. Cập nhật Q-learning . . . 111
Câu 4. Khám phá và khai thác . . . 112
Câu 5. Phần thưởng và mục tiêu kinh doanh . . . 112
Câu 6. Đánh giá hiệu năng . . . 113
Câu 7. Tốc độ học và độ ổn định . . . 113
Câu 8. Tương tác nhiều agent và tính không dừng . . . 114
Câu 9. Hạn chế và độ bền vững . . . 115
Câu 10. Thiết kế agent tốt hơn . . . 117
6.6 Thảo luận, hạn chế và đề xuất . . . 120
6.6.1 Phân biệt quan sát và giải thích . . . 120
6.6.2 Các nhận xét chung . . . 120
6.6.3 Hạn chế của chính báo cáo này . . . 121
6.6.4 Đề xuất cải tiến . . . 121
6.7 Kết luận . . . 121

Tài liệu tham khảo **122**

Danh mục hình ảnh

1.1 Một số mốc phát triển AI và sự mở rộng cách biểu diễn tri thức. 3

2.1 Luồng học có giám sát và vị trí cần ngăn rò rỉ dữ liệu. 12
2.2 Phân bố nhãn, độ dài nhận xét và các thống kê mô tả của customer comments. . 15
2.3 Các từ xuất hiện nhiều theo lớp nhãn; tần suất từ chỉ mô tả dữ liệu, chưa chứng minh giá trị dự báo. 16
2.4 So sánh các metric phân loại customer comments trong notebook gốc; biểu đồ gốc có thêm Random Forest ngoài ba mô hình được chọn. 19
2.5 Ma trận nhầm lẫn chuẩn hóa của benchmark gốc; đối chiếu ba mô hình trong Bảng 2.2. 20
2.6 Giao diện deploy customer comments với nhận xét đầu vào, mô hình và kết quả dự đoán. 20
2.7 Phân phối biến số trên diabetes large theo nhãn. 21
2.8 Tỷ lệ tiểu đường theo các biến phân loại trong dữ liệu khảo sát. 22
2.9 Biểu đồ metric diabetes large của notebook gốc, gồm thêm Gaussian Naive Bayes; bảng trong tiểu luận chỉ so sánh ba mô hình đã chọn. 26
2.10 Ma trận nhầm lẫn của benchmark gốc; cần đặc biệt quan sát số âm tính giả ở lớp dương. 27
2.11 Giao diện deploy diabetes large với các chỉ số đầu vào, mô hình và kết quả dự đoán. 28
2.12 Khảo sát giá nhà, dữ liệu thiếu và đặc trưng bất động sản. 30
2.13 Phân phối các biến số chính trong mẫu nhà được xử lý. 30
2.14 Biểu đồ so sánh hồi quy của benchmark gốc, có thêm Ridge Regression ngoài ba mô hình của tiểu luận. 34
2.15 Phân tích phần dư của mô hình tốt nhất để nhận diện các miền giá khó dự báo. . 34
2.16 Giao diện deploy house price với thông tin căn nhà, mô hình và giá dự đoán. . . 35

3.1 Kernel 3×3 trượt trên ảnh 5×5 với stride 1, không padding. 38
3.2 Ảnh mẫu đại diện cho mười lớp chữ số trong MNIST. 42
3.3 Phân bố nhãn train và test của MNIST. 42

3.4 Loss và accuracy theo epoch của ba CNN trên MNIST. 45
3.5 So sánh Accuracy, Macro-Precision, Macro-Recall và Macro-F1 trên MNIST. . 47
3.6 Ma trận nhầm lẫn chuẩn hóa của ba CNN MNIST. 47
3.7 Giao diện deploy MNIST với chữ số đầu vào, mô hình và kết quả phân loại. . . 48
3.8 Ví dụ ảnh của từng lớp CIFAR-10: cùng một nhãn vẫn có nhiều góc nhìn và nền khác nhau. 50
3.9 Phân bố cân bằng của CIFAR-10 trên train và test. 50
3.10 Đường học của ba CNN trên CIFAR-10; validation cho thấy độ khó lớn hơn MNIST. 54
3.11 So sánh các metric phân loại CIFAR-10 trên cùng full test set. 56
3.12 Ma trận nhầm lẫn theo lớp của ba CNN CIFAR-10. 56
3.13 Giao diện deploy CIFAR-10 với ảnh đầu vào, mô hình và nhãn dự đoán. 57

4.1 Một RNN dùng chung trọng số tại các bước để dự báo sau cửa sổ quan sát. . . 60
4.2 Minh họa sự suy giảm gradient khi truyền qua nhiều bước thời gian. 60
4.3 Chuỗi Close, phân phối mức giá, thay đổi ngày và volume trong dữ liệu Amazon. 64
4.4 Rolling statistics, tương quan các cột OHLCV và thay đổi theo năm. 64
4.5 Learning curves, giá thực so với dự báo và residual trên test Amazon. 68
4.6 Giao diện deploy Amazon với chuỗi giá Close, framework và giá phiên kế tiếp dự đoán. 69
4.7 Tổng quan mức giá, sản lượng và khác biệt giữa các nhóm Avocado. 71
4.8 Mẫu hình mùa vụ, tương quan và biến động giá Avocado. 71
4.9 Đường học, actual–predicted và residual của Avocado trong notebook. 75
4.10 Giao diện deploy Avocado với region, type, chuỗi giá, framework và kết quả dự đoán. 76

5.1 Agent chọn một đường xử lý phù hợp rồi trả câu trả lời kèm nguồn và công cụ. 81
5.2 Khảo sát chuỗi giá, phân bố Close, biến động và Volume của Amazon. 83
5.3 Amazon: hỏi Close ngày 05/12/2023 qua Knowledge Graph. 86
5.4 Amazon: câu hỏi tóm tắt đi qua RAG và Qwen3. 86
5.5 Phân bố nhãn và các đặc trưng trong diabetes_large. 87
5.6 Diabetes: tỷ lệ nhãn 1 ở nữ được trả từ Knowledge Graph. 90
5.7 Diabetes: câu hỏi tóm tắt đi qua RAG và Qwen3. 91

6.1 Vòng tương tác của hai agent trong một bước thời gian. 93
6.2 Thí nghiệm B: thưởng mỗi bước theo episode, trung bình cửa sổ $K = 100$, tỷ lệ giảm giá của RL và ε (seed 42). 104
6.3 Giá trị kỳ vọng chính xác của chính sách tham lam trong lúc huấn luyện (seed 42). 104

6.4 Quỹ đạo tám giá trị Q cuối mỗi episode (seed 42). 105
6.5 Thí nghiệm C: đường học (TB $\pm$ SD trên 10 seed, cửa sổ 100 episode). 106
6.6 Thí nghiệm C: hiệu $Q((0,0),0) - Q((0,0),1)$ theo episode, một lần chạy với seed 1042 cho mỗi α. 106
6.7 Thí nghiệm C: thưởng đánh giá theo α. 106
6.8 Thí nghiệm D: đường học theo cấu hình khám phá (TB $\pm$ SD trên 10 seed). . . 107
6.9 Thí nghiệm D: thưởng đánh giá theo cấu hình khám phá. 108
6.10 Thí nghiệm E: RL và luật trên cùng chuỗi nhu cầu theo từng seed. 109
6.11 Câu 8: thưởng và tỷ lệ giảm giá trong huấn luyện khi đối thủ cố định và khi hai agent cùng học (TB 10 seed). 114
6.12 Câu 9–10: lợi nhuận đánh giá khi thị trường thay đổi. 116

Danh mục bảng

1.1 Các cột mốc tiêu biểu trong lịch sử AI 2
1.2 Ba cách tiếp cận trong lịch sử AI và câu hỏi đánh giá phù hợp 8

2.1 Ba dataset được dùng trong Chương 2 14
2.2 Ba mô hình được chọn trên customer comments (test set) 18
2.3 Ba mô hình được chọn trên diabetes large (test set) 25
2.4 Ba mô hình được chọn trên house price (test set) 33

3.1 Kích thước và cách chia dữ liệu CNN 39
3.2 Ba CNN trên full test set MNIST 46
3.3 Ba CNN trên full test set CIFAR-10 55

4.1 Nhiệm vụ dự báo và cách chia dữ liệu RNN 61
4.2 Ba mô hình trên test Amazon Stock Price 67
4.3 Ba mô hình trên test Avocado Price 73

5.1 Ranh giới giữa ba nhánh hỏi đáp Assignment 07 81
5.2 Câu hỏi minh họa trên Amazon Stock Price 85
5.3 Câu hỏi minh họa trên diabetes_large 90

6.1 Lợi nhuận kỳ vọng một bước của agent i theo nhu cầu d và hành động hiện tại b của đối thủ (output của chương trình). 94
6.2 Các thành phần chính của chương trình. 95
6.3 Tham số tham chiếu và cấu hình tái lập. 96
6.4 Kết quả đánh giá baseline (seed 42, 10.000 bước đánh giá). 102
6.5 Thưởng trung bình mỗi bước trong huấn luyện theo cửa sổ 500 episode (seed 42). 103
6.6 Ảnh hưởng của tốc độ học (10 seed). "Tối ưu" là số seed có chính sách cuối đạt 35,00 kỳ vọng; "Đổi chính sách" là số lần chính sách tham lam thay đổi trong 2.500 episode cuối. 105
6.7 Ảnh hưởng của khám phá (10 seed). Thưởng huấn luyện là trung bình 5.000 episode. 107

6.8 Bảng kết quả theo gợi ý của đề bài (đánh giá tham lam, 10 seed, 10.000 bước mỗi seed). 108
6.9 So sánh từng cặp RL–luật trên cùng chuỗi nhu cầu (`multi_seed_summary.csv`). 109
6.10 So sánh agent RL và agent dựa trên luật. 110
6.11 Đối thủ cố định và hai agent cùng học (đánh giá tham lam, 10 seed). 114
6.12 Giá trị kỳ vọng chính xác mỗi bước của các chính sách mốc trong từng kịch bản. 116
6.13 Lợi nhuận đánh giá mỗi bước (TB $\pm$ SD, 10 seed) và số seed đạt tối ưu. Kịch bản doanh thu báo lợi nhuận thực tế, không phải doanh thu. 116
6.14 Regret = tối ưu $-$ giá trị kỳ vọng chính xác (TB 10 seed). Kịch bản doanh thu tính theo đơn vị doanh thu. 119
6.15 Quan sát đo được và mức độ chắc chắn của giải thích. 120

Chương 1

Lịch sử phát triển của trí tuệ nhân tạo

1.1 Từ câu hỏi triết học đến bài toán tính toán

Trí tuệ nhân tạo (Artificial Intelligence – AI) là nỗ lực xây dựng hệ thống có thể thực hiện những công việc vốn cần đến trí tuệ con người: suy luận, học từ kinh nghiệm, nhận biết thế giới và sử dụng ngôn ngữ. Lịch sử AI không đi theo một đường thẳng liên tục. Có giai đoạn các nhà nghiên cứu tin rằng chỉ cần lập trình đủ quy tắc; có giai đoạn trọng tâm chuyển sang xác suất, dữ liệu và tối ưu; và gần đây là thời kỳ các mô hình quy mô lớn học biểu diễn từ lượng dữ liệu rất lớn. Để hiểu các kỹ thuật ML, CNN và RNN trong các chương sau, cần nhìn thấy câu hỏi khoa học nào đã dẫn đến từng bước chuyển ấy.

Năm 1950, Alan Turing đề xuất trong bài *Computing Machinery and Intelligence* một phép thử dựa trên đối thoại để thảo luận khả năng biểu hiện hành vi thông minh của máy [1]. Ông không cung cấp một thuật toán duy nhất cho AI. Giá trị của công trình nằm ở việc chuyển một câu hỏi khó định nghĩa, "máy có nghĩ được không", thành một tình huống có thể quan sát và tranh luận. Phép thử cũng bộc lộ giới hạn: biểu hiện ngôn ngữ giống con người không tự động chứng minh máy hiểu thế giới hay hành động đáng tin cậy.

Trước khi có tên gọi AI, nhiều nền tảng đã hình thành. Logic hình thức cho phép viết suy luận thành các bước có thể kiểm tra; lý thuyết tính toán làm rõ lớp bài toán máy có thể thực hiện; điều khiển học nghiên cứu phản hồi và thích nghi. Ý tưởng mạng nơ-ron ban đầu lấy cảm hứng từ hệ thần kinh, còn thống kê cung cấp cách xử lý bất định. Những dòng nghiên cứu này có điểm chung là xem "trí tuệ" như một quá trình có cấu trúc, có thể biểu diễn và thử nghiệm, thay vì chỉ là phẩm chất bí ẩn của con người.

Để theo dõi lịch sử này, có thể tách một hệ AI thành bốn câu hỏi. Hệ biểu diễn thông tin đầu vào như thế nào? Nó biến thông tin ấy thành kết luận theo quy tắc hay theo tham số học được? Nguồn tri thức đến từ chuyên gia, dữ liệu gán nhãn hay phản hồi từ môi trường? Cuối cùng, kết luận được kiểm tra bằng phép đo nào? Các giai đoạn lịch sử khác nhau chủ yếu ở câu trả lời cho bốn câu hỏi ấy. Ví dụ, hệ chuyên gia lưu tri thức dưới dạng luật do con người viết; mô hình phân loại ảnh học bộ lọc từ ảnh gán nhãn; hệ RAG ở Chương 5 tìm tài liệu rồi dùng mô hình ngôn ngữ diễn đạt bằng chứng.

Một phép thử hành vi như phép thử Turing đánh giá điều quan sát được ở đầu ra, còn một

benchmark thuật toán đánh giá hiệu quả trên một nhiệm vụ có đáp án. Hai loại đánh giá trả lời các câu hỏi khác nhau. Một chương trình đối thoại mạch lạc vẫn có thể sai khi tính toán; một bộ phân loại đạt accuracy cao có thể thất bại khi ảnh chụp khác môi trường huấn luyện. Do đó, ngay từ phần lịch sử cần phân biệt *năng lực được trình diễn*, *cơ chế tạo kết quả* và *độ tin cậy khi sử dụng*.

1.2 Dartmouth và sự ra đời của một lĩnh vực

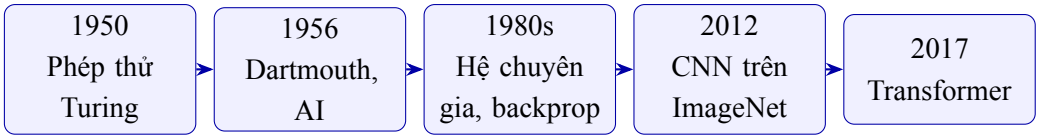
Đề xuất dự án nghiên cứu mùa hè tại Dartmouth được viết năm 1955, với dự định tổ chức vào năm 1956, đã dùng tên gọi "artificial intelligence" cho một chương trình nghiên cứu chung [2]. Tác giả đặt giả thuyết rằng những khía cạnh của học tập và trí thông minh có thể được mô tả đủ chính xác để máy mô phỏng. Đây là tuyên bố định hướng, không phải bằng chứng rằng bài toán đã được giải. Nó tạo ra một cộng đồng nghiên cứu có ngôn ngữ, mục tiêu và những câu hỏi riêng.

Trong thời kỳ đầu, máy tính rất hạn chế về bộ nhớ và tốc độ nhưng các bài toán nhỏ lại hứa hẹn. Chương trình có thể chứng minh định lý, giải trò chơi hoặc thao tác biểu thức ký hiệu. Cách tiếp cận chủ đạo là biểu diễn tri thức bằng ký hiệu rồi tìm kiếm trên các phép biến đổi hợp lệ. Nếu một bài toán có trạng thái ban đầu, đích và tập luật chuyển trạng thái, thuật toán tìm kiếm có thể xây dựng chuỗi hành động dẫn tới lời giải. Thành công trên miền hẹp khiến nhiều người kỳ vọng hệ thống tổng quát sẽ đến nhanh hơn thực tế.

Bảng 1.1: Các cột mốc tiêu biểu trong lịch sử AI

Thời điểm	Cột mốc và ý nghĩa
1950	Turing đặt vấn đề đánh giá hành vi thông minh qua đối thoại.
1955–1956	Đề xuất và hội thảo Dartmouth định danh lĩnh vực AI.
1958	Perceptron thúc đẩy nghiên cứu học từ ví dụ.
1960–1970	Tìm kiếm, suy luận ký hiệu và hệ thống hội thoại trong miền hẹp.
1980	Hệ chuyên gia được triển khai trong một số nghiệp vụ; học bằng lan truyền ngược phát triển.
1990–2000	Học thống kê và dữ liệu quy mô lớn trở thành nền tảng ứng dụng.
2012	CNN sâu tạo bước ngoặt trong phân loại ảnh ImageNet.
2017	Kiến trúc Transformer mở rộng cách học quan hệ trong chuỗi.

Hình 1.1 đặt một số mốc cạnh nhau để thấy rõ sự mở rộng của nguồn tri thức: từ đối thoại và luật ký hiệu, qua tham số học từ dữ liệu, đến mô hình sâu. Các hướng cũ vẫn tiếp tục tồn tại sau khi hướng mới xuất hiện; đường thời gian chỉ biểu thị mốc phát triển, không biểu thị việc một phương pháp bị thay thế hoàn toàn.



Hình 1.1: Một số mốc phát triển AI và sự mở rộng cách biểu diễn tri thức.

1.3 AI biểu tượng: sức mạnh và giới hạn

AI biểu tượng xem trí thông minh như khả năng thao tác các biểu diễn có nghĩa. Một hệ thống luật dạng "nếu–thì" có thể giải thích tại sao nó đi tới một kết luận. Tìm kiếm theo chiều rộng, tìm kiếm theo chiều sâu và các heuristic giúp giải các bài toán lập kế hoạch hoặc trò chơi. Trong những miền có luật rõ ràng, đây là một ưu điểm lớn: kiến thức chuyên gia có thể được mã hóa thành quy tắc và kiểm tra từng bước suy luận.

Vấn đề xuất hiện khi hệ thống phải đối mặt thế giới mở. Một bức ảnh không đi vào máy dưới dạng "con mèo đang ngồi trên ghế" mà dưới dạng hàng nghìn hoặc hàng triệu giá trị điểm ảnh. Một câu nhận xét của khách hàng có thể chứa tiếng lóng, phủ định, lỗi chính tả và hàm ý. Muốn xử lý bằng luật, người thiết kế phải liệt kê khối lượng lớn trường hợp, trong khi ngoại lệ luôn tiếp tục xuất hiện. AI biểu tượng vì vậy gặp nút thắt về thu nhận tri thức và độ bền khi môi trường thay đổi.

Hệ chuyên gia của thập niên 1980 minh họa rõ cả hai mặt. Khi tri thức chuyên môn ổn định và phạm vi hẹp, hệ có thể hỗ trợ quyết định khá hiệu quả. Khi luật cần cập nhật liên tục, các quy tắc phụ thuộc nhau trở nên khó bảo trì. Hệ thống cũng khó định lượng mức độ không chắc chắn của dữ liệu đầu vào. Bài học quan trọng là một biểu diễn minh bạch vẫn cần cơ chế học và kiểm định bằng dữ liệu thực tế. Ngày nay, luật và tri thức cấu trúc không biến mất; chúng thường kết hợp với mô hình thống kê trong các hệ thống hoàn chỉnh.

Về kỹ thuật, một hệ ký hiệu thường gồm tập sự kiện, tập luật và bộ suy diễn. Nếu có luật "nếu A và B thì C", bộ suy diễn chỉ kết luận C khi hai tiền đề đã được xác nhận. Trong tìm kiếm trạng thái, một lời giải là đường đi từ trạng thái đầu tới trạng thái đích; heuristic ước lượng trạng thái nào đáng mở rộng trước. Tính kiểm tra được là ưu điểm: người dùng có thể lần theo luật hoặc đường đi. Giới hạn nằm ở đầu vào và độ bao phủ luật. Nếu khái niệm A được trích sai từ ảnh hoặc câu văn, suy diễn hợp lệ về mặt hình thức vẫn có thể dẫn tới kết luận sai về thực tế.

Nút thắt thu nhận tri thức không chỉ là số lượng luật. Các chuyên gia có thể dùng những thuật ngữ khác nhau cho cùng hiện tượng, ngoại lệ có thể mâu thuẫn với quy tắc chung, và tri thức thay đổi theo thời gian. Để duy trì hệ thống, người thiết kế phải xác định phạm vi hiệu lực, thứ tự ưu tiên và cách xử lý dữ kiện thiếu. Điều này giải thích vì sao một cơ sở tri thức có cấu trúc ở Chương 5 rất hữu ích cho câu hỏi truy vấn xác định, nhưng không tự động thay thế mô hình học từ dữ liệu hoặc phép tính thống kê.

1.4 Các thời kỳ suy giảm kỳ vọng

Thuật ngữ "AI winter" mô tả những giai đoạn đầu tư và niềm tin vào AI suy giảm sau khi kỳ vọng vượt quá năng lực kỹ thuật. Có nhiều nguyên nhân cùng lúc: phần cứng đắt, bộ nhớ ít, dữ liệu chưa đủ, bài toán thực tế phức tạp hơn bài toán trình diễn và một số hệ thương mại không đạt lợi ích như hứa hẹn. Không nên coi các giai đoạn này là lúc nghiên cứu dừng hẳn. Nhiều ý tưởng nền tảng vẫn tiếp tục được phát triển trong thống kê, nhận dạng mẫu, tối ưu và khoa học nhận thức.

Từ góc nhìn phương pháp, các đợt suy giảm là một lời nhắc về cách đánh giá tiến bộ. Một chương trình thắng trong trò chơi nhỏ không chứng minh nó có năng lực giải quyết mọi vấn đề. Một mô hình đạt điểm cao trên tập kiểm tra quen thuộc cũng chưa chắc hoạt động tốt khi dữ liệu thay đổi. Vì vậy, benchmark phải ghi rõ tập dữ liệu, cách chia train/test, metric và giới hạn ngoại suy. Tinh thần ấy được áp dụng ở ba chương thực nghiệm của tiểu luận này.

Một nguyên nhân thường bị bỏ qua là sự lệch nhau giữa *bài toán trình diễn* và *bài toán vận hành*. Chương trình suy luận trong một thế giới nhỏ có thể giả định mọi sự kiện cần thiết đã được nhập và các quy tắc không mâu thuẫn. Ứng dụng thực tế phải xử lý dữ kiện sai, thiếu hoặc thay đổi; đồng thời cần giải thích khi nào nó không biết đáp án. Tương tự, bộ ảnh chuẩn có nhãn sạch hơn ảnh người dùng tải lên từ điện thoại. Khi công bố kết quả, cần nêu rõ điều kiện mà kết luận đúng. Những giai đoạn kỳ vọng giảm đã góp phần thúc đẩy thói quen đối chiếu hệ thống với điều kiện sử dụng thay vì chỉ với ví dụ đẹp.

1.5 Từ quy tắc viết tay sang học máy thống kê

Học máy (Machine Learning – ML) thay đổi câu hỏi thiết kế: thay vì viết mọi quy tắc, ta xác định một họ mô hình, một mục tiêu tối ưu và dữ liệu ví dụ. Mô hình học tham số để dự đoán nhãn hoặc giá trị chưa biết. Một bài toán giám sát có thể viết thành

$$\hat{\theta} = \arg\min_{\theta} \left[\frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f_\theta(x_i), y_i) + \lambda \Omega(\theta) \right]. \tag{1.1}$$

Ở đây hàm mất mát đo độ sai, còn thành phần điều chuẩn hạn chế việc khớp quá mức vào tập huấn luyện. Tư duy này đưa xác suất và khả năng tổng quát hóa trở thành trung tâm của AI ứng dụng.

Trong công thức trên, n là số ví dụ train, x_i là đầu vào thứ i, y_i là nhãn hoặc giá trị cần dự đoán, f_θ là mô hình có tham số θ, còn $\mathcal{L}$ đo sai số trên một ví dụ. Trung bình $\frac{1}{n}\sum_i \mathcal{L}$ là rủi ro thực nghiệm: giá trị nhỏ cho biết mô hình khớp dữ liệu train. $\Omega(\theta)$ phạt một số dạng mô hình quá phức tạp, còn $\lambda \geq 0$ quyết định độ mạnh của phạt. Ký hiệu arg min trả về bộ tham số đạt mục tiêu nhỏ nhất, không phải giá trị nhỏ nhất của hàm. Với hồi quy tuyến tính, chọn loss bình phương và $\Omega(\theta) = \|\theta\|_2^2$ dẫn tới một dạng điều chuẩn L2; với phân loại, loss thường là cross-entropy.

Nếu chỉ tối thiểu hóa lỗi train, mô hình có thể ghi nhớ nhiễu hoặc trường hợp hiếm. Chẳng hạn một cây quyết định đủ sâu có thể tách từng dòng train thành lá riêng và đạt lỗi gần bằng

không, nhưng một dòng mới ở vùng ít quan sát có thể được dự đoán kém. Validation giúp chọn độ sâu hoặc λ mà không dùng test; test chỉ ước lượng kết quả sau khi các lựa chọn đã hoàn tất. Sự khác biệt giữa ba tập này là khác biệt về *vai trò*, không chỉ về tỷ lệ số dòng. Khi thử nhiều cấu hình rồi chọn theo test, test đã tham gia quá trình học gián tiếp và kết quả thường lạc quan.

Các phương pháp tuyến tính, cây quyết định, Naive Bayes và mô hình hạt nhân phát triển mạnh khi dữ liệu số hóa ngày càng nhiều. Khả năng đánh giá bằng tập dữ liệu tách riêng giúp cộng đồng so sánh phương pháp rõ ràng hơn. Tuy nhiên, mô hình truyền thống thường cần kỹ thuật đặc trưng. Với văn bản, ta phải chuyển từ thành vector như TF-IDF; với bất động sản, ta cần mã hóa địa điểm, diện tích và thời gian; với ảnh, các đặc trưng thiết kế tay thường không nắm được hết cấu trúc. Những khó khăn ấy mở đường cho học biểu diễn bằng mạng nơ-ron.

Một bước chuyển khác là thừa nhận tính bất định. Dự đoán không chỉ là một nhãn; nó có thể là xác suất hoặc khoảng giá trị. Trong bài toán mất cân bằng, ngưỡng ra quyết định quan trọng không kém khả năng xếp hạng. Ví dụ ở dữ liệu tiểu đường, lớp dương chỉ chiếm 8,5%; mô hình đoán mọi người "không bệnh" vẫn đạt 91,5% accuracy nhưng không phát hiện được ca nào. Vì vậy sự trưởng thành của ML gắn với cả lý thuyết mô hình lẫn phương pháp kiểm định.

1.6 Mạng nơ-ron và học sâu trở lại

Perceptron của Rosenblatt là một trong những mô hình học trọng số từ ví dụ sớm và có ảnh hưởng [3]. Một perceptron đơn chỉ tạo ra ranh giới tuyến tính, nên không giải được mọi dạng quan hệ. Khi ghép nhiều tầng phi tuyến, mạng có khả năng biểu diễn phong phú hơn; thách thức là tính gradient và tối ưu hiệu quả. Công trình về lan truyền ngược những năm 1980 giúp huấn luyện mạng nhiều tầng thực tế hơn [4]. Dù vậy, dữ liệu và năng lực tính toán thời kỳ đó vẫn hạn chế việc mở rộng.

Học sâu phát triển mạnh khi ba điều kiện gặp nhau: dữ liệu số lớn, phần cứng tính toán song song và kỹ thuật tối ưu tốt hơn. CNN tận dụng cấu trúc không gian của ảnh bằng bộ lọc chia sẻ trọng số. RNN xử lý tín hiệu tuần tự bằng trạng thái ẩn truyền theo thời gian. Các mô hình này không loại bỏ nhu cầu thiết kế thí nghiệm; chúng chuyển phần việc trích đặc trưng từ người lập trình sang quá trình học tham số. Sự khác biệt giữa dữ liệu ảnh, văn bản và chuỗi thời gian vẫn quyết định kiến trúc phù hợp.

Năm 2012, hệ CNN sâu trong nghiên cứu ImageNet của Krizhevsky và cộng sự cho thấy hiệu quả nổi bật của huấn luyện mạng sâu trên tập ảnh lớn [5]. Cột mốc này thường được nhắc tới vì nó chứng minh học biểu diễn có thể vượt nhiều quy trình đặc trưng thủ công trên một benchmark lớn. Sau đó, kết nối tắt, chuẩn hóa và các kỹ thuật regularization khiến việc huấn luyện mạng sâu ổn định hơn. Thành công của học sâu không nên được hiểu là mọi bài toán đều cần mạng lớn: ở dữ liệu bảng nhỏ hoặc dữ liệu có ít tín hiệu, một cây quyết định hay hồi quy tuyến tính vẫn có thể là lựa chọn tốt hơn.

1.7 Từ mô hình chuyên biệt đến mô hình nền tảng

Trong xử lý chuỗi, RNN và các biến thể LSTM, GRU từng là kiến trúc chủ lực vì có thể cập nhật trạng thái theo từng bước. Hạn chế của chúng là khó song song hóa toàn bộ chuỗi và có thể gặp vấn đề gradient khi phụ thuộc kéo dài. Kiến trúc Transformer năm 2017 dùng attention để học quan hệ giữa các vị trí mà không cần truyền trạng thái tuần tự theo đúng cách của RNN [6]. Từ đó, mô hình ngôn ngữ quy mô lớn trở thành một hướng phát triển quan trọng của AI hiện đại.

Mô hình nền tảng có thể được tiền huấn luyện trên dữ liệu rất lớn rồi thích nghi với nhiều nhiệm vụ. Điều này thay đổi cách xây dựng sản phẩm: thay vì luôn huấn luyện từ đầu, người phát triển có thể dùng mô hình có sẵn, tinh chỉnh hoặc bổ sung dữ liệu bằng truy xuất. Đồng thời, các vấn đề về nguồn dữ liệu, sai lệch, chi phí tính toán, quyền riêng tư và độ tin cậy trở nên rõ nét hơn. Một câu trả lời trôi chảy không đảm bảo đúng sự thật; một dự đoán có xác suất cao trong môi trường thử nghiệm không đảm bảo an toàn ở môi trường vận hành.

Attention trong Transformer gán trọng số cho mối liên hệ giữa các vị trí của một chuỗi; trọng số này được học từ dữ liệu và phụ thuộc ngữ cảnh đầu vào. Cơ chế đó khác RNN ở chỗ nhiều vị trí có thể được xử lý song song trong một tầng, nhưng chi phí bộ nhớ vẫn tăng theo chiều dài ngữ cảnh ở self-attention đầy đủ. Khả năng sinh ngôn ngữ mạnh không tương đương khả năng truy cập số liệu mới hoặc số liệu riêng của bài thực nghiệm. Vì vậy hệ RAG cung cấp tài liệu tại thời điểm hỏi, còn Knowledge Graph cung cấp khóa và quan hệ tường minh; hai thành phần này nối những câu hỏi lịch sử về biểu diễn tri thức với ứng dụng hiện đại.

Một head của scaled dot-product attention được viết là

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{1.2}$$

Q là ma trận truy vấn, K là ma trận khóa, V là ma trận giá trị; mỗi hàng của $QK^\top$ đo mức liên hệ giữa một vị trí đang xét và các vị trí được phép tham chiếu. d_k là số chiều của vector khóa; chia cho $\sqrt{d_k}$ giúp điểm không tăng quá lớn khi chiều tăng. Softmax biến các điểm của từng hàng thành trọng số không âm có tổng bằng 1, rồi nhân với V để trộn thông tin. Công thức mô tả một phép biến đổi biểu diễn, không tự kiểm tra tính đúng của dữ kiện mà mô hình viết ra. Trong hệ hỏi đáp của Chương 5, văn bản nguồn vẫn phải được truy xuất và gắn ID trước khi mô hình sinh câu trả lời [6, 13].

Nhìn toàn bộ lịch sử, AI liên tục di chuyển giữa hai yêu cầu: biểu diễn được tri thức và học được từ kinh nghiệm. AI biểu tượng mạnh ở cấu trúc, học thống kê mạnh ở suy luận từ dữ liệu, còn học sâu mạnh ở việc rút trích biểu diễn phức tạp. Ba chương sau đi từ mô hình ML trên bảng và văn bản, qua CNN trên ảnh, đến RNN trên chuỗi thời gian. Mục tiêu không phải tuyên bố kiến trúc nào luôn tốt nhất, mà là chỉ ra dữ liệu, cách đánh giá và chi phí triển khai quyết định kết luận kỹ thuật như thế nào.

1.8 Những dòng nghiên cứu song song

Lịch sử AI thường được kể như chuỗi các kiến trúc thay thế nhau, nhưng trên thực tế nhiều hướng tồn tại đồng thời. Nghiên cứu tìm kiếm và lập kế hoạch tập trung vào hành động: với một mục tiêu và các phép chuyển trạng thái, máy phải chọn đường đi. Nghiên cứu tri thức tập trung vào nội dung: máy cần biết điều gì về thế giới để suy luận. Nghiên cứu nhận dạng mẫu tập trung vào quan sát: từ tín hiệu có nhiễu, máy phải nhận ra một cấu trúc. Ba câu hỏi ấy không thể giải quyết chỉ bằng một thuật toán. Một robot chẳng hạn cần nhìn thấy vật, suy ra vị trí, lập kế hoạch di chuyển và xử lý thay đổi bất ngờ.

Các hệ dựa trên luật có thể diễn đạt điều kiện nghiệp vụ rõ ràng, nhưng thường gặp khó khi dữ liệu không đầy đủ. Hướng xác suất đưa vào cách nói có điều kiện: một dấu hiệu không làm kết luận chắc chắn, mà thay đổi mức tin tưởng vào các giả thuyết. Cách nhìn này quan trọng đối với chẩn đoán, nhận dạng và dự báo. Nếu hai biến cùng phụ thuộc vào nguyên nhân thứ ba, coi chúng là độc lập có thể dẫn tới bằng chứng bị đếm hai lần. Đó là lý do mô hình xác suất cần được xây dựng và kiểm tra trên dữ liệu, không chỉ dựa vào trực giác về quan hệ giữa các biến.

Trong nghiên cứu mạng nơ-ron, ý tưởng học trọng số cạnh tranh với cách viết luật. Người thiết kế không cần viết quy tắc cho từng pixel của chữ số 8; họ cung cấp ví dụ và một hàm mục tiêu. Đổi lại, kết quả thường khó diễn giải bằng vài luật ngắn. Sự đánh đổi giữa minh bạch và khả năng học từ tín hiệu phức tạp đã tồn tại từ rất sớm. Các hệ thống hiện đại thường phối hợp nhiều cách: bộ lọc quy tắc kiểm tra điều kiện cứng, mô hình học máy ước lượng xác suất, và con người rà soát các trường hợp có rủi ro cao.

Một dòng khác là học tăng cường, nơi tác nhân chọn hành động và nhận phản hồi từ môi trường. Khác bài toán giám sát có đáp án cho từng ví dụ, phản hồi có thể đến trễ sau nhiều hành động. Điều này làm vấn đề thử hay khai thác trở nên trung tâm: tác nhân phải thử phương án mới để học nhưng vẫn muốn tối đa hóa phần thưởng hiện tại. Lập kế hoạch, điều khiển tối ưu và học tăng cường kết nối rất sâu với nhau. Tuy các notebook trong tiểu luận chỉ dùng học có giám sát, sự tồn tại của hướng này cho thấy AI rộng hơn việc phân loại và hồi quy.

1.9 Dữ liệu, phép đo và khả năng tái lập

Từ cuối thế kỷ XX, dữ liệu số ngày càng nhiều đã thay đổi đơn vị tiến bộ của AI. Một phương pháp có thể được đánh giá trên cùng bộ dữ liệu với phương pháp khác; các phép đo thống nhất giúp phát hiện cải thiện thật và cả cải thiện do thay đổi điều kiện thử. Tập train dùng để học, validation để chọn cấu hình, test để đánh giá cuối cùng. Nếu xem test nhiều lần rồi điều chỉnh mô hình, test dần trở thành một phần của quá trình chọn lựa và không còn đại diện cho dữ liệu chưa thấy. Đây là vấn đề phương pháp luận có thể làm sai lệch cả kết luận của một công trình nghiên cứu.

Khả năng tái lập đòi hỏi ghi lại không chỉ tên mô hình mà cả phiên bản dữ liệu, seed, quy trình tiền xử lý, cách chia tập, metric và phần cứng. Chẳng hạn một con số accuracy của CNN là mô tả thiếu: nó không cho biết ảnh thuộc dataset nào, phép augmentation ra sao, có dùng tập

test để chọn epoch hay không. Trong bài chuỗi thời gian, tách ngẫu nhiên có thể làm rò rỉ tương lai; trong bài văn bản, fit TF-IDF trước khi tách dữ liệu có thể làm lộ từ vựng của test. Nhiều tiến bộ đáng tin cậy của AI đến từ kỷ luật thí nghiệm chứ không chỉ từ một công thức mới.

Để một kết quả có thể lặp lại, chỉ lưu seed chưa đủ. Các thư viện có thể dùng thuật toán khác nhau giữa CPU và GPU; một số phép tính song song không xác định hoàn toàn thứ tự cộng số thực. Báo cáo nên ghi phiên bản thư viện, cách khởi tạo tham số, số epoch, quy tắc chọn checkpoint, cách xử lý giá trị thiếu và liệu phép đo thời gian có gồm khởi tạo mô hình hay không. Nếu hai lần chạy khác nhau vài phần nghìn điểm F_1, mức chênh đó có thể nhỏ hơn biến thiên do seed hoặc phân chia dữ liệu. Việc nêu giới hạn của phép đo khiến kết luận kỹ thuật có thể kiểm chứng và tránh biến một lần chạy thành khẳng định phổ quát.

Khi dữ liệu lớn lên, vấn đề đại diện cũng nổi bật hơn. Một tập ảnh chụp trong điều kiện sáng tốt có thể không bao phủ ảnh thiếu sáng; một tập nhận xét từ một cửa hàng không phản ánh khách hàng ở quốc gia khác. Sai số trung bình có thể che giấu sai số cao ở một nhóm nhỏ. Vì vậy, bên cạnh metric chung cần xem phân bố dữ liệu, ma trận nhầm lẫn, phần dư và thay đổi theo thời gian. Các hình EDA ở Chương 2–4 có vai trò đó: chúng mô tả môi trường mà con số benchmark có hiệu lực.

Bảng 1.2: Ba cách tiếp cận trong lịch sử AI và câu hỏi đánh giá phù hợp

Cách tiếp cận	Nguồn năng lực chính	Câu hỏi kiểm định
Luật và tìm kiếm	Tri thức viết tay, không gian trạng thái	Luật có bao phủ ngoại lệ và suy luận có đúng không?
Học máy thống kê	Đặc trưng và dữ liệu có nhãn	Mô hình tổng quát hóa ra sao trên dữ liệu chưa thấy?
Học sâu	Biểu diễn học từ dữ liệu lớn	Chất lượng có bền khi đổi miền, chi phí và dữ liệu không?

1.10 Các bước chuyển trong thị giác và ngôn ngữ

Sự phát triển của thị giác máy tính cho thấy rõ quá trình đi từ đặc trưng thiết kế tay sang đặc trưng học được. Trước học sâu, một hệ nhận dạng thường tách việc tạo đặc trưng khỏi bộ phân loại: nhà nghiên cứu chọn phép lọc cạnh, mô tả hình dạng hoặc kết cấu, sau đó đưa vector đặc trưng vào mô hình thống kê. CNN gộp hai bước vào cùng một quá trình tối ưu. Bộ lọc ở tầng đầu và phép phân loại ở tầng cuối được điều chỉnh theo cùng loss. Điều này làm mô hình linh hoạt hơn, nhưng cũng đòi hỏi nhiều ví dụ được gắn nhãn và khả năng tính toán đủ lớn.

Đột phá của CNN trên benchmark ảnh lớn năm 2012 không xuất hiện độc lập. Dữ liệu ảnh quy mô lớn cung cấp đủ biến thể để mạng học biểu diễn có ích; phần cứng song song rút ngắn thời gian thử nghiệm; và kỹ thuật huấn luyện làm giảm khó khăn tối ưu. Sau đó, kiến trúc

residual cho phép tín hiệu đi qua đường tắt giữa các tầng, góp phần mở rộng độ sâu mà không khiến việc huấn luyện trở nên quá khó [7]. Điều này không có nghĩa cứ tăng tầng là tốt: nếu dữ liệu ít, nhãn nhiễu hoặc đầu vào đơn giản, mạng lớn có thể tốn tài nguyên mà cải thiện rất nhỏ. Hai dataset ở Chương 3 minh họa điều đó: MNIST dễ hơn nhiều so với CIFAR-10.

Trong ngôn ngữ, các hệ dựa trên luật ngữ pháp và từ điển từng chiếm vị trí quan trọng. Mô hình thống kê sau đó học mẫu từ kho văn bản, nhưng biểu diễn rời rạc khó nắm quan hệ xa giữa các từ. RNN đem lại cách đọc từng token theo thứ tự, lưu thông tin trong trạng thái ẩn; LSTM và GRU giúp xử lý phụ thuộc dài hơn. Attention đặt câu hỏi khác: khi tạo một đầu ra, mô hình nên nhìn vào vị trí nào của chuỗi đầu vào? Transformer mở rộng ý tưởng này bằng cách dùng attention làm thành phần chủ đạo [6]. Khả năng tính toán song song hơn trong huấn luyện là một lý do khiến hướng này mở rộng quy mô mạnh.

Dù kiến trúc học sâu hiện đại gây chú ý, những bài toán nhỏ vẫn cần phương pháp cơ bản. Một bảng vài chục nghìn dòng có thể được giải tốt bằng cây và hồi quy; một chuỗi giá có thể được dự báo rất mạnh bằng giá gần nhất. Lịch sử kỹ thuật không phải cuộc thay thế hoàn toàn, mà là việc bổ sung công cụ với giả định và chi phí khác nhau. Một người làm AI cần hiểu baseline để biết phần cải thiện của mô hình mới đến từ đâu. Kết luận có giá trị nhất là kết luận có thể kiểm tra: trên dataset nào, dưới cách chia nào, theo metric nào và với chi phí nào.

1.11 AI hiện đại như một hệ thống hoàn chỉnh

Sự dịch chuyển từ phòng thí nghiệm sang sản phẩm diễn ra qua nhiều lớp công nghệ. Ở giai đoạn các hệ chuyên gia phổ biến, phần lớn tri thức nằm trong bộ luật và công việc tốn kém là hỏi chuyên gia, chuẩn hóa câu trả lời rồi kiểm tra xung đột giữa các luật. Khi học máy thống kê phát triển, công việc chuyển sang thu thập nhãn, thiết kế đặc trưng và lựa chọn metric. Với học sâu, một phần thiết kế đặc trưng được tự động hóa nhưng nhu cầu dữ liệu, tính toán và theo dõi chất lượng trở nên lớn hơn. Do đó sự thay đổi của AI không chỉ là danh sách thuật toán mới; nó còn là sự thay đổi cách tổ chức công việc kỹ thuật.

Thành tựu trong từng giai đoạn thường phụ thuộc vào bài toán được chọn để đo. Chứng minh định lý ưu tiên suy luận chính xác theo luật; nhận dạng ảnh ưu tiên khả năng tổng quát hóa trước biến thiên pixel; mô hình ngôn ngữ được đánh giá bằng nhiều dạng nhiệm vụ và tương tác. Một hệ có thể rất mạnh theo phép đo của giai đoạn này nhưng yếu ở phép đo khác. Ví dụ, mô hình ảnh có thể nhận lớp chính xác mà không hiểu quan hệ nhân quả giữa các vật trong ảnh. Mô hình ngôn ngữ có thể viết câu mạch lạc nhưng vẫn đưa ra chi tiết sai. Vì vậy tiến bộ nên được mô tả bằng năng lực cụ thể, điều kiện thử và giới hạn, thay vì một khẳng định chung về trí thông minh.

Sự đa dạng của phần cứng cũng định hình hướng nghiên cứu. Thuật toán có chi phí tính toán quá lớn trong một thời kỳ có thể trở nên khả thi khi bộ xử lý song song và bộ nhớ phát triển. Ngược lại, một mô hình rất lớn có thể không phù hợp thiết bị biên hoặc ứng dụng cần đáp ứng tức thời. Trong sản phẩm, lựa chọn một mô hình nhỏ, đủ chính xác và dễ giám sát đôi khi tốt hơn chạy mô hình đạt benchmark cao nhất. Lịch sử AI vì thế gắn liền với kinh tế của tính toán: dữ liệu nào có thể lưu, mô hình nào có thể huấn luyện và dự báo nào có thể phục vụ ở quy mô

thực tế.

Ngày nay, một mô hình thường chỉ là một thành phần trong hệ thống AI. Dữ liệu cần được thu thập và làm sạch; kết quả dự đoán cần được trình bày theo cách người sử dụng hiểu; lỗi đầu vào phải được xử lý; phiên bản mô hình phải gắn với đúng phiên bản bộ chuẩn hóa. Nếu một app tải trọng số CNN mới nhưng vẫn giữ mean và standard deviation của bản cũ, kết quả có thể sai dù chương trình chạy không báo lỗi. Câu chuyện này kết nối trực tiếp nghiên cứu thuật toán với triển khai phần mềm.

Một hệ thống sử dụng AI cần xác định tác động của hai loại lỗi. Trong phân loại nhận xét, bỏ sót review không hài lòng có thể làm mất cơ hội hỗ trợ khách hàng. Trong đánh giá nguy cơ bệnh, bỏ sót một ca dương có hậu quả khác với một cảnh báo sai. Trong giá nhà, sai số lớn ở tài sản đắt tiền làm tăng RMSE đáng kể. Vì thế không có một metric duy nhất đủ để đại diện cho mọi mục tiêu. Việc lựa chọn ngưỡng, hiển thị độ tin cậy và cơ chế con người kiểm tra là quyết định thiết kế gắn với bối cảnh sử dụng.

Lịch sử AI cũng cho thấy không nên đồng nhất sự trôi chảy của giao diện với trí tuệ đáng tin cậy. Hệ ký hiệu có thể giải thích từng luật nhưng thiếu linh hoạt; mạng học sâu linh hoạt nhưng có thể khó dự đoán khi đầu vào lệch khỏi phân bố huấn luyện. Nghiên cứu hiện đại quan tâm nhiều hơn đến đánh giá ngoài miền, theo dõi sau triển khai và hiệu chỉnh mô hình. Đối với sinh viên xây dựng một ứng dụng localhost, bài học thực tế là phải giữ mối liên hệ giữa dataset, checkpoint, bộ tiền xử lý và kết quả giao diện. Mỗi thành phần là một phần của lời giải.

Chương 2

Các kỹ thuật học máy cơ bản

2.1 Bài toán, quy trình và nguyên tắc so sánh

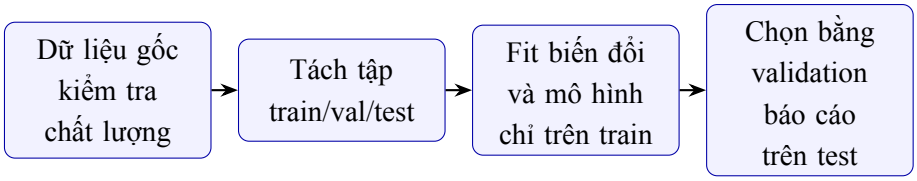
Assignment 03 gồm các bài toán rất khác nhau: phân loại nhận xét bằng văn bản, phát hiện tiểu đường bằng dữ liệu bảng và dự báo giá bất động sản bằng hồi quy. Điểm chung của chúng là một pipeline học có giám sát: khảo sát dữ liệu, tách tập, học bước biến đổi trên train, huấn luyện mô hình, chọn tham số trên validation và báo cáo kết quả trên test. Ba mô hình được chọn cho từng dataset phản ánh ba mức độ giả định khác nhau, từ tuyến tính hoặc xác suất đơn giản đến mạng MLP và mô hình cây phi tuyến.

$$D = D_{\text{train}} \cup D_{\text{val}} \cup D_{\text{test}}, \qquad D_i \cap D_j = \varnothing \quad (i \neq j). \tag{2.1}$$

Tách dữ liệu trước khi fit bộ chuẩn hóa, bộ mã hóa hay TF-IDF ngăn thông tin của test len vào quá trình học. Đây là điều kiện cơ bản để con số đánh giá mang ý nghĩa dự báo trên dữ liệu mới. Với lớp mất cân bằng, chia phân tầng giữ tỷ lệ nhãn gần nhau giữa các tập. Với dữ liệu giá nhà, mọi xử lý log target phải được đảo lại trước khi tính sai số bằng đơn vị tiền tệ.

Ký hiệu D chỉ toàn bộ các cặp đầu vào–đáp án; ba tập con trong công thức đôi một không giao nhau. Train dùng để ước lượng trọng số, validation dùng để chọn siêu tham số và ngưỡng, còn test đóng vai trò phép đo cuối. Điều kiện không giao nhau áp dụng cho *đơn vị quan sát độc lập*: nếu nhiều dòng thuộc cùng một khách hàng, hộ gia đình hoặc chuỗi thời gian, tách ngẫu nhiên từng dòng có thể vẫn làm lộ thông tin giữa các tập. Với văn bản, ngay cả từ vựng và trọng số IDF cũng phải học từ train; với bảng, mean, độ lệch chuẩn và cách điền giá trị thiếu cũng phải được fit trong phần train của từng lần đánh giá.

Hình 2.1 biểu diễn luồng đánh giá. Một lần chạy thử có thể dùng validation nhiều lần để so sánh cấu hình, nhưng số test nên được công bố sau khi chốt pipeline. Ví dụ, nếu chọn ngưỡng phân loại diabetes bằng cách tối đa hóa F_1 trên test, cùng tập test đó không còn cho ước lượng độc lập của F_1 nữa. Kiểm định chéo là một cách tận dụng tập train cho lựa chọn mô hình khi dữ liệu ít; nó không làm thay đổi vai trò cuối cùng của test.



Hình 2.1: Luồng học có giám sát và vị trí cần ngăn rò rỉ dữ liệu.

2.1.1 Các họ mô hình cơ bản

Logistic Regression ước lượng xác suất lớp dương bằng $p(y = 1 \mid x) = \sigma(w^\top x + b)$, với $\sigma(z) = 1/(1+e^{-z})$. Ranh giới quyết định của mô hình là tuyến tính trong không gian đặc trưng đã mã hóa. Vì vậy nó dễ huấn luyện, dễ giải thích nhưng có thể bỏ lỡ tương tác phi tuyến. Naive Bayes dùng định lý Bayes và giả định các đặc trưng độc lập có điều kiện theo lớp; giả định ấy thường không hoàn toàn đúng, song mô hình đặc biệt hiệu quả với văn bản thưa và số chiều cao.

Trong Logistic Regression, x là vector đặc trưng, w là trọng số, b là bias và $w^\top x + b$ là log-odds trước khi qua sigmoid. Giá trị sigmoid nằm giữa 0 và 1; với ngưỡng 0,5, mô hình chọn lớp dương khi $w^\top x + b \geq 0$. Trên dữ liệu mất cân bằng, ngưỡng 0,5 chỉ là quy tắc mặc định, không phải ngưỡng luôn phù hợp. Ví dụ nếu mục tiêu là giảm âm tính giả, ngưỡng thấp hơn có thể tăng recall nhưng thường cũng tăng dương tính giả. Hệ số w_j diễn tả thay đổi log-odds khi đặc trưng thứ j tăng một đơn vị trong khi các đặc trưng khác giữ nguyên; diễn giải này phụ thuộc cách mã hóa và chuẩn hóa dữ liệu.

Naive Bayes chọn lớp theo $\hat{y} = \arg\max_y p(y) \prod_j p(x_j \mid y)$. $p(y)$ là xác suất tiên nghiệm của lớp, còn $p(x_j \mid y)$ đo mức phù hợp của đặc trưng thứ j với lớp ấy. Tích các xác suất nhỏ dễ gây tràn số xuống 0, nên triển khai thường cộng log xác suất. Với nhận xét mua sắm, những từ hoặc n-gram gắn mạnh với lớp có thể tạo tín hiệu hữu ích dù giả định độc lập giữa từ không hoàn toàn đúng. Xác suất đầu ra của Naive Bayes có thể kém hiệu chỉnh; chất lượng xếp hạng và chất lượng xác suất cần được đánh giá riêng.

Decision Tree tạo các phép chia tuần tự theo đặc trưng, hình thành những quy tắc kiểu ngưỡng. Random Forest lấy trung bình nhiều cây huấn luyện trên các mẫu và tập đặc trưng khác nhau để giảm phương sai. MLP học phép biến đổi $a^{(l)} = \phi(W^{(l)} a^{(l-1)} + b^{(l)})$ qua nhiều tầng. Nếu không có phi tuyến, nhiều tầng tuyến tính chỉ tương đương một phép biến đổi tuyến tính; vì vậy ReLU và các hàm kích hoạt khác là thành phần cốt lõi. MLP trong Assignment 03 được hiện thực bằng NumPy, giúp quan sát trực tiếp forward, backward, mini-batch, điều chuẩn và early stopping.

Một cây chọn phép chia nhằm giảm độ hỗn tạp của nhãn. Với xác suất lớp p_c trong một node, độ hỗn tạp Gini là $G = 1 - \sum_c p_c^2$: node chỉ có một lớp cho $G = 0$, còn node gồm hai lớp cân bằng cho $G = 0{,}5$. Cây sâu biểu diễn tương tác phi tuyến và ngưỡng dễ đọc nhưng có phương sai cao: thay vài dòng dữ liệu có thể đổi cấu trúc cây. Random Forest huấn luyện nhiều cây trên mẫu bootstrap và thường chỉ xét một tập con đặc trưng ở mỗi phép chia; dự báo phân loại bằng bỏ phiếu hoặc trung bình xác suất. Trung bình nhiều cây giảm độ nhạy của dự đoán với một mẫu train cụ thể, đổi lại khó diễn giải toàn bộ mô hình bằng một đường luật duy nhất.

Trong công thức MLP, $a^{(l-1)}$ là vector kích hoạt đi vào tầng l, $W^{(l)}$ và $b^{(l)}$ là tham số,

còn ϕ là hàm phi tuyến. Forward pass tính đầu ra và loss; backward pass dùng quy tắc dây chuyền để tính $\partial \mathcal{L}/\partial W^{(l)}$ từ tầng cuối về tầng đầu. Với mini-batch B, cập nhật gradient cơ bản là $W \leftarrow W - \eta \frac{1}{|B|} \sum_{i \in B} \nabla_W \mathcal{L}_i$, trong đó η là learning rate. Nếu η quá lớn, loss có thể dao động; nếu quá nhỏ, quá trình học chậm. Early stopping theo validation ngăn việc tiếp tục giảm lỗi train khi lỗi trên dữ liệu chưa thấy bắt đầu tăng.

2.1.2 Metric theo bản chất nhiệm vụ

Với phân loại, accuracy đo tỷ lệ đúng chung nhưng dễ gây hiểu nhầm khi nhãn lệch. Precision trả lời trong các dự đoán dương có bao nhiêu dương thật; recall trả lời trong các dương thật đã tìm ra bao nhiêu. $F_1 = 2PR/(P+R)$ cân bằng hai đại lượng. ROC-AUC và PR-AUC đo khả năng xếp hạng qua nhiều ngưỡng; PR-AUC đặc biệt hữu ích khi lớp dương hiếm. Với hồi quy, MAE diễn giải trực tiếp thành mức sai số trung bình, RMSE phạt mạnh ngoại lệ, RMSLE phản ánh sai số tỷ lệ ở thang log, còn R^2 so với dự đoán trung bình. Không nên so sánh trực tiếp giá trị metric của hai dataset khác đơn vị hoặc khác phân bố.

Gọi TP là dương thật được dự báo dương, FP là âm thật bị dự báo dương, FN là dương thật bị bỏ sót và TN là âm thật được dự báo âm. Khi đó

$$\text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}, \qquad P = \frac{TP}{TP+FP}, \qquad R = \frac{TP}{TP+FN}. \tag{2.2}$$

F_1 là trung bình điều hòa của P và R, nên giảm mạnh nếu một trong hai gần 0. Với 8.500 nhãn dương trong 100.000 dòng diabetes, mô hình luôn đoán âm đạt 91,5% accuracy nhưng có $R = 0$; đây là ví dụ cho thấy accuracy đơn lẻ che giấu lỗi bỏ sót. ROC vẽ true-positive rate theo false-positive rate khi thay ngưỡng, còn đường precision–recall nhấn mạnh chất lượng dự báo dương. AUC là diện tích dưới từng đường, không phải accuracy tại một ngưỡng cụ thể.

Với hồi quy, đặt $e_i = y_i - \hat{y}_i$ và $\bar{y}$ là trung bình nhãn thật trên tập được đánh giá. Các phép đo chính có dạng

$$\text{MAE} = \frac{1}{n}\sum_i |e_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum_i e_i^2}, \quad R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2}. \tag{2.3}$$

MAE và RMSE có cùng đơn vị với target, ví dụ USD cho giá nhà. RMSE lớn hơn khi một vài lỗi rất lớn vì sai số được bình phương trước khi lấy trung bình. $R^2 = 0$ tương ứng với dự báo bằng $\bar{y}$ trên chính tập đánh giá; R^2 có thể âm nếu mô hình còn tệ hơn mốc ấy. Với target không âm, $\text{RMSLE} = \sqrt{\frac{1}{n}\sum_i [\log(1+\hat{y}_i) - \log(1+y_i)]^2}$ đo chênh lệch trên thang log, nhấn mạnh sai số tương đối ở các mức giá khác nhau. Nó không dùng được nếu giá trị cần lấy log nhỏ hơn -1 và không thay thế báo cáo lỗi theo USD.

2.2 Customer comments: phân loại nhận xét mua sắm

Mã nguồn GitHub: https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2003/customer_comments

Bảng 2.1: Ba dataset được dùng trong Chương 2

Dataset	Dữ liệu gốc	Dùng trong notebook	Đầu ra
Customer comments	23.486 review	22.642 review hợp lệ	`Recommended IND`: 0/1
Diabetes large	100.000 người	100.000 người	`diabetes`: 0/1
House price	2.226.382 căn	mẫu 200.000 căn	`price`: giá USD

Nguồn dữ liệu: https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews

Tệp `Womens Clothing E-Commerce Reviews.csv` có 23.486 dòng. Notebook giữ 22.642 nhận xét hợp lệ sau bước làm sạch văn bản. Cột đầu vào chính là `Review Text`; nhãn là `Recommended IND`. Một dòng mẫu có nội dung "Absolutely wonderful - silky and sexy and comfortable", điểm đánh giá 4 và nhãn khuyến nghị bằng 1. Nội dung review có thể dài, rất ngắn hoặc trống; vì thế số dòng thật sự đi vào benchmark nhỏ hơn số dòng CSV. Trong dữ liệu gốc có 19.314 nhãn 1 và 4.172 nhãn 0, tương ứng khoảng 82,2% và 17,8%; tỷ lệ sau lọc có thể thay đổi nhẹ.

Cell 11 — EDA (notebook gốc).

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()

label_counts = df_model["label"].value_counts().sort_index()
axes[0].bar(["Không recommend", "Recommend"], label_counts.values,
↪  color=["#E45756", "#4C78A8"])
axes[0].set_title("Phân phối nhãn Recommended IND")
axes[0].set_ylabel("Số review")

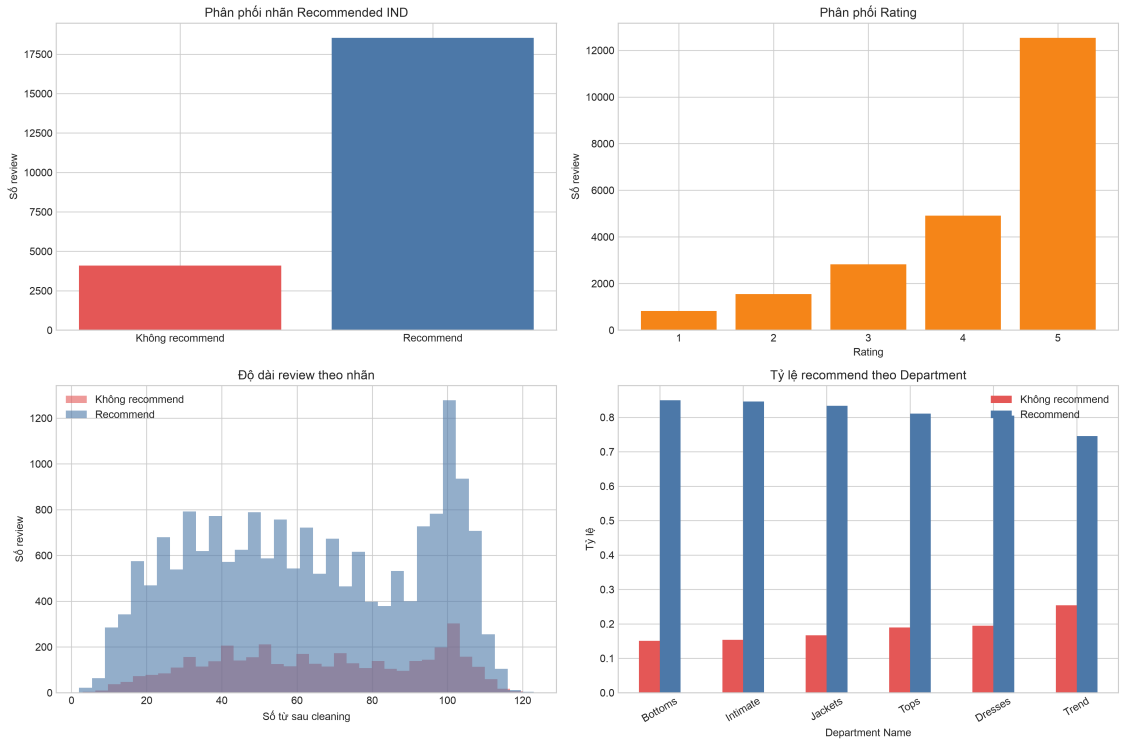
rating_counts = df_model["Rating"].value_counts().sort_index()
axes[1].bar(rating_counts.index.astype(str), rating_counts.values,
↪  color="#F58518")
axes[1].set_title("Phân phối Rating")
axes[1].set_xlabel("Rating")
axes[1].set_ylabel("Số review")

for label, name, color in [(0, "Không recommend", "#E45756"), (1,
↪  "Recommend", "#4C78A8")]:
    subset = df_model.loc[df_model["label"] == label, "word_count"]
    axes[2].hist(subset, bins=35, alpha=0.6, label=name, color=color)
axes[2].set_title("Độ dài review theo nhãn")
axes[2].set_xlabel("Số từ sau cleaning")
axes[2].set_ylabel("Số review")
axes[2].legend()
```

```
department_label_rate = pd.crosstab(df_model["Department Name"],
↪  df_model["label"], normalize="index")
department_label_rate = department_label_rate.sort_values(1,
↪  ascending=False)
department_label_rate.plot(kind="bar", ax=axes[3], color=["#E45756",
↪  "#4C78A8"])
axes[3].set_title("Tỷ lệ recommend theo Department")
axes[3].set_xlabel("Department Name")
axes[3].set_ylabel("Tỷ lệ")
axes[3].legend(["Không recommend", "Recommend"])
axes[3].tick_params(axis="x", rotation=30)

plt.tight_layout()
save_current_figure("customer_comments_eda_overview.png")
plt.show()
```

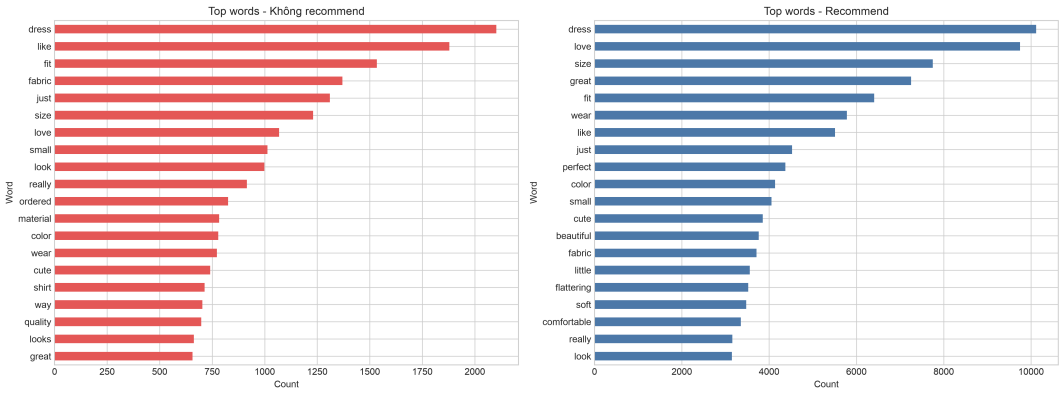
Output hình của cell: Hình 2.2.



Hình 2.2: Phân bố nhãn, độ dài nhận xét và các thống kê mô tả của customer comments.

Văn bản được chuẩn hóa rồi chuyển sang TF-IDF tối đa 1.000 chiều. Với từ t trong tài liệu d, TF-IDF tăng theo tần suất của từ trong d nhưng giảm nếu từ xuất hiện ở nhiều tài liệu. Bộ từ vựng phải được fit trên train; validation và test chỉ được biến đổi bằng bộ từ vựng đã học. Do ma trận này thưa và có nhiều chiều, Multinomial Naive Bayes là baseline phù hợp. Logistic Regression là baseline phân biệt tuyến tính, còn MLP NumPy học thêm tương tác phi tuyến của các đặc trưng văn bản.

Cell 21 — huấn luyện baseline và đánh giá (notebook gốc).



Hình 2.3: Các từ xuất hiện nhiều theo lớp nhãn; tần suất từ chỉ mô tả dữ liệu, chưa chứng minh giá trị dự báo.

```
X_train_dense = X_train_tfidf.toarray().astype(np.float32)
X_test_dense = X_test_tfidf.toarray().astype(np.float32)

baseline_models = {
    "Logistic Regression": {
        "model": LogisticRegression(
            max_iter=1000,
            class_weight="balanced",
            random_state=RANDOM_STATE,
        ),
        "train_X": X_train_tfidf,
        "test_X": X_test_tfidf,
    },
    "Multinomial Naive Bayes": {
        "model": MultinomialNB(alpha=1.0),
        "train_X": X_train_tfidf,
        "test_X": X_test_tfidf,
    },
    "Random Forest": {
        "model": RandomForestClassifier(
            n_estimators=100,
            max_depth=24,
            min_samples_leaf=2,
            class_weight="balanced_subsample",
            n_jobs=-1,
            random_state=RANDOM_STATE,
        ),
        "train_X": X_train_dense,
        "test_X": X_test_dense,
    },
}

benchmark_results = []
test_predictions = {}
```

```
for model_name, config in baseline_models.items():
    model = config["model"]
    train_X = config["train_X"]
    test_X = config["test_X"]

    print(f"Training {model_name}...")
    start = time.perf_counter()
    model.fit(train_X, y_train)
    train_time = time.perf_counter() - start

    start = time.perf_counter()
    y_prob = positive_probability(model, test_X)
    predict_time = time.perf_counter() - start

    result = evaluate_classifier(model_name, y_test, y_prob, train_time,
    ↪  predict_time)
    benchmark_results.append(result)
    test_predictions[model_name] = {
        "probability": y_prob,
        "prediction": (y_prob >= 0.5).astype(int),
    }

    print(f"  Test F1-Score: {result['F1-Score']:.4f}, ROC-AUC:
    ↪  {result['ROC-AUC']:.4f}")

baseline_results_df = display_classification_results(benchmark_results)
```

Output của cell:

```
Training Logistic Regression...
  Test F1-Score: 0.9156, ROC-AUC: 0.9348
Training Multinomial Naive Bayes...
  Test F1-Score: 0.9212, ROC-AUC: 0.9299
Training Random Forest...
  Test F1-Score: 0.9074, ROC-AUC: 0.9016

index=0; Model=Multinomial Naive Bayes; Accuracy=0.8618; Precision=0.8643;
↪  Recall=0.9860; F1-Score=0.9212
  ROC-AUC=0.9299; Train time (s)=0.002; Predict time (s)=0.001
index=1; Model=Logistic Regression; Accuracy=0.8686; Precision=0.9659;
↪  Recall=0.8703; F1-Score=0.9156
  ROC-AUC=0.9348; Train time (s)=0.024; Predict time (s)=0.000
index=2; Model=Random Forest; Accuracy=0.8527; Precision=0.9353;
↪  Recall=0.8811; F1-Score=0.9074
  ROC-AUC=0.9016; Train time (s)=1.341; Predict time (s)=0.026
```

Cell 25 — huấn luyện MLP NumPy (notebook gốc).

```
mlp = NumpyMLPTextClassifier(
    hidden_layers=(64, 16),
```

```
    learning_rate=0.05,
    epochs=20,
    batch_size=256,
    l2=1e-4,
    random_state=RANDOM_STATE,
    verbose=True,
)

start = time.perf_counter()
mlp.fit(X_train_dense, y_train)
mlp_train_time = time.perf_counter() - start

start = time.perf_counter()
mlp_y_prob = mlp.predict_proba(X_test_dense)
mlp_predict_time = time.perf_counter() - start

mlp_result = evaluate_classifier(
    "NumPy MLP (1000-64-16-1)",
    y_test,
    mlp_y_prob,
    mlp_train_time,
    mlp_predict_time,
)

benchmark_results.append(mlp_result)
test_predictions["NumPy MLP (1000-64-16-1)"] = {
    "probability": mlp_y_prob,
    "prediction": (mlp_y_prob >= 0.5).astype(int),
}

print(f"Test F1-Score: {mlp_result['F1-Score']:.4f}, ROC-AUC:
↪ {mlp_result['ROC-AUC']:.4f}")
```

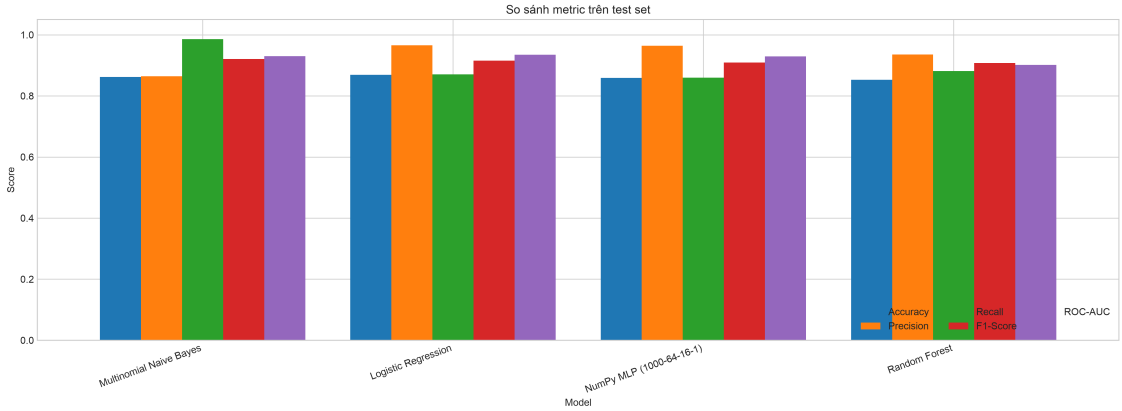
Output của cell:

```
Epoch 001 | loss=0.7007 | train_acc=0.5058 | train_f1=0.6133
Epoch 005 | loss=0.6911 | train_acc=0.6987 | train_f1=0.7913
Epoch 010 | loss=0.6177 | train_acc=0.7787 | train_f1=0.8482
Epoch 015 | loss=0.4049 | train_acc=0.8577 | train_f1=0.9079
Epoch 020 | loss=0.3274 | train_acc=0.8755 | train_f1=0.9202
Test F1-Score: 0.9091, ROC-AUC: 0.9297
```

Bảng 2.2: Ba mô hình được chọn trên customer comments (test set)

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Train (s)
Multinomial Naive Bayes	0.8618	0.8643	**0.9860**	**0.9212**	0.9299	**0.0015**
Logistic Regression	**0.8686**	**0.9659**	0.8703	0.9156	**0.9348**	0.0244
NumPy MLP	0.8591	0.9640	0.8601	0.9091	0.9297	0.6276

Naive Bayes đạt F_1 và recall cao nhất trong ba mô hình, nhưng precision thấp hơn Logistic Regression và MLP. Điều này có nghĩa nó gần như không bỏ sót review khuyến nghị, đổi lại nhận nhầm thêm một phần review không khuyến nghị. Nếu hệ thống chỉ muốn hiển thị review thật sự tích cực, Logistic Regression có thể thích hợp hơn vì precision đạt 0.9659. MLP không vượt baseline tuyến tính trong lần chạy này, cho thấy độ phức tạp lớn hơn không mặc nhiên đem lại chất lượng cao hơn trên TF-IDF 1.000 chiều.



Hình 2.4: So sánh các metric phân loại customer comments trong notebook gốc; biểu đồ gốc có thêm Random Forest ngoài ba mô hình được chọn.

Ma trận nhầm lẫn nên được đọc theo từng hàng nhãn thật vì lớp khuyến nghị đông hơn nhiều. Độ chính xác tổng thể gần 0.86 có thể che khuất lỗi trên lớp không khuyến nghị. Trong vận hành, nhận diện review không hài lòng thường có giá trị riêng cho chăm sóc khách hàng; vì vậy cần cân nhắc ngưỡng xác suất và mục tiêu nghiệp vụ, thay vì chỉ chọn một con số F_1.

2.3 Diabetes large: phân loại nguy cơ tiểu đường

Mã nguồn GitHub: `https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2003/diabetes_large`

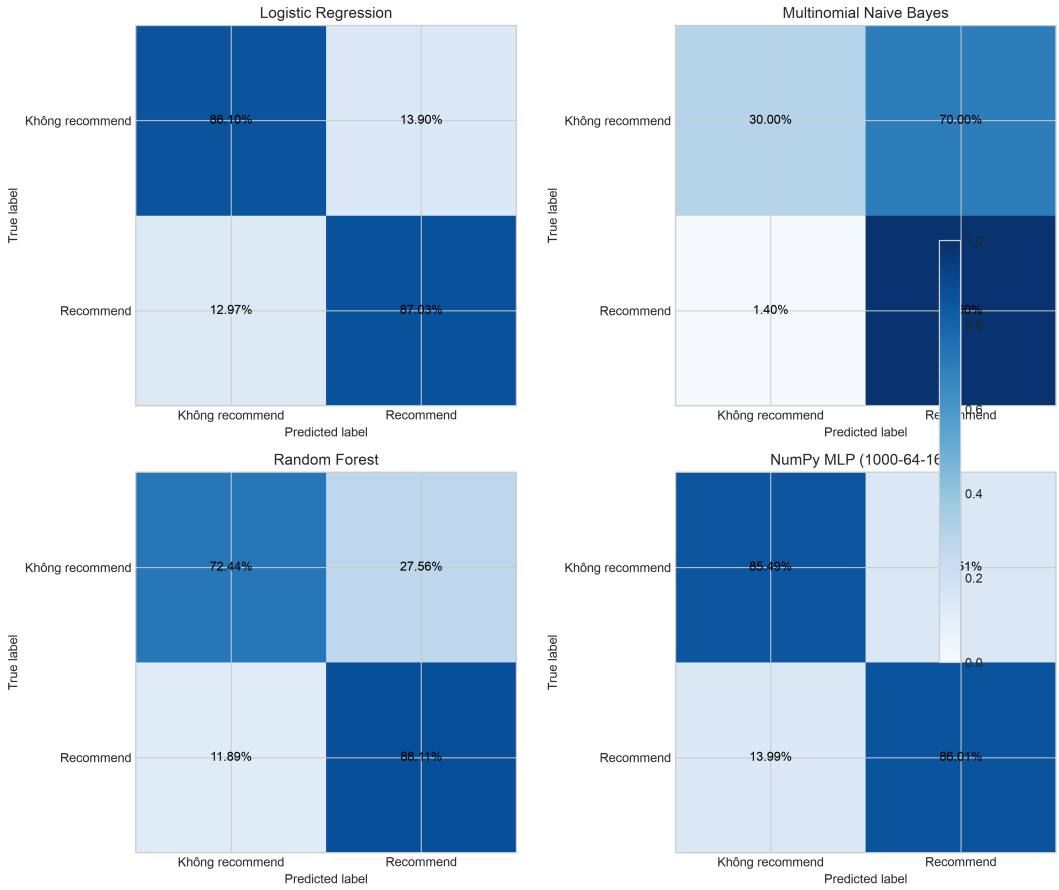
Nguồn dữ liệu: `https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset`

Tệp `diabetes_prediction_dataset.csv` gồm 100.000 dòng và chín cột: tám biến đầu vào cùng nhãn `diabetes`. Các đặc trưng bao gồm tuổi, BMI, tình trạng tăng huyết áp, bệnh tim, tiền sử hút thuốc, HbA1c và đường huyết. Một mẫu đầu tiên có giới tính nữ, tuổi 80, BMI 25.19, HbA1c 6.6, đường huyết 140 và nhãn 0. Có 8.500 dòng dương (8,5%) và 91.500 dòng âm (91,5%), tạo ra chênh lệch lớp rất lớn.

Cell 9 — EDA (notebook gốc).

```
fig, axes = plt.subplots(2, 3, figsize=(16, 9))
axes = axes.ravel()

target_counts = df["diabetes"].value_counts().sort_index()
```



Hình 2.5: Ma trận nhầm lẫn chuẩn hóa của benchmark gốc; đối chiếu ba mô hình trong Bảng 2.2.



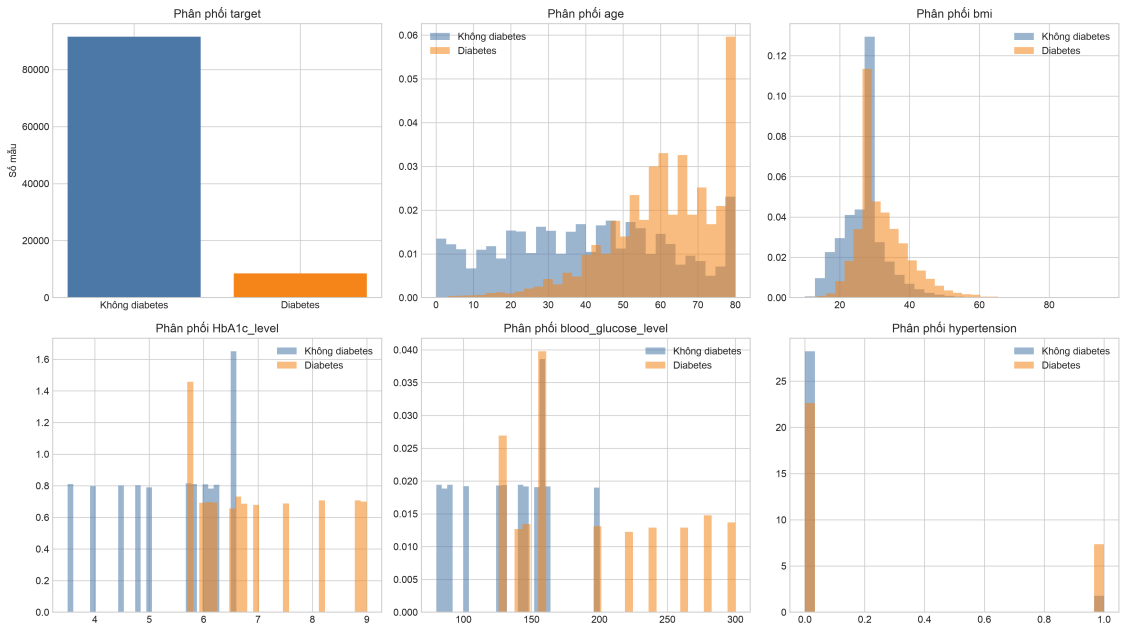
Hình 2.6: Giao diện deploy customer comments với nhận xét đầu vào, mô hình và kết quả dự đoán.

```
axes[0].bar(["Không diabetes", "Diabetes"], target_counts.values,
 ↪  color=["#4C78A8", "#F58518"])
axes[0].set_title("Phân phối target")
axes[0].set_ylabel("Số mẫu")

numeric_for_plot = ["age", "bmi", "HbA1c_level", "blood_glucose_level",
 ↪  "hypertension"]
for ax, col in zip(axes[1:], numeric_for_plot):
    for target_value, label, color in [(0, "Không diabetes", "#4C78A8"),
     ↪  (1, "Diabetes", "#F58518")]:
        ax.hist(
            df.loc[df["diabetes"] == target_value, col],
            bins=30,
            alpha=0.55,
            density=True,
            label=label,
            color=color,
        )
    ax.set_title(f"Phân phối {col}")
    ax.legend()

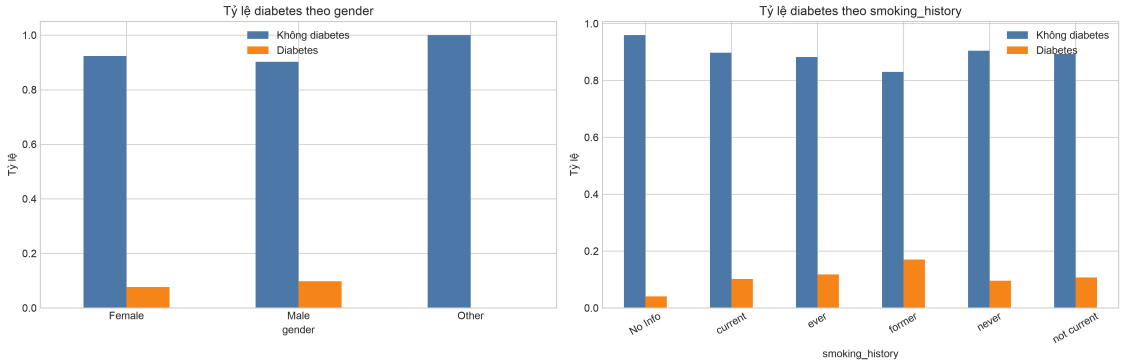
plt.tight_layout()
save_current_figure("diabetes_large_eda_numeric_distributions.png")
plt.show()
```

Output hình của cell: Hình 2.7.



Hình 2.7: Phân phối biến số trên diabetes large theo nhãn.

Notebook one-hot encode các biến phân loại, chuẩn hóa biến số và chia train, validation, test theo tỷ lệ 70/15/15 có phân tầng. Các bước có trạng thái chỉ được fit trên train. Logistic Regression và Decision Tree dùng trọng số lớp cân bằng, trong khi MLP NumPy áp trọng số



Hình 2.8: Tỷ lệ tiểu đường theo các biến phân loại trong dữ liệu khảo sát.

trong loss; cách này hướng quá trình tối ưu quan tâm hơn tới lớp dương hiếm. Threshold dự đoán được chọn trên validation thay vì cố định 0.5. Sau đó test set độc lập mới được dùng để báo cáo kết quả.

Cell 18 — huấn luyện baseline và chọn ngưỡng (notebook gốc).

```
baseline_models = {
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        class_weight="balanced",
        random_state=RANDOM_STATE,
    ),
    "Gaussian Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=8,
        min_samples_leaf=50,
        class_weight="balanced",
        random_state=RANDOM_STATE,
    ),
}

benchmark_results = []
test_predictions = {}
validation_predictions = {}

for model_name, model in baseline_models.items():
    print(f"Training {model_name}...")

    start = time.perf_counter()
    model.fit(X_train, y_train)
    train_time = time.perf_counter() - start

    val_prob = predict_positive_probability(model, X_val)
    best_threshold, best_val_f1 = find_best_threshold(y_val, val_prob)

    start = time.perf_counter()
    test_prob = predict_positive_probability(model, X_test)
    predict_time = time.perf_counter() - start
```

```
    result = evaluate_binary_model(
        model_name,
        y_test,
        test_prob,
        best_threshold,
        train_time,
        predict_time,
    )
    benchmark_results.append(result)
    validation_predictions[model_name] = val_prob
    test_predictions[model_name] = {
        "probability": test_prob,
        "prediction": (test_prob >= best_threshold).astype(int),
        "threshold": best_threshold,
    }

    print(f"  Best validation F1: {best_val_f1:.4f} tại
    ↪  threshold={best_threshold:.3f}")
    print(f"  Test F1: {result['F1']:.4f}, PR-AUC:
    ↪  {result['PR-AUC']:.4f}")

baseline_results_df = display_results(benchmark_results)
```

Output của cell:

```
Training Logistic Regression...
  Best validation F1: 0.7311 tại threshold=0.875
  Test F1: 0.7300, PR-AUC: 0.8209
Training Gaussian Naive Bayes...
  Best validation F1: 0.4673 tại threshold=0.950
  Test F1: 0.4507, PR-AUC: 0.5043
Training Decision Tree...
  Best validation F1: 0.8140 tại threshold=0.890
  Test F1: 0.8039, PR-AUC: 0.8514

index=0; Model=Decision Tree; Threshold=0.890; Accuracy=0.9721;
↪  Precision=1.0000; Recall=0.6722
  F1=0.8039; ROC-AUC=0.9731; PR-AUC=0.8514; Train time (s)=0.071; Predict
  ↪  time (s)=0.000
index=1; Model=Logistic Regression; Threshold=0.875; Accuracy=0.9573;
↪  Precision=0.7900; Recall=0.6784
  F1=0.7300; ROC-AUC=0.9629; PR-AUC=0.8209; Train time (s)=0.032; Predict
  ↪  time (s)=0.000
index=2; Model=Gaussian Naive Bayes; Threshold=0.950; Accuracy=0.8265;
↪  Precision=0.3083; Recall=0.8376
  F1=0.4507; ROC-AUC=0.9093; PR-AUC=0.5043; Train time (s)=0.010; Predict
  ↪  time (s)=0.003
```

Cell 22 — huấn luyện MLP NumPy (notebook gốc).

```
mlp = NumpyMLPClassifier(
    hidden_layers=(64, 32, 16),
    learning_rate=0.01,
    epochs=80,
    batch_size=1024,
    l2=1e-4,
    patience=10,
    min_delta=1e-4,
    random_state=RANDOM_STATE,
    verbose=True,
)

start = time.perf_counter()
mlp.fit(X_train, y_train, X_val, y_val)
mlp_train_time = time.perf_counter() - start

mlp_val_prob = mlp.predict_proba(X_val)
mlp_threshold, mlp_val_f1 = find_best_threshold(y_val, mlp_val_prob)

start = time.perf_counter()
mlp_test_prob = mlp.predict_proba(X_test)
mlp_predict_time = time.perf_counter() - start

mlp_result = evaluate_binary_model(
    "NumPy MLP (64-32-16)",
    y_test,
    mlp_test_prob,
    mlp_threshold,
    mlp_train_time,
    mlp_predict_time,
)

benchmark_results.append(mlp_result)
validation_predictions["NumPy MLP (64-32-16)"] = mlp_val_prob
test_predictions["NumPy MLP (64-32-16)"] = {
    "probability": mlp_test_prob,
    "prediction": (mlp_test_prob >= mlp_threshold).astype(int),
    "threshold": mlp_threshold,
}

print(f"Best epoch: {mlp.best_epoch_}")
print(f"Best validation PR-AUC: {mlp.best_validation_pr_auc_:.4f}")
print(f"Best validation F1: {mlp_val_f1:.4f} tại
 ↪  threshold={mlp_threshold:.3f}")
print(f"Test F1: {mlp_result['F1']:.4f}, PR-AUC:
 ↪  {mlp_result['PR-AUC']:.4f}")
```

Output của cell:

```
Epoch 001 | train_loss=0.4545 | val_loss=0.4528 | val_f1=0.5466 |
 ↪  val_pr_auc=0.5497
```

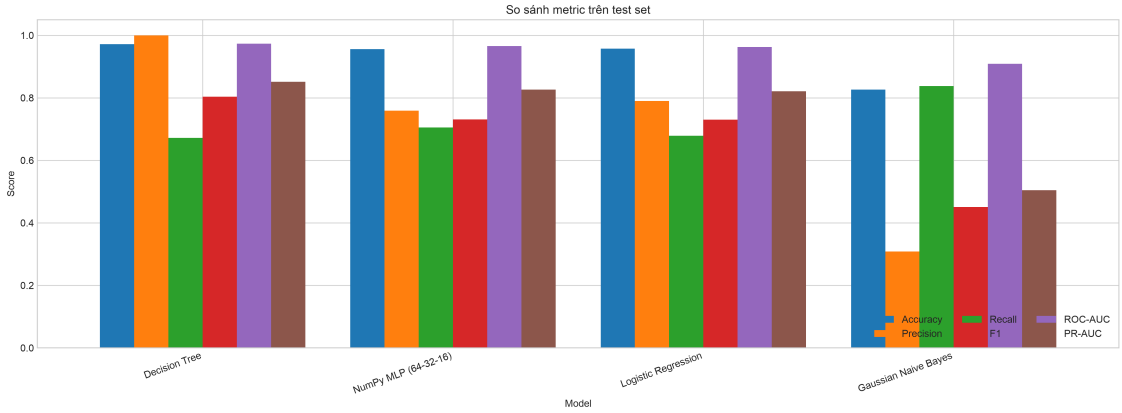
```
Epoch 005 | train_loss=0.2812 | val_loss=0.2779 | val_f1=0.6764 |
↪  val_pr_auc=0.7671
Epoch 010 | train_loss=0.2614 | val_loss=0.2585 | val_f1=0.6977 |
↪  val_pr_auc=0.7901
Epoch 015 | train_loss=0.2558 | val_loss=0.2527 | val_f1=0.7073 |
↪  val_pr_auc=0.7994
Epoch 020 | train_loss=0.2524 | val_loss=0.2503 | val_f1=0.7115 |
↪  val_pr_auc=0.8027
Epoch 025 | train_loss=0.2502 | val_loss=0.2477 | val_f1=0.7162 |
↪  val_pr_auc=0.8083
Epoch 030 | train_loss=0.2480 | val_loss=0.2463 | val_f1=0.7210 |
↪  val_pr_auc=0.8101
Epoch 035 | train_loss=0.2465 | val_loss=0.2453 | val_f1=0.7233 |
↪  val_pr_auc=0.8117
Epoch 040 | train_loss=0.2448 | val_loss=0.2437 | val_f1=0.7273 |
↪  val_pr_auc=0.8144
Epoch 045 | train_loss=0.2433 | val_loss=0.2422 | val_f1=0.7282 |
↪  val_pr_auc=0.8166
Epoch 050 | train_loss=0.2425 | val_loss=0.2420 | val_f1=0.7307 |
↪  val_pr_auc=0.8165
Epoch 055 | train_loss=0.2409 | val_loss=0.2402 | val_f1=0.7315 |
↪  val_pr_auc=0.8189
Epoch 060 | train_loss=0.2398 | val_loss=0.2390 | val_f1=0.7327 |
↪  val_pr_auc=0.8208
Epoch 065 | train_loss=0.2387 | val_loss=0.2382 | val_f1=0.7343 |
↪  val_pr_auc=0.8215
Epoch 070 | train_loss=0.2377 | val_loss=0.2374 | val_f1=0.7363 |
↪  val_pr_auc=0.8225
Epoch 075 | train_loss=0.2367 | val_loss=0.2367 | val_f1=0.7368 |
↪  val_pr_auc=0.8227
Epoch 080 | train_loss=0.2359 | val_loss=0.2357 | val_f1=0.7367 |
↪  val_pr_auc=0.8246
Best epoch: 80
Best validation PR-AUC: 0.8246
Best validation F1: 0.7367 tại threshold=0.845
Test F1: 0.7312, PR-AUC: 0.8265
```

Bảng 2.3: Ba mô hình được chọn trên diabetes large (test set)

Mô hình	Ngưỡng	Precision	Recall	F1	ROC-AUC	PR-AUC	Train (s)
Decision Tree	0.890	**1.0000**	0.6722	**0.8039**	**0.9731**	**0.8514**	0.0709
NumPy MLP (64-32-16)	0.845	0.7593	**0.7051**	0.7312	0.9658	0.8265	18.7019
Logistic Regression	0.875	0.7900	0.6784	0.7300	0.9629	0.8209	**0.0318**

Decision Tree dẫn đầu theo F_1 và PR-AUC. Precision bằng 1.0000 tại ngưỡng 0.890 là kết quả trên tập test cụ thể, không phải cam kết rằng ngoài thực tế sẽ không có dương tính giả. Recall 0.6722 cho thấy khoảng một phần ba ca dương chưa được phát hiện. MLP có recall cao hơn trong ba mô hình nhưng precision thấp hơn; đây là đánh đổi quan trọng nếu mục đích là sàng lọc. Logistic Regression huấn luyện nhanh nhất và có F_1 sát MLP, nên là baseline khó bỏ

qua.



Hình 2.9: Biểu đồ metric diabetes large của notebook gốc, gồm thêm Gaussian Naive Bayes; bảng trong tiểu luận chỉ so sánh ba mô hình đã chọn.

Về mặt ứng dụng, chỉ số y sinh nhạy cảm với khác biệt giữa quần thể khảo sát và quần thể triển khai. Dataset này dùng cho bài thực nghiệm thuật toán, không phải cơ sở để đưa ra chẩn đoán y khoa. Khi thay đổi tỷ lệ dương ngoài thực tế, precision có thể đổi mạnh dù ROC-AUC gần như giữ nguyên. Cần hiệu chỉnh ngưỡng và kiểm định ngoài mẫu nếu muốn sử dụng trong bối cảnh mới.

2.4 House price: hồi quy giá bất động sản

Mã nguồn GitHub: `https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2003/house_price`

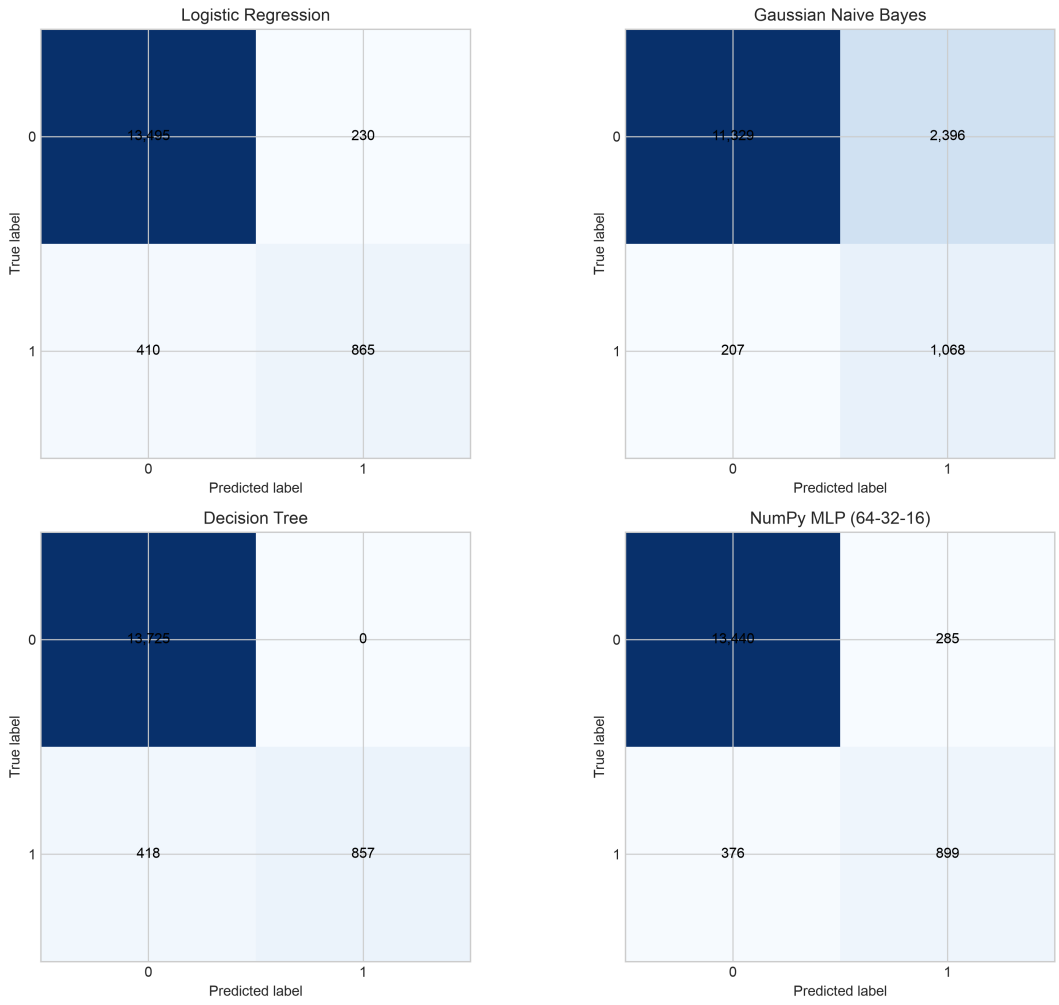
Nguồn dữ liệu: `https://www.kaggle.com/datasets/ahmedshahriarsakib/usa-real-estate-dataset`

Tệp `realtor-data.zip.csv` có 2.226.382 dòng và 12 cột. Notebook lấy mẫu 200.000 dòng để giảm chi phí xử lý và huấn luyện. Target là `price`; các đặc trưng gồm số phòng ngủ, phòng tắm, diện tích nhà, diện tích đất, trạng thái, thành phố, bang, mã ZIP và ngày bán trước. Một dòng mẫu đầu có trạng thái `for_sale`, giá 105.000 USD, ba phòng ngủ, hai phòng tắm, diện tích 920 foot vuông ở Puerto Rico. Trong CSV gốc, khoảng 1,389 triệu dòng có trạng thái `for_sale`, 812 nghìn `sold` và 25 nghìn `ready_to_build`; các nhóm này có thể có ý nghĩa khác nhau đối với giá.

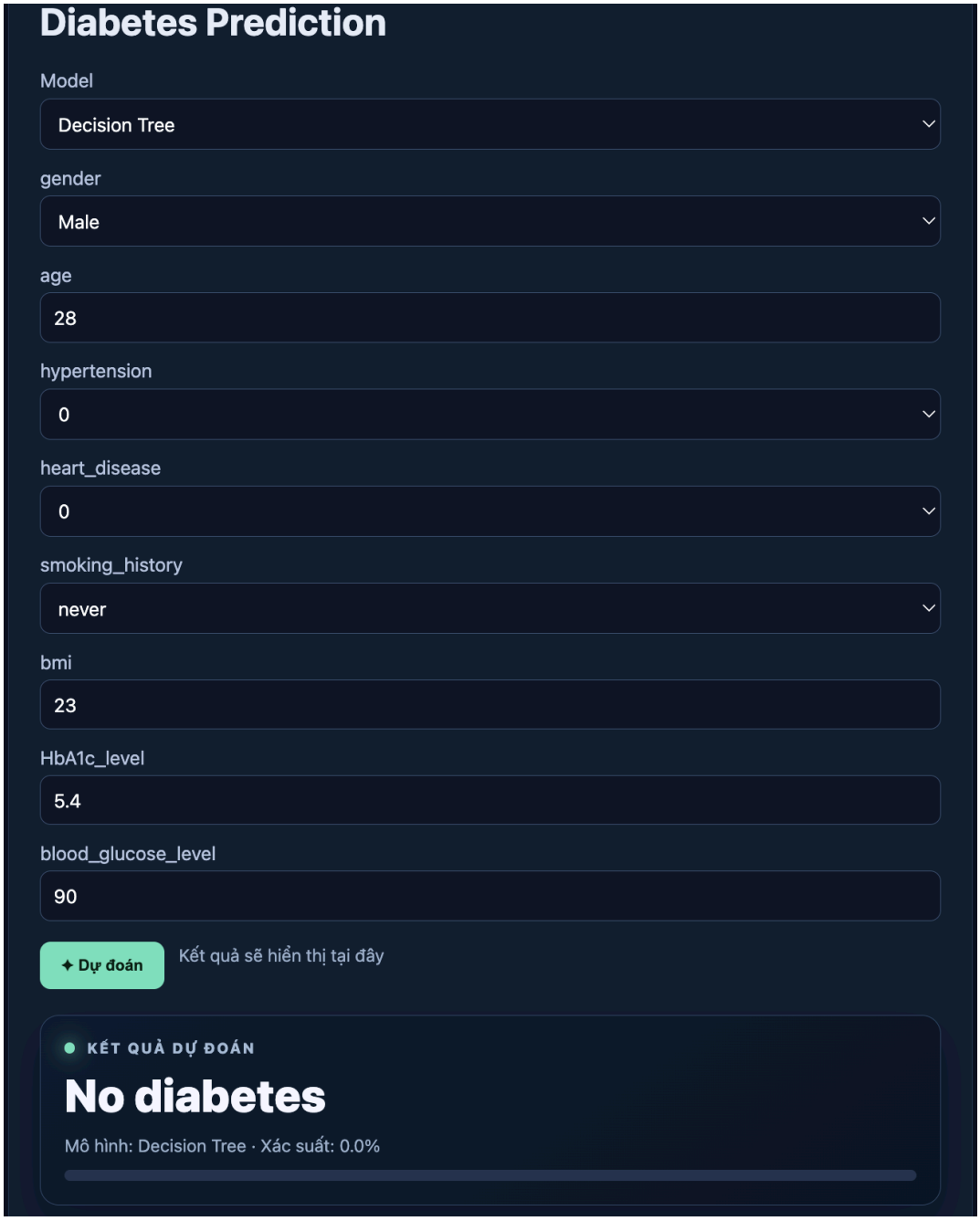
Cell 9 — EDA (notebook gốc).

```
price_numeric = pd.to_numeric(df_raw["price"], errors="coerce")
valid_price = price_numeric[price_numeric > 0]

fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()
```



Hình 2.10: Ma trận nhầm lẫn của benchmark gốc; cần đặc biệt quan sát số âm tính giả ở lớp dương.



Hình 2.11: Giao diện deploy diabetes large với các chỉ số đầu vào, mô hình và kết quả dự đoán.

```
axes[0].hist(np.log1p(valid_price), bins=60, color="#4C78A8", alpha=0.85)
axes[0].set_title("Phân phối log1p(price)")
axes[0].set_xlabel("log1p(price)")
axes[0].set_ylabel("Số mẫu")

missing_percent = df_raw.isna().mean().sort_values(ascending=False) * 100
axes[1].barh(missing_percent.index, missing_percent.values,
↪  color="#F58518")
axes[1].set_title("Missing values theo cột")
axes[1].set_xlabel("Missing (%)")

df_raw["status"].value_counts().head(10).plot(kind="bar", ax=axes[2],
↪  color="#54A24B")
axes[2].set_title("Top status")
axes[2].set_xlabel("status")
axes[2].set_ylabel("Số mẫu")
axes[2].tick_params(axis="x", rotation=20)

df_raw["state"].value_counts().head(12).plot(kind="bar", ax=axes[3],
↪  color="#E45756")
axes[3].set_title("Top state")
axes[3].set_xlabel("state")
axes[3].set_ylabel("Số mẫu")
axes[3].tick_params(axis="x", rotation=45)

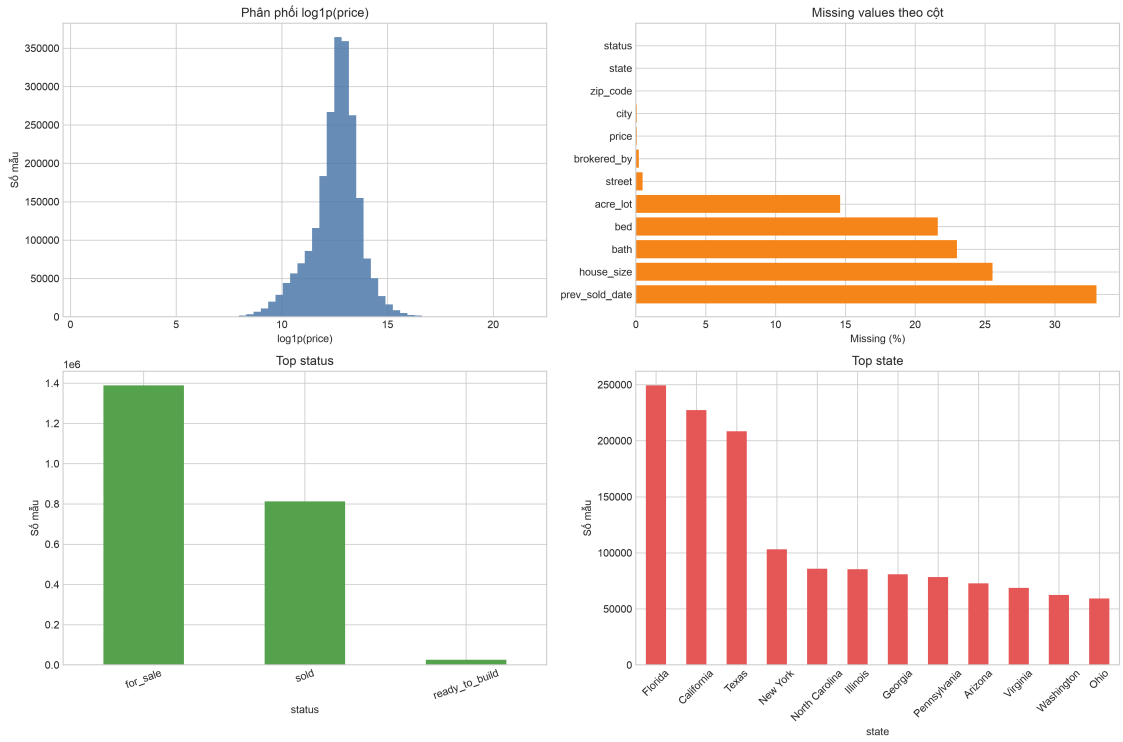
plt.tight_layout()
save_current_figure("house_price_eda_overview.png")
plt.show()
```

Output hình của cell: Hình 2.12.

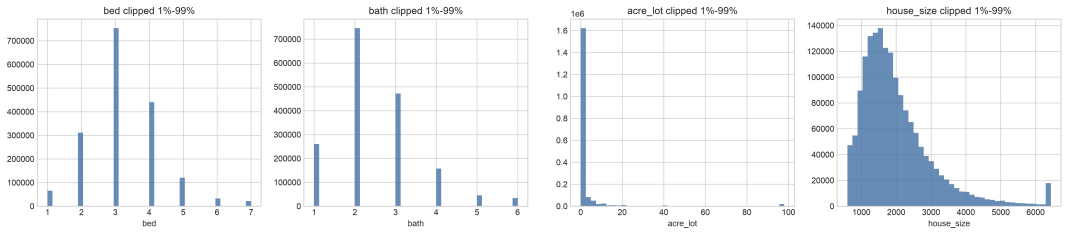
Giá nhà lệch phải và có ngoại lệ lớn, nên notebook học trên $\log(1 + \texttt{price})$ rồi đổi ngược về USD khi đánh giá. Các cột phân loại có số giá trị rất lớn như thành phố hoặc ZIP được biểu diễn gọn hơn bằng frequency encoding; thời gian bán trước được tách thành đặc trưng liên quan. Biến đổi này không nên fit trên toàn bộ dữ liệu trước khi chia, vì thống kê tần suất có thể làm lộ thông tin của test. Mô hình hồi quy phải học đồng thời quan hệ giữa diện tích, vị trí, loại giao dịch và giá trị tài sản.

Cell 21 — huấn luyện baseline hồi quy (notebook gốc).

```
baseline_models = {
    "Ridge Regression": Ridge(alpha=1.0),
    "Decision Tree Regressor": DecisionTreeRegressor(
        max_depth=18,
        min_samples_leaf=50,
        random_state=RANDOM_STATE,
    ),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=60,
        max_depth=16,
        min_samples_leaf=30,
```



Hình 2.12: Khảo sát giá nhà, dữ liệu thiếu và đặc trưng bất động sản.



Hình 2.13: Phân phối các biến số chính trong mẫu nhà được xử lý.

```
        max_features="sqrt",
        n_jobs=-1,
        random_state=RANDOM_STATE,
    ),
}

benchmark_results = []
test_predictions = {}
validation_predictions = {}

for model_name, model in baseline_models.items():
    print(f"Training {model_name}...")

    start = time.perf_counter()
    model.fit(X_train, y_train_log)
    train_time = time.perf_counter() - start

    val_pred_log = model.predict(X_val)

    start = time.perf_counter()
    test_pred_log = model.predict(X_test)
    predict_time = time.perf_counter() - start

    result = evaluate_regression_model(
        model_name,
        y_test,
        test_pred_log,
        train_time,
        predict_time,
    )
    benchmark_results.append(result)
    validation_predictions[model_name] = val_pred_log
    test_predictions[model_name] = {
        "log_prediction": test_pred_log,
        "price_prediction": log_price_to_price(test_pred_log),
    }

    val_metrics = regression_metrics(y_val, val_pred_log)
    print(f"  Validation RMSLE: {val_metrics['RMSLE']:.4f}, MAE:
    ↪  ${val_metrics['MAE']:,.0f}")
    print(f"  Test RMSLE: {result['RMSLE']:.4f}, MAE:
    ↪  ${result['MAE']:,.0f}")

baseline_results_df = display_regression_results(benchmark_results)
```

Output của cell:

```
Training Ridge Regression...
  Validation RMSLE: 0.7694, MAE: $264,134
  Test RMSLE: 0.7717, MAE: $268,912
Training Decision Tree Regressor...
```

```
  Validation RMSLE: 0.7156, MAE: $238,756
  Test RMSLE: 0.7205, MAE: $241,108
Training Random Forest Regressor...
  Validation RMSLE: 0.7183, MAE: $245,358
  Test RMSLE: 0.7216, MAE: $247,668

index=0; Model=Decision Tree Regressor; MAE=$241,108; RMSE=$1,169,286;
↪  MedianAE=$87,561; MAPE=4952.28%
  RMSLE=0.7205; R2=0.2408; Train time (s)=1.107; Predict time (s)=0.002
index=1; Model=Random Forest Regressor; MAE=$247,668; RMSE=$1,241,475;
↪  MedianAE=$87,537; MAPE=2614.12%
  RMSLE=0.7216; R2=0.1442; Train time (s)=1.010; Predict time (s)=0.031
index=2; Model=Ridge Regression; MAE=$268,912; RMSE=$1,228,158;
↪  MedianAE=$92,871; MAPE=2871.93%
  RMSLE=0.7717; R2=0.1624; Train time (s)=0.416; Predict time (s)=0.000
```

Cell 25 — huấn luyện MLP NumPy (notebook gốc).

```
mlp = NumpyMLPRegressor(
    hidden_layers=(128, 64, 32),
    learning_rate=0.001,
    epochs=60,
    batch_size=2048,
    l2=1e-4,
    patience=8,
    min_delta=1e-4,
    random_state=RANDOM_STATE,
    verbose=True,
)

start = time.perf_counter()
mlp.fit(X_train, y_train_log, X_val, y_val_log)
mlp_train_time = time.perf_counter() - start

mlp_val_pred_log = mlp.predict(X_val)

start = time.perf_counter()
mlp_test_pred_log = mlp.predict(X_test)
mlp_predict_time = time.perf_counter() - start

mlp_result = evaluate_regression_model(
    "NumPy MLP (128-64-32)",
    y_test,
    mlp_test_pred_log,
    mlp_train_time,
    mlp_predict_time,
)

benchmark_results.append(mlp_result)
validation_predictions["NumPy MLP (128-64-32)"] = mlp_val_pred_log
test_predictions["NumPy MLP (128-64-32)"] = {
```

```
    "log_prediction": mlp_test_pred_log,
    "price_prediction": log_price_to_price(mlp_test_pred_log),
}

print(f"Best epoch: {mlp.best_epoch_}")
print(f"Best validation RMSLE: {mlp.best_validation_rmsle_:.4f}")
print(f"Test RMSLE: {mlp_result['RMSLE']:.4f}, MAE:
 ↪  ${mlp_result['MAE']:,.0f}")
```

Output của cell:

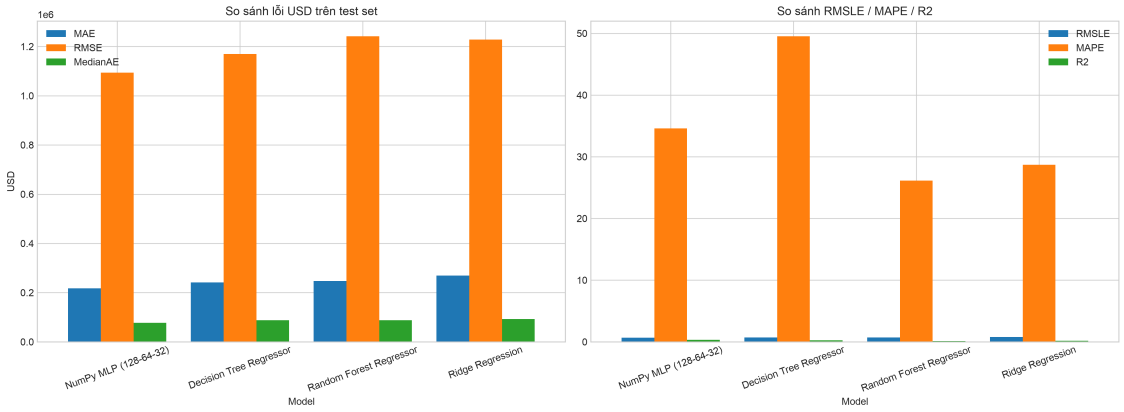
```
Epoch 001 | train_loss=0.3425 | val_rmsle=0.7666 | val_mae=$268,726
Epoch 005 | train_loss=0.1998 | val_rmsle=0.6864 | val_mae=$235,936
Epoch 010 | train_loss=0.1892 | val_rmsle=0.6783 | val_mae=$233,038
Epoch 015 | train_loss=0.1786 | val_rmsle=0.6651 | val_mae=$225,513
Epoch 020 | train_loss=0.1735 | val_rmsle=0.6624 | val_mae=$225,419
Epoch 025 | train_loss=0.1719 | val_rmsle=0.6639 | val_mae=$226,255
Epoch 030 | train_loss=0.1711 | val_rmsle=0.6604 | val_mae=$224,019
Epoch 035 | train_loss=0.1660 | val_rmsle=0.6595 | val_mae=$220,152
Epoch 040 | train_loss=0.1648 | val_rmsle=0.6595 | val_mae=$221,415
Epoch 045 | train_loss=0.1634 | val_rmsle=0.6587 | val_mae=$218,413
Epoch 050 | train_loss=0.1615 | val_rmsle=0.6580 | val_mae=$221,877
Epoch 055 | train_loss=0.1604 | val_rmsle=0.6579 | val_mae=$218,556
Epoch 060 | train_loss=0.1621 | val_rmsle=0.6629 | val_mae=$231,845
Early stopping tại epoch 60. Best epoch: 52
Best epoch: 52
Best validation RMSLE: 0.6561
Test RMSLE: 0.6578, MAE: $217,538
```

Bảng 2.4: Ba mô hình được chọn trên house price (test set)

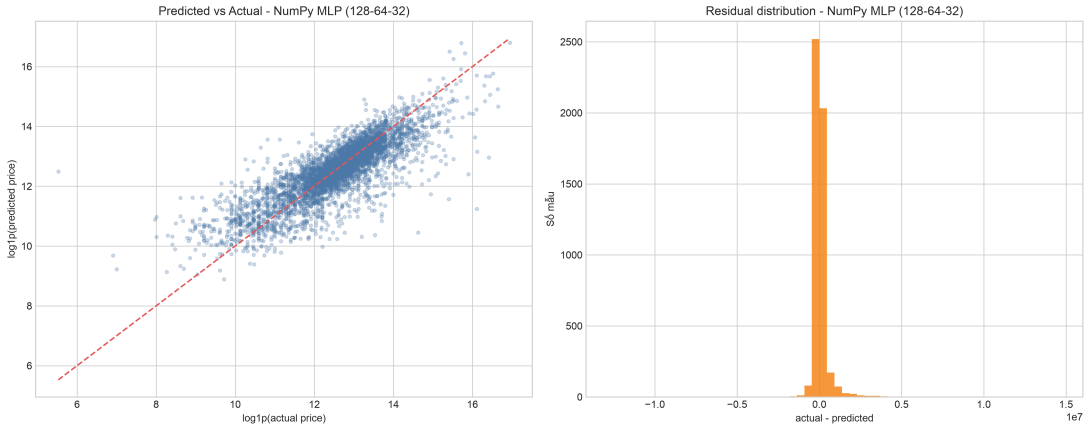
Mô hình	**MAE (USD)**	**RMSE (USD)**	**MedianAE**	**MAPE**	**RMSLE**	R^2
NumPy MLP (128-64-32)	**217.538**	**1.094.000**	**77.432**	34.6065	**0.6578**	**0.3354**
Decision Tree Regressor	241.108	1.169.300	87.561	49.5228	0.7205	0.2408
Random Forest Regressor	247.668	1.241.500	87.537	**26.1412**	0.7216	0.1442

MLP đạt MAE, RMSLE và R^2 tốt nhất trong ba mô hình. MAE khoảng 217 nghìn USD vẫn là sai số lớn về mặt kinh tế. MedianAE thấp hơn MAE nhiều, còn RMSE vượt một triệu USD, cho thấy một số ngoại lệ gây sai số đặc biệt lớn. Random Forest có MAPE thấp nhất, nhắc rằng cách xếp hạng mô hình phụ thuộc metric và mục tiêu sử dụng: sai số tuyệt đối bằng USD hay sai số tương đối tính theo giá nhà.

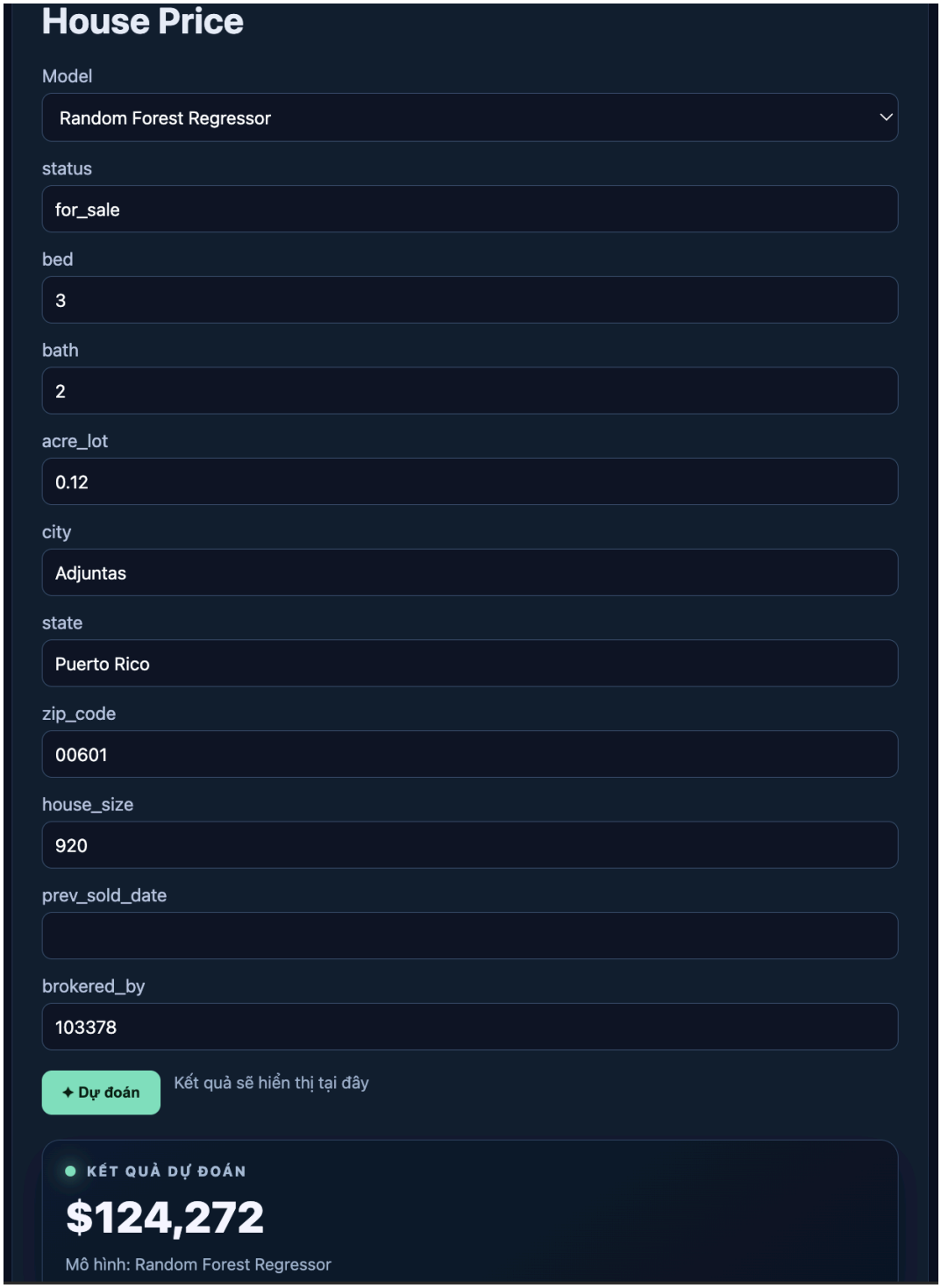
Các giá trị lớn và những khu vực hiếm có thể không được biểu diễn đủ trong tập lấy mẫu. Nếu triển khai, cần đánh giá riêng theo vùng địa lý, phân khúc giá và thời gian giao dịch; một điểm số chung trên tập trộn toàn quốc không cho biết sai số tại từng thị trường. Tập dữ liệu lịch sử cũng có thể lệch với mặt bằng giá hiện tại. Đây là giới hạn quan trọng của mọi dự báo giá nhà dựa trên snapshot dữ liệu.



Hình 2.14: Biểu đồ so sánh hồi quy của benchmark gốc, có thêm Ridge Regression ngoài ba mô hình của tiểu luận.



Hình 2.15: Phân tích phần dư của mô hình tốt nhất để nhận diện các miền giá khó dự báo.



Hình 2.16: Giao diện deploy house price với thông tin căn nhà, mô hình và giá dự đoán.

2.5 Tổng hợp Chương 2

Ba bài toán cho thấy không có mô hình duy nhất dẫn đầu trong mọi hoàn cảnh. Naive Bayes đạt F_1 cao nhất trên TF-IDF nhận xét; Decision Tree nổi bật trên dữ liệu tiểu đường có quy tắc ngưỡng; MLP dẫn đầu về phần lớn metric giá nhà. Các kết quả này là kết quả của pipeline, cách chia dữ liệu và cấu hình notebook hiện tại. So sánh công bằng phải đọc kèm tỷ lệ lớp, số mẫu thực sự dùng, ngưỡng ra quyết định và đơn vị của metric. Việc đưa mô hình lên giao diện localhost giúp kiểm tra luồng inference, nhưng không thay thế việc đánh giá thống kê trên test set.

Chương 3

Mạng nơ-ron tích chập

3.1 Từ ảnh số đến đặc trưng không gian

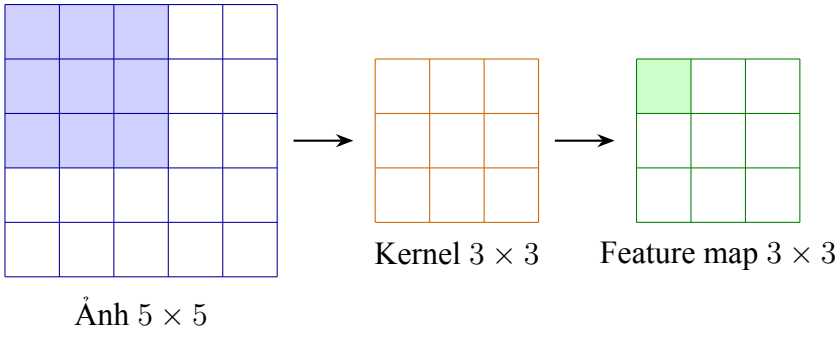
Ảnh là một lưới pixel có quan hệ không gian. Nếu trải phẳng ảnh $32 \times 32 \times 3$ thành vector, một MLP phải học lại từ dữ liệu khái niệm gần nhau, cạnh, góc và họa tiết. CNN đưa vào một giả định hữu ích: cùng một mẫu cục bộ có thể xuất hiện ở nhiều vị trí. Một kernel nhỏ trượt trên ảnh và dùng chung trọng số, nhờ đó số tham số giảm và đặc trưng có tính dịch chuyển. Với ảnh đầu vào X và kernel K, một feature map được tính xấp xỉ bởi

$$Y_{i,j,c} = \phi\left(b_c + \sum_{u,v,k} K_{u,v,k,c} X_{i+u,j+v,k}\right). \tag{3.1}$$

Tầng đầu có thể học cạnh hoặc màu; các tầng sau kết hợp chúng thành bộ phận và vật thể. Tăng độ sâu làm vùng ảnh ảnh hưởng tới một neuron, gọi là receptive field, lớn hơn. Pooling hoặc phép tích chập stride lớn giảm độ phân giải, tiết kiệm tính toán và làm biểu diễn bớt nhạy với dịch chuyển nhỏ. Cuối mạng, các đặc trưng được gom và biến thành điểm số cho từng lớp qua softmax.

Trong công thức tích chập, (i, j) là vị trí không gian của ô đầu ra, c là kênh đầu ra, (u, v) là vị trí trong kernel và k duyệt kênh đầu vào. $K_{u,v,k,c}$ là trọng số được *dùng lại* ở mọi vị trí (i, j); b_c là bias của kênh c; ϕ có thể là ReLU. Các thư viện học sâu thường tính tương quan chéo, tức không lật kernel như định nghĩa tích chập toán học truyền thống, nhưng vẫn gọi thao tác này là convolution. Ở biên ảnh, chỉ số $i + u, j + v$ phụ thuộc cách padding; công thức trên được hiểu ở những vị trí hợp lệ sau khi đã quy định padding và stride.

Hình 3.1 minh họa một trường hợp một kênh: ô đầu ra đầu tiên dùng đúng vùng 3×3 được tô trên ảnh 5×5. Khi kernel trượt sang phải một ô, cùng chín trọng số lại được nhân với chín pixel mới. Nếu có nhiều kênh đầu vào, mỗi filter trải qua tất cả các kênh ấy; nếu có nhiều filter, mỗi filter tạo một kênh đầu ra riêng. Sự chia sẻ trọng số giúp số tham số ít hơn tầng nối đầy đủ, nhưng không khiến CNN bất biến hoàn toàn với mọi phép dịch hoặc xoay.



Hình 3.1: Kernel 3×3 trượt trên ảnh 5×5 với stride 1, không padding.

3.1.1 Huấn luyện và các kỹ thuật cải thiện

Loss thường dùng cho phân loại đa lớp là cross-entropy $-\sum_c y_c \log \hat{p}_c$. Gradient truyền từ loss qua tầng fully connected, pooling, activation và convolution để cập nhật kernel. ReLU giúp tối ưu dễ hơn sigmoid trong các tầng sâu; Batch Normalization ổn định phân phối kích hoạt; Dropout và weight decay giảm nguy cơ quá khớp. Data augmentation như dịch, crop và lật ảnh tạo biến thể hợp lý mà vẫn giữ nhãn. Với CIFAR-10, residual connection cho phép block học phần hiệu chỉnh $F(x)$ và trả về $F(x) + x$, giúp huấn luyện mạng sâu hơn [7].

Nếu y_c là nhãn one-hot, chỉ lớp đúng có $y_c = 1$; khi đó cross-entropy bằng $-\log \hat{p}_{\text{đúng}}$. Ví dụ mô hình gán xác suất 0,9 cho lớp đúng thì loss khoảng 0,105, còn gán 0,1 thì loss khoảng 2,303. Loss vì vậy phạt mạnh dự đoán tự tin nhưng sai. Khi softmax đi cùng cross-entropy, đạo hàm theo logit lớp c rút gọn thành $\hat{p}_c - y_c$; tín hiệu này được truyền ngược qua các tầng theo quy tắc dây chuyền. Trong convolution, gradient của một trọng số kernel cộng đóng góp từ *mọi* vị trí mà trọng số ấy đã được dùng. Điều này phản ánh đúng cơ chế chia sẻ trọng số của forward pass.

Batch Normalization chuẩn hóa kích hoạt trong mini-batch rồi học thêm tham số scale và shift; khi suy luận nó dùng thống kê đã tích lũy trong quá trình train. Dropout ngẫu nhiên tắt một phần kích hoạt khi train để mạng bớt phụ thuộc vào một đường truyền riêng lẻ, nhưng phải được tắt hoặc chuyển sang cơ chế suy luận tương đương khi predict. Weight decay phạt trọng số lớn trong mục tiêu tối ưu. Augmentation chỉ nên tạo biến đổi giữ nguyên nhãn: lật ngang một ảnh ô tô có thể hợp lý, nhưng lật một chữ số viết tay có thể đổi ý nghĩa. Residual connection tạo đường truyền trực tiếp $x \mapsto x + F(x)$; nếu block cần gần như giữ nguyên thông tin, nó chỉ phải học $F(x)$ gần 0.

Việc so sánh ba cách cài đặt trong Assignment 04 có hai mục đích. NumPy CNN cho thấy các phép tính cơ bản có thể tự cài đặt và kiểm chứng. PyTorch và Keras cho thấy lợi ích của framework, backend tối ưu và kiến trúc lớn hơn. Tuy nhiên đây không phải phép thử tuyệt đối về framework: NumPy dùng ít ảnh train hơn và mạng nhỏ hơn. Do đó khoảng cách metric gồm cả ảnh hưởng của dữ liệu, kiến trúc, augmentation và tối ưu hóa, không thể quy hết cho thư viện.

Ảnh được đưa về khoảng $[0, 1]$ rồi chuẩn hóa theo mean và độ lệch chuẩn tính trên framework train set. Validation, test và subset NumPy dùng cùng thống kê đã học. Train/validation

Bảng 3.1: Kích thước và cách chia dữ liệu CNN

Dataset	Gốc train/test	Framework train/val	NumPy train/val	Ảnh và lớp
MNIST	60.000/10.000	54.000/6.000	20.000/3.000	$28 \times 28 \times 1$, 10 chữ số
CIFAR-10	50.000/10.000	45.000/5.000	15.000/3.000	$32 \times 32 \times 3$, 10 vật thể

được chia có phân tầng, seed 42; test gốc của mỗi dataset giữ nguyên. Các mô hình đều báo cáo trên cùng full test set. Macro-F1 lấy trung bình F_1 từng lớp, để lớp có ít hay nhiều mẫu đều đóng góp như nhau.

3.1.2 Kích thước feature map và số tham số

Để thấy vì sao CNN phù hợp ảnh, xét một tầng convolution nhận ảnh $H \times W \times C_{\text{in}}$, dùng C_{out} filter cỡ $k \times k$. Nếu có bias, số tham số là $k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$, không phụ thuộc trực tiếp vào H và W. Một tầng fully connected nối mọi pixel với cùng số neuron đầu ra sẽ cần khoảng $HWC_{\text{in}}C_{\text{out}}$ trọng số. Khi kích thước ảnh tăng, khác biệt này rất lớn. Chia sẻ trọng số cũng buộc một bộ dò cạnh học ở góc trái có thể áp dụng tại góc phải.

Ví dụ ảnh RGB có $C_{\text{in}} = 3$ và dùng 16 filter 3×3: tầng convolution có $(3^2 \cdot 3+1)\cdot 16 = 448$ tham số. Con số 448 giữ nguyên dù ảnh là 32×32 hay lớn hơn, miễn số kênh đầu vào không đổi. Tuy nhiên, số phép nhân cộng tăng theo số vị trí đầu ra. Nếu nối phẳng toàn bộ ảnh $32 \times 32 \times 3$ vào 16 neuron, riêng trọng số đã là $32 \cdot 32 \cdot 3 \cdot 16 = 49.152$. So sánh này chỉ nhằm làm rõ hiệu quả chia sẻ trọng số; tầng fully connected và tầng convolution không hoàn toàn tương đương về chức năng.

Output của phép tích chập phụ thuộc padding và stride: $H_{\text{out}} = \lfloor (H + 2p - k)/s \rfloor + 1$, tương tự cho chiều rộng. Nếu dùng padding để giữ kích thước, các tầng đầu có thể học chi tiết ở độ phân giải gốc; nếu pooling 2×2 sau block, độ rộng và cao giảm một nửa. Qua nhiều block, số kênh thường tăng khi độ phân giải giảm, vì biểu diễn chuyển dần từ vị trí cụ thể sang kiểu đặc trưng. Số tham số ít không có nghĩa chi phí chạy thấp: convolution vẫn cần nhiều phép nhân cộng trên từng vị trí, nên backend tăng tốc và bố trí tensor rất quan trọng.

Ở công thức kích thước, H là chiều cao đầu vào, p là số ô đệm mỗi phía, k là cạnh kernel và s là bước trượt; phần lấy sàn xuất hiện khi kernel không khớp chính xác vị trí cuối. Với $H = 32, k = 3, p = 1, s = 1$, đầu ra cao 32; đổi thành $s = 2$ cho 16. Nếu không padding, $H = 5, k = 3, s = 1$ cho 3 như Hình 3.1. Receptive field của một ô ở tầng sau gồm tất cả vị trí đầu vào có thể ảnh hưởng đến nó. Hai tầng kernel 3×3 stride 1 liên tiếp tạo receptive field 5×5 ở tầng thứ hai; phép pooling hoặc stride lớn làm vùng này tăng nhanh hơn nhưng cũng bỏ bớt chi tiết không gian.

Trong mạng phân loại, tầng softmax biến vector logit z thành $\hat{p}_c = e^{z_c} / \sum_j e^{z_j}$. Lớp dự đoán là lớp có logit cao nhất; confidence thường được hiển thị là $\max_c \hat{p}_c$. Giá trị này hữu ích cho giao diện nhưng không tự động là xác suất được hiệu chỉnh. Một mô hình có thể rất tự tin khi ảnh đầu vào khác phân bố train, chẳng hạn ảnh trắng đen CIFAR-10, ảnh MNIST viết ngược hoặc ảnh mờ. Khi phân tích lỗi, cần xem cả nhãn sai, confidence và nội dung ảnh.

Trong softmax, mỗi z_c là điểm chưa chuẩn hóa của lớp c; các $\hat{p}_c$ không âm và cộng lại bằng 1. Để tính ổn định với logit lớn, phần mềm thường trừ cùng một hằng số $\max_j z_j$ khỏi mọi logit trước khi lấy mũ; phép này không đổi kết quả. Confidence cao chỉ mô tả phân bố đầu ra của mạng. Muốn diễn giải nó như tần suất đúng trong thực tế, cần kiểm tra hiệu chỉnh xác suất trên dữ liệu phù hợp, chẳng hạn so sánh nhóm dự đoán khoảng 80% với tỷ lệ đúng quan sát được của nhóm ấy.

3.1.3 Công bằng trong benchmark và ý nghĩa thời gian

Các mô hình dùng full test set giống nhau nhưng không có cùng ngân sách train. Việc tách train theo tỷ lệ 90/10 cho framework tạo 54.000 ảnh MNIST và 45.000 ảnh CIFAR-10; NumPy dùng subset 20.000 và 15.000 ảnh. Do đó benchmark kiểm tra ba pipeline đã hiện thực, không cô lập tác động của ngôn ngữ lập trình. Nếu muốn trả lời riêng câu hỏi PyTorch và Keras khác nhau về tốc độ bao nhiêu, cần cố định kiến trúc, batch size, thiết bị, precision, số epoch và quy trình đo. Ngay cả khi đó, kết quả chỉ đại diện cho môi trường thử.

Thời gian dự đoán cũng khác thời gian huấn luyện. Một app localhost nhận một ảnh tại một thời điểm chịu chi phí khởi tạo model, chuyển dữ liệu và xử lý HTTP; số đo inference theo batch lớn trong notebook không nói hết độ trễ người dùng cảm nhận. Ngược lại, hệ thống xử lý hàng nghìn ảnh có thể tận dụng batch trên GPU. Vì vậy phần deploy ở mỗi dataset minh họa chức năng suy luận, còn các bảng test mô tả chất lượng thống kê. Hai phép kiểm tra bổ sung cho nhau.

3.2 MNIST: chữ số viết tay

Mã nguồn GitHub: https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2004/mnist

Nguồn dữ liệu: https://keras.io/api/datasets/mnist/

MNIST gồm 60.000 ảnh train gốc và 10.000 ảnh test, mỗi ảnh xám 28×28 với nhãn từ 0 đến 9. Trong Hình 3.2, mỗi cột cho một kiểu nét viết; sự khác biệt về nét, độ nghiêng và độ dày tạo ra các trường hợp khó dù nền ảnh đơn giản. Tập train gốc có lớp 1 nhiều nhất, 6.742 ảnh, còn lớp 5 có 5.421 ảnh. Mức chênh này không quá lớn nhưng đủ để Macro-F1 có ý nghĩa hơn việc chỉ xem accuracy.

Cell 14 — EDA (notebook gốc).

```
# Hiển thị một ảnh đại diện cho mỗi lớp.
fig, axes = plt.subplots(2, 5, figsize=(13, 5))
for label, ax in enumerate(axes.ravel()):
    image = x_train_full[np.flatnonzero(y_train_full == label)[0]]
    if image.shape[-1] == 1:
        ax.imshow(image.squeeze(-1), cmap="gray")
    else:
        ax.imshow(image)
```

```
    ax.set_title(f"{label}: {CLASS_NAMES[label]}")
    ax.axis("off")
fig.suptitle(f"Ảnh mẫu của 10 lớp - {DATASET_NAME}", fontsize=14)
plt.tight_layout()
plt.show()

train_counts = np.bincount(y_train_full, minlength=10)
test_counts = np.bincount(y_test_full, minlength=10)
distribution_df = pd.DataFrame({
    "class_id": np.arange(10),
    "class_name": CLASS_NAMES,
    "train_count": train_counts,
    "test_count": test_counts,
})
display(distribution_df)

ax = distribution_df.set_index("class_name")[["train_count",
↪  "test_count"]].plot(
    kind="bar", figsize=(12, 5), color=["#4C78A8", "#F58518"]
)
ax.set_title(f"Phân phối lớp của {DATASET_NAME}")
ax.set_xlabel("Lớp")
ax.set_ylabel("Số ảnh")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

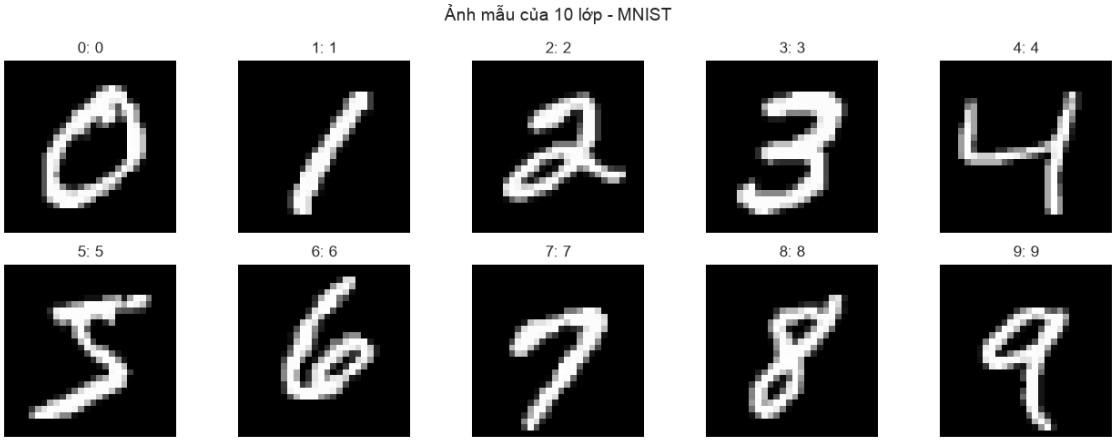
Output của cell:

```
index=0; class_id=0; class_name=0; train_count=5923; test_count=980
index=1; class_id=1; class_name=1; train_count=6742; test_count=1135
index=2; class_id=2; class_name=2; train_count=5958; test_count=1032
index=3; class_id=3; class_name=3; train_count=6131; test_count=1010
index=4; class_id=4; class_name=4; train_count=5842; test_count=982
index=5; class_id=5; class_name=5; train_count=5421; test_count=892
index=6; class_id=6; class_name=6; train_count=5918; test_count=958
index=7; class_id=7; class_name=7; train_count=6265; test_count=1028
index=8; class_id=8; class_name=8; train_count=5851; test_count=974
index=9; class_id=9; class_name=9; train_count=5949; test_count=1009
```

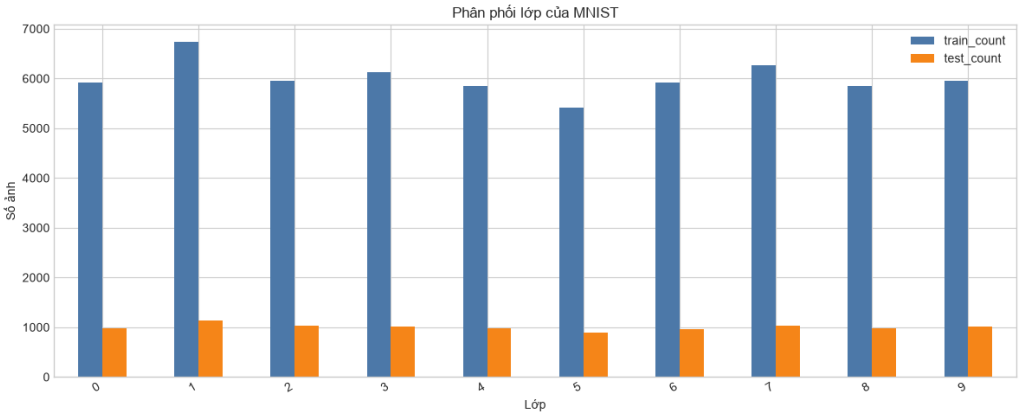
Output hình của cell: Hình 3.2, Hình 3.3.

Một mẫu đầu vào cụ thể là ma trận 784 giá trị pixel; output mong đợi là một nhãn 0–9. Các giá trị pixel ban đầu từ 0 đến 255. Trong notebook, mean và độ lệch chuẩn học từ train lần lượt khoảng 0.1307 và 0.3082 sau khi chia 255. CNN NumPy dùng hai tầng tích chập với 8 và 16 filter, hai bước pooling và tầng ẩn 64, tổng 52.138 tham số. Phiên bản PyTorch và Keras dùng hai block convolution–BatchNorm–ReLU–pooling, dropout rồi Dense 128; mỗi mạng khoảng 468 nghìn tham số. Chúng cùng loại kiến trúc nhưng không có số tham số hoàn toàn bằng nhau do chi tiết implementation.

Cell 26 — huấn luyện CNN NumPy (notebook gốc).



Hình 3.2: Ảnh mẫu đại diện cho mười lớp chữ số trong MNIST.



Hình 3.3: Phân bố nhãn train và test của MNIST.

```
numpy_model = NumpyCNN(
    input_shape=x_train_numpy.shape[1:],
    conv_filters=NUMPY_FILTERS,
    hidden_units=NUMPY_HIDDEN_UNITS,
    num_classes=10,
    random_state=RANDOM_STATE,
)
print("NumPy CNN parameters:", f"{numpy_model.parameter_count():,}")

start = time.perf_counter()
numpy_history = train_numpy_cnn(
    numpy_model,
    x_train_numpy,
    y_train_numpy,
    x_val_numpy,
    y_val_numpy,
    epochs=NUMPY_EPOCHS,
    batch_size=NUMPY_BATCH_SIZE,
    learning_rate=NUMPY_LEARNING_RATE,
    weight_decay=NUMPY_WEIGHT_DECAY,
    dataset_key=DATASET_KEY,
    random_state=RANDOM_STATE,
)
numpy_train_time = time.perf_counter() - start

start = time.perf_counter()
numpy_test_probabilities = predict_numpy_proba(
    numpy_model,
    x_test,
    batch_size=NUMPY_BATCH_SIZE,
)
numpy_predict_time = time.perf_counter() - start

benchmark_results.append(evaluate_predictions(
    "NumPy CNN from scratch",
    y_test,
    numpy_test_probabilities,
    numpy_train_time,
    numpy_predict_time,
    numpy_model.parameter_count(),
    len(y_train_numpy),
    numpy_history["best_epoch"],
    numpy_history["learning_rate"][-1],
))
prediction_store["NumPy CNN from scratch"] = numpy_test_probabilities
history_store["NumPy CNN from scratch"] = numpy_history
print(
    f"Train time: {numpy_train_time:.2f}s | Test accuracy: "
    f"{accuracy_score(y_test, np.argmax(numpy_test_probabilities,
    ↪  axis=1)):.4f}"
```

```
)
```

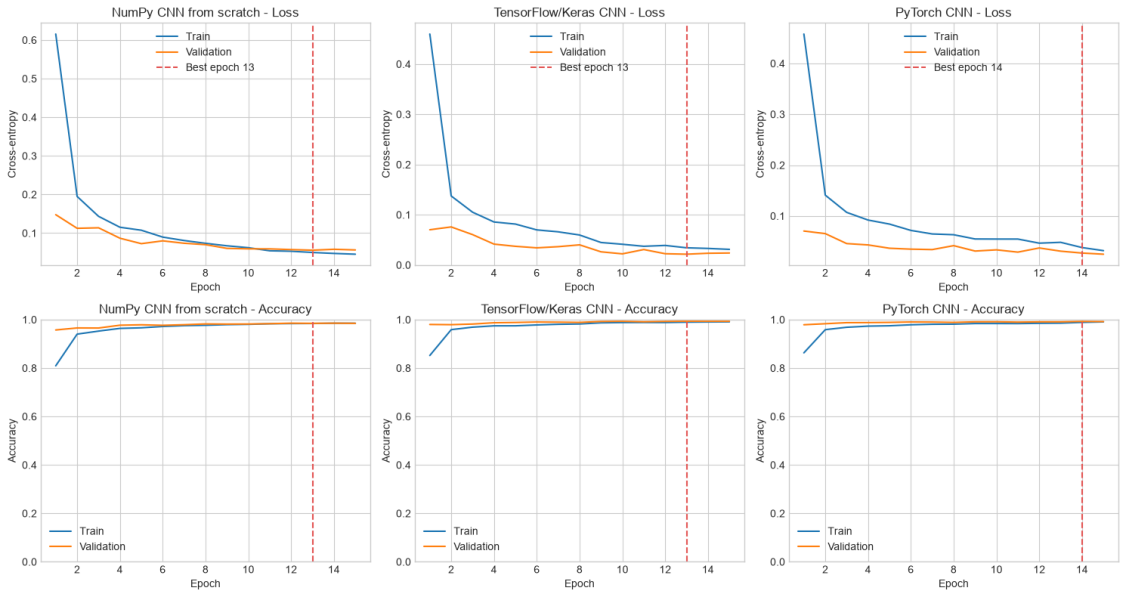
Output của cell:

```
NumPy CNN parameters: 52,138
Epoch 001/15 | lr=0.001000 | loss=0.6155 | acc=0.8094 | val_loss=0.1477 |
↪  val_acc=0.9573
Epoch 002/15 | lr=0.000988 | loss=0.1949 | acc=0.9402 | val_loss=0.1124 |
↪  val_acc=0.9657
Epoch 003/15 | lr=0.000953 | loss=0.1435 | acc=0.9526 | val_loss=0.1138 |
↪  val_acc=0.9653
Epoch 004/15 | lr=0.000896 | loss=0.1155 | acc=0.9639 | val_loss=0.0878 |
↪  val_acc=0.9780
Epoch 005/15 | lr=0.000821 | loss=0.1071 | acc=0.9666 | val_loss=0.0717 |
↪  val_acc=0.9787
Epoch 006/15 | lr=0.000731 | loss=0.0893 | acc=0.9718 | val_loss=0.0754 |
↪  val_acc=0.9793
Epoch 007/15 | lr=0.000631 | loss=0.0803 | acc=0.9755 | val_loss=0.0719 |
↪  val_acc=0.9797
Epoch 008/15 | lr=0.000525 | loss=0.0731 | acc=0.9770 | val_loss=0.0713 |
↪  val_acc=0.9810
Epoch 009/15 | lr=0.000419 | loss=0.0674 | acc=0.9789 | val_loss=0.0597 |
↪  val_acc=0.9827
Epoch 010/15 | lr=0.000319 | loss=0.0620 | acc=0.9811 | val_loss=0.0592 |
↪  val_acc=0.9827
Epoch 011/15 | lr=0.000229 | loss=0.0547 | acc=0.9831 | val_loss=0.0588 |
↪  val_acc=0.9840
Epoch 012/15 | lr=0.000154 | loss=0.0532 | acc=0.9841 | val_loss=0.0582 |
↪  val_acc=0.9850
Epoch 013/15 | lr=0.000097 | loss=0.0501 | acc=0.9839 | val_loss=0.0567 |
↪  val_acc=0.9853
Epoch 014/15 | lr=0.000062 | loss=0.0477 | acc=0.9849 | val_loss=0.0581 |
↪  val_acc=0.9853
Epoch 015/15 | lr=0.000050 | loss=0.0453 | acc=0.9852 | val_loss=0.0567 |
↪  val_acc=0.9850
Restored NumPy weights from epoch 13 (val_loss=0.0567).
Train time: 98.05s | Test accuracy: 0.9850
```

Learning curve cho thấy cả ba mạng hội tụ, nhưng tốc độ và mức cuối khác nhau. CNN NumPy chọn checkpoint ở epoch 13 theo validation loss; PyTorch đạt validation accuracy tốt nhất tại epoch 14, Keras tại epoch 13. Chọn checkpoint bằng validation tránh việc một epoch cuối kém hơn làm lệch phép thử. Tuy nhiên việc dùng tập NumPy nhỏ hơn khiến không thể dùng thời gian huấn luyện để tuyên bố implementation NumPy nhanh hơn một cách tổng quát.

Cell 42 — tổng hợp kết quả (notebook gốc).

```
ordered_columns = [
    "Model",
    "Accuracy",
    "Macro Precision",
```



Hình 3.4: Loss và accuracy theo epoch của ba CNN trên MNIST.

```
    "Macro Recall",
    "Macro F1",
    "Train samples",
    "Best epoch",
    "Final LR",
    "Parameters",
    "Train time (s)",
    "Predict time (s)",
]
results_df = pd.DataFrame(benchmark_results)[ordered_columns]
results_df = results_df.sort_values(
    ["Macro F1", "Accuracy"],
    ascending=False,
).reset_index(drop=True)
display(results_df.style.format({
    "Accuracy": "{:.4f}",
    "Macro Precision": "{:.4f}",
    "Macro Recall": "{:.4f}",
    "Macro F1": "{:.4f}",
    "Train samples": "{:,}",
    "Best epoch": "{:d}",
    "Final LR": "{:.6f}",
    "Parameters": "{:,}",
    "Train time (s)": "{:.2f}",
    "Predict time (s)": "{:.3f}",
}))

metric_columns = [
    "Accuracy",
    "Macro Precision",
    "Macro Recall",
    "Macro F1",
```

```
]
ax = results_df.set_index("Model")[metric_columns].plot(
    kind="bar",
    figsize=(13, 6),
)
ax.set_title(
    f"So sánh ba cách cài đặt CNN trên {DATASET_NAME}"
)
ax.set_ylabel("Score")
ax.set_ylim(0, 1)
plt.xticks(rotation=15, ha="right")
plt.tight_layout()
plt.show()
```

Output của cell:

```
index=0; Model=PyTorch CNN; Accuracy=0.9956; Macro Precision=0.9957; Macro
↪  Recall=0.9955; Macro F1=0.9956
  Train samples=54,000; Best epoch=14; Final LR=0.000090;
   ↪  Parameters=468,010; Train time (s)=100.57; Predict time (s)=0.353
index=1; Model=TensorFlow/Keras CNN; Accuracy=0.9955; Macro
↪  Precision=0.9955; Macro Recall=0.9955; Macro F1=0.9955
  Train samples=54,000; Best epoch=13; Final LR=0.000027;
   ↪  Parameters=468,394; Train time (s)=482.54; Predict time (s)=1.509
index=2; Model=NumPy CNN from scratch; Accuracy=0.9850; Macro
↪  Precision=0.9848; Macro Recall=0.9849; Macro F1=0.9848
  Train samples=20,000; Best epoch=13; Final LR=0.000050;
   ↪  Parameters=52,138; Train time (s)=98.05; Predict time (s)=1.375
```

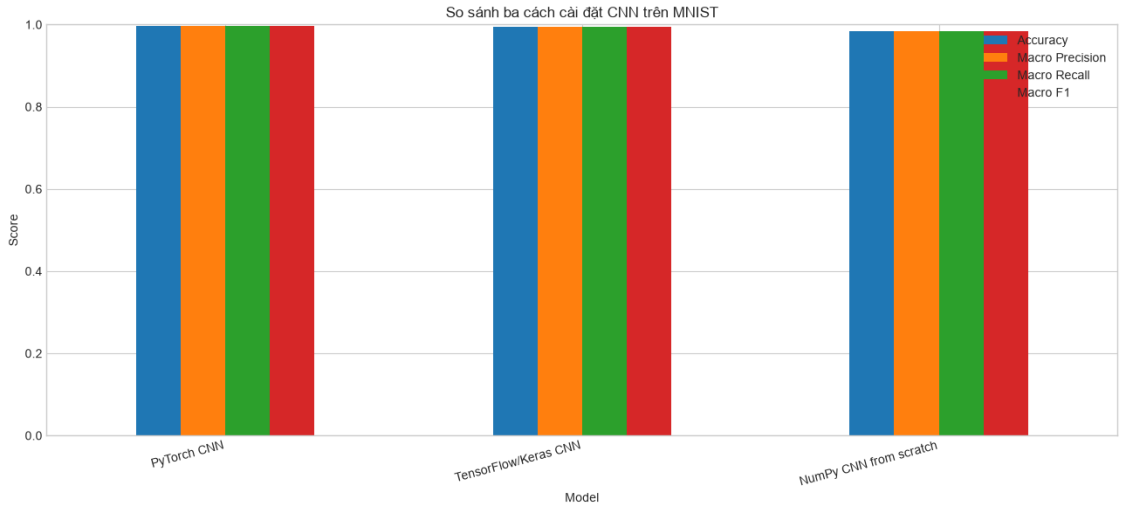
Output hình của cell: Hình 3.5.

Bảng 3.2: Ba CNN trên full test set MNIST

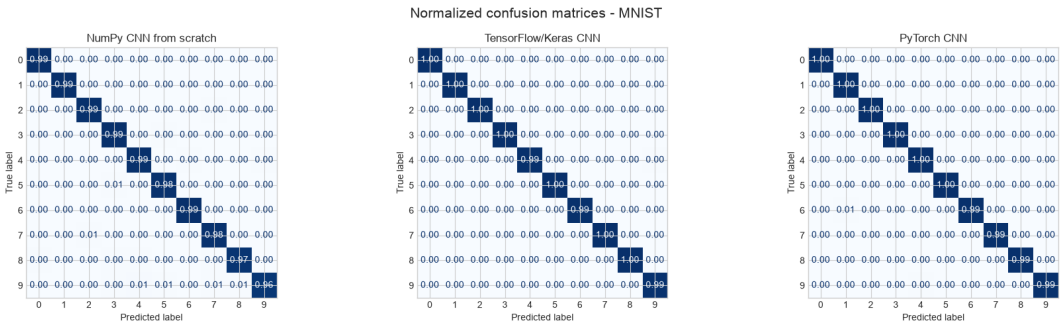
Mô hình	Accuracy	Macro-P	Macro-R	Macro-F1	Tham số	Train (s)
PyTorch CNN	**0.9956**	**0.9957**	**0.9955**	**0.9956**	468.010	100.57
Keras CNN	0.9955	0.9955	0.9955	0.9955	468.394	482.54
NumPy CNN	0.9850	0.9848	0.9849	0.9848	**52.138**	**98.05**

PyTorch cao hơn Keras 0.0001 Macro-F1, một chênh lệch nhỏ trên tập 10.000 ảnh. CNN NumPy thấp hơn khoảng 1,1 điểm phần trăm nhưng dùng ít ảnh train hơn, mạng nhỏ hơn và chỉ có primitive tự cài đặt. Kết quả 0.9848 vẫn xác nhận pipeline convolution/backprop thủ công hoạt động hiệu quả trên dữ liệu đơn giản. Nếu mục tiêu là học thuật toán, NumPy có giá trị giải thích; nếu mục tiêu là độ chính xác cao với công sức triển khai nhỏ hơn, framework phù hợp hơn.

Ma trận nhầm lẫn cho biết cặp chữ số nào dễ bị đánh đồng; ảnh dự đoán sai cần được xem trực tiếp vì một số nét viết vốn mơ hồ ngay cả đối với con người. Độ tin cậy softmax cao không đồng nghĩa xác suất đã được hiệu chỉnh tốt. Nếu dùng CNN trong hệ nhập liệu thật, ảnh



Hình 3.5: So sánh Accuracy, Macro-Precision, Macro-Recall và Macro-F1 trên MNIST.



Hình 3.6: Ma trận nhầm lẫn chuẩn hóa của ba CNN MNIST.

chụp từ điện thoại, giấy có đường kẻ hoặc chữ viết sát mép có thể khác đáng kể với ảnh MNIST chuẩn.



Hình 3.7: Giao diện deploy MNIST với chữ số đầu vào, mô hình và kết quả phân loại.

3.3 CIFAR-10: ảnh vật thể tự nhiên

Mã nguồn GitHub: https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2004/cifar10

Nguồn dữ liệu: https://www.cs.toronto.edu/~kriz/cifar.html

CIFAR-10 có 50.000 ảnh train gốc và 10.000 ảnh test, kích thước $32 \times 32 \times 3$. Mười nhãn là máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu và xe tải. Mỗi lớp có đúng 5.000 ảnh train và 1.000 ảnh test [9]. Một mẫu đầu vào là ảnh màu độ phân giải thấp; output là một trong mười nhãn. Khác MNIST, vật thể có thể nhỏ, bị che, xuất hiện trên nền phức tạp hoặc có hình dạng gần giống lớp khác.

Cell 14 — EDA (notebook gốc).

```
# Hiển thị một ảnh đại diện cho mỗi lớp.
fig, axes = plt.subplots(2, 5, figsize=(13, 5))
for label, ax in enumerate(axes.ravel()):
    image = x_train_full[np.flatnonzero(y_train_full == label)[0]]
    if image.shape[-1] == 1:
        ax.imshow(image.squeeze(-1), cmap="gray")
    else:
        ax.imshow(image)
    ax.set_title(f"{label}: {CLASS_NAMES[label]}")
    ax.axis("off")
fig.suptitle(f"Ảnh mẫu của 10 lớp - {DATASET_NAME}", fontsize=14)
plt.tight_layout()
plt.show()

train_counts = np.bincount(y_train_full, minlength=10)
test_counts = np.bincount(y_test_full, minlength=10)
distribution_df = pd.DataFrame({
    "class_id": np.arange(10),
    "class_name": CLASS_NAMES,
    "train_count": train_counts,
    "test_count": test_counts,
})
display(distribution_df)

ax = distribution_df.set_index("class_name")[["train_count",
↪  "test_count"]].plot(
    kind="bar", figsize=(12, 5), color=["#4C78A8", "#F58518"]
)
ax.set_title(f"Phân phối lớp của {DATASET_NAME}")
ax.set_xlabel("Lớp")
ax.set_ylabel("Số ảnh")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

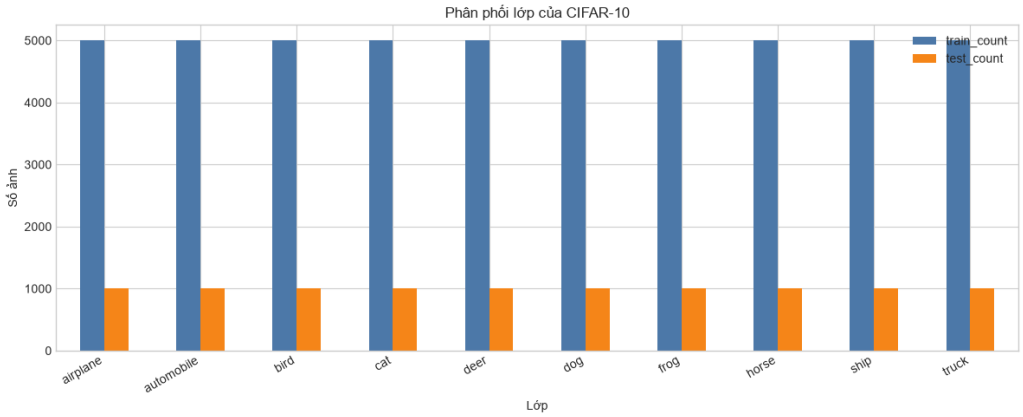
Output của cell:

```
index=0; class_id=0; class_name=airplane; train_count=5000;
↪  test_count=1000
index=1; class_id=1; class_name=automobile; train_count=5000;
↪  test_count=1000
index=2; class_id=2; class_name=bird; train_count=5000; test_count=1000
index=3; class_id=3; class_name=cat; train_count=5000; test_count=1000
index=4; class_id=4; class_name=deer; train_count=5000; test_count=1000
index=5; class_id=5; class_name=dog; train_count=5000; test_count=1000
index=6; class_id=6; class_name=frog; train_count=5000; test_count=1000
index=7; class_id=7; class_name=horse; train_count=5000; test_count=1000
index=8; class_id=8; class_name=ship; train_count=5000; test_count=1000
index=9; class_id=9; class_name=truck; train_count=5000; test_count=1000
```

Output hình của cell: Hình 3.8, Hình 3.9.



Hình 3.8: Ví dụ ảnh của từng lớp CIFAR-10: cùng một nhãn vẫn có nhiều góc nhìn và nền khác nhau.



Hình 3.9: Phân bố cân bằng của CIFAR-10 trên train và test.

Notebook tách 45.000 train và 5.000 validation cho PyTorch/Keras. NumPy chỉ dùng 15.000 train và 3.000 validation. Các giá trị trung bình và độ lệch chuẩn của ba channel được fit trên train rồi tái dùng. NumPy CNN có hai block với 12 và 24 filter, tầng ẩn 128 và 200.978 tham số. Hai framework dùng ResNet-20 với ba stage kênh $16 \to 32 \to 64$ và các residual block; số tham số khoảng 272–274 nghìn. Các kỹ thuật crop, lật ngang, label smoothing và lịch learning rate giúp hai mạng framework khai thác dữ liệu phong phú hơn.

Cell 26 — huấn luyện CNN NumPy (notebook gốc).

```
numpy_model = NumpyCNN(
    input_shape=x_train_numpy.shape[1:],
    conv_filters=NUMPY_FILTERS,
    hidden_units=NUMPY_HIDDEN_UNITS,
    num_classes=10,
    random_state=RANDOM_STATE,
)
print("NumPy CNN parameters:", f"{numpy_model.parameter_count():,}")
```

```
start = time.perf_counter()
numpy_history = train_numpy_cnn(
    numpy_model,
    x_train_numpy,
    y_train_numpy,
    x_val_numpy,
    y_val_numpy,
    epochs=NUMPY_EPOCHS,
    batch_size=NUMPY_BATCH_SIZE,
    learning_rate=NUMPY_LEARNING_RATE,
    weight_decay=NUMPY_WEIGHT_DECAY,
    dataset_key=DATASET_KEY,
    random_state=RANDOM_STATE,
)
numpy_train_time = time.perf_counter() - start

start = time.perf_counter()
numpy_test_probabilities = predict_numpy_proba(
    numpy_model,
    x_test,
    batch_size=NUMPY_BATCH_SIZE,
)
numpy_predict_time = time.perf_counter() - start

benchmark_results.append(evaluate_predictions(
    "NumPy CNN from scratch",
    y_test,
    numpy_test_probabilities,
    numpy_train_time,
    numpy_predict_time,
    numpy_model.parameter_count(),
    len(y_train_numpy),
    numpy_history["best_epoch"],
    numpy_history["learning_rate"][-1],
))
prediction_store["NumPy CNN from scratch"] = numpy_test_probabilities
history_store["NumPy CNN from scratch"] = numpy_history
print(
    f"Train time: {numpy_train_time:.2f}s | Test accuracy: "
    f"{accuracy_score(y_test, np.argmax(numpy_test_probabilities,
    ↪  axis=1)):.4f}"
)
```

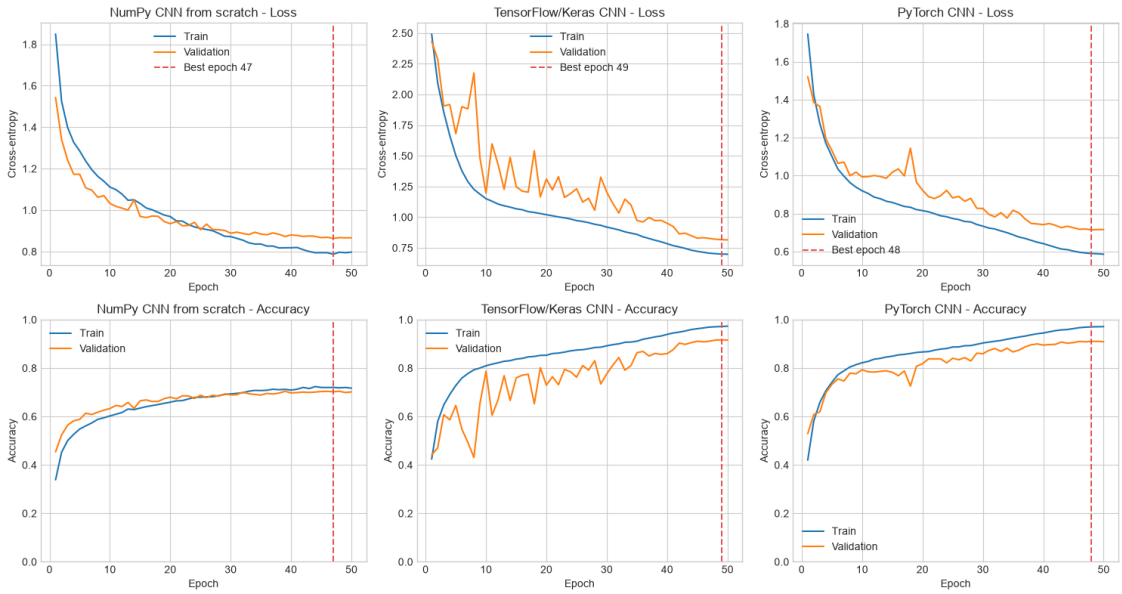
Output của cell:

```
NumPy CNN parameters: 200,978
Epoch 001/50 | lr=0.001000 | loss=1.8495 | acc=0.3387 | val_loss=1.5432 |
↪  val_acc=0.4543
Epoch 002/50 | lr=0.000999 | loss=1.5245 | acc=0.4527 | val_loss=1.3395 |
↪  val_acc=0.5243
Epoch 003/50 | lr=0.000996 | loss=1.3982 | acc=0.5005 | val_loss=1.2409 |
↪  val_acc=0.5643
```

```
Epoch 004/50 | lr=0.000991 | loss=1.3271 | acc=0.5263 | val_loss=1.1730 |
↪  val_acc=0.5820
Epoch 005/50 | lr=0.000984 | loss=1.2850 | acc=0.5483 | val_loss=1.1727 |
↪  val_acc=0.5883
Epoch 006/50 | lr=0.000976 | loss=1.2364 | acc=0.5613 | val_loss=1.1071 |
↪  val_acc=0.6137
Epoch 007/50 | lr=0.000965 | loss=1.1947 | acc=0.5735 | val_loss=1.0963 |
↪  val_acc=0.6083
Epoch 008/50 | lr=0.000953 | loss=1.1627 | acc=0.5883 | val_loss=1.0621 |
↪  val_acc=0.6180
Epoch 009/50 | lr=0.000939 | loss=1.1391 | acc=0.5948 | val_loss=1.0696 |
↪  val_acc=0.6260
Epoch 010/50 | lr=0.000923 | loss=1.1111 | acc=0.6023 | val_loss=1.0322 |
↪  val_acc=0.6330
Epoch 011/50 | lr=0.000906 | loss=1.0985 | acc=0.6094 | val_loss=1.0179 |
↪  val_acc=0.6457
Epoch 012/50 | lr=0.000887 | loss=1.0774 | acc=0.6165 | val_loss=1.0094 |
↪  val_acc=0.6410
Epoch 013/50 | lr=0.000866 | loss=1.0474 | acc=0.6309 | val_loss=0.9998 |
↪  val_acc=0.6583
Epoch 014/50 | lr=0.000844 | loss=1.0497 | acc=0.6285 | val_loss=1.0491 |
↪  val_acc=0.6333
Epoch 015/50 | lr=0.000821 | loss=1.0318 | acc=0.6347 | val_loss=0.9695 |
↪  val_acc=0.6647
Epoch 016/50 | lr=0.000797 | loss=1.0104 | acc=0.6406 | val_loss=0.9642 |
↪  val_acc=0.6683
Epoch 017/50 | lr=0.000771 | loss=1.0014 | acc=0.6453 | val_loss=0.9713 |
↪  val_acc=0.6620
Epoch 018/50 | lr=0.000745 | loss=0.9884 | acc=0.6493 | val_loss=0.9701 |
↪  val_acc=0.6623
Epoch 019/50 | lr=0.000717 | loss=0.9759 | acc=0.6547 | val_loss=0.9446 |
↪  val_acc=0.6740
Epoch 020/50 | lr=0.000689 | loss=0.9687 | acc=0.6591 | val_loss=0.9344 |
↪  val_acc=0.6797
Epoch 021/50 | lr=0.000660 | loss=0.9480 | acc=0.6647 | val_loss=0.9448 |
↪  val_acc=0.6733
Epoch 022/50 | lr=0.000631 | loss=0.9469 | acc=0.6658 | val_loss=0.9240 |
↪  val_acc=0.6853
Epoch 023/50 | lr=0.000601 | loss=0.9302 | acc=0.6725 | val_loss=0.9258 |
↪  val_acc=0.6850
Epoch 024/50 | lr=0.000571 | loss=0.9189 | acc=0.6785 | val_loss=0.9410 |
↪  val_acc=0.6743
Epoch 025/50 | lr=0.000540 | loss=0.9115 | acc=0.6800 | val_loss=0.9052 |
↪  val_acc=0.6877
Epoch 026/50 | lr=0.000510 | loss=0.9064 | acc=0.6810 | val_loss=0.9326 |
↪  val_acc=0.6783
Epoch 027/50 | lr=0.000479 | loss=0.9005 | acc=0.6817 | val_loss=0.9063 |
↪  val_acc=0.6877
Epoch 028/50 | lr=0.000449 | loss=0.8878 | acc=0.6859 | val_loss=0.9055 |
↪  val_acc=0.6840
Epoch 029/50 | lr=0.000419 | loss=0.8738 | acc=0.6912 | val_loss=0.9018 |
↪  val_acc=0.6930
```

```
Epoch 030/50 | lr=0.000390 | loss=0.8722 | acc=0.6935 | val_loss=0.8884 |
↪  val_acc=0.6893
Epoch 031/50 | lr=0.000361 | loss=0.8637 | acc=0.6956 | val_loss=0.8935 |
↪  val_acc=0.6900
Epoch 032/50 | lr=0.000333 | loss=0.8551 | acc=0.6981 | val_loss=0.8874 |
↪  val_acc=0.7000
Epoch 033/50 | lr=0.000305 | loss=0.8421 | acc=0.7045 | val_loss=0.8818 |
↪  val_acc=0.6940
Epoch 034/50 | lr=0.000279 | loss=0.8361 | acc=0.7079 | val_loss=0.8932 |
↪  val_acc=0.6910
Epoch 035/50 | lr=0.000253 | loss=0.8362 | acc=0.7073 | val_loss=0.8845 |
↪  val_acc=0.6887
Epoch 036/50 | lr=0.000229 | loss=0.8265 | acc=0.7085 | val_loss=0.8809 |
↪  val_acc=0.6953
Epoch 037/50 | lr=0.000206 | loss=0.8262 | acc=0.7126 | val_loss=0.8899 |
↪  val_acc=0.6930
Epoch 038/50 | lr=0.000184 | loss=0.8174 | acc=0.7107 | val_loss=0.8830 |
↪  val_acc=0.6967
Epoch 039/50 | lr=0.000163 | loss=0.8181 | acc=0.7121 | val_loss=0.8727 |
↪  val_acc=0.7040
Epoch 040/50 | lr=0.000144 | loss=0.8183 | acc=0.7092 | val_loss=0.8800 |
↪  val_acc=0.6973
Epoch 041/50 | lr=0.000127 | loss=0.8196 | acc=0.7127 | val_loss=0.8779 |
↪  val_acc=0.6987
Epoch 042/50 | lr=0.000111 | loss=0.8088 | acc=0.7201 | val_loss=0.8729 |
↪  val_acc=0.7010
Epoch 043/50 | lr=0.000097 | loss=0.8001 | acc=0.7155 | val_loss=0.8753 |
↪  val_acc=0.6993
Epoch 044/50 | lr=0.000085 | loss=0.7943 | acc=0.7235 | val_loss=0.8731 |
↪  val_acc=0.7010
Epoch 045/50 | lr=0.000074 | loss=0.7945 | acc=0.7200 | val_loss=0.8669 |
↪  val_acc=0.7033
Epoch 046/50 | lr=0.000066 | loss=0.7943 | acc=0.7198 | val_loss=0.8687 |
↪  val_acc=0.7040
Epoch 047/50 | lr=0.000059 | loss=0.7872 | acc=0.7201 | val_loss=0.8634 |
↪  val_acc=0.7023
Epoch 048/50 | lr=0.000054 | loss=0.7968 | acc=0.7185 | val_loss=0.8669 |
↪  val_acc=0.7043
Epoch 049/50 | lr=0.000051 | loss=0.7943 | acc=0.7199 | val_loss=0.8659 |
↪  val_acc=0.6990
Epoch 050/50 | lr=0.000050 | loss=0.7972 | acc=0.7172 | val_loss=0.8663 |
↪  val_acc=0.7017
Restored NumPy weights from epoch 47 (val_loss=0.8634).
Train time: 478.93s | Test accuracy: 0.7012
```

CNN NumPy dừng gần vùng accuracy 0.70, trong khi hai mạng ResNet-20 vượt 0.90. Chênh lệch không thể quy cho một yếu tố. NumPy dùng ít ảnh hơn ba lần, kiến trúc nông hơn, không có residual connection và có chính sách tối ưu đơn giản hơn. Ngược lại, đường train/validation gần nhau của CNN NumPy gợi ý hạn chế thiên về năng lực biểu diễn và dữ liệu, hơn là overfitting nghiêm trọng. Với ảnh tự nhiên, receptive field lớn và kết hợp đặc trưng qua nhiều tầng tạo lợi thế rõ.



Hình 3.10: Đường học của ba CNN trên CIFAR-10; validation cho thấy độ khó lớn hơn MNIST.

Cell 42 — tổng hợp kết quả (notebook gốc).

```
ordered_columns = [
    "Model",
    "Accuracy",
    "Macro Precision",
    "Macro Recall",
    "Macro F1",
    "Train samples",
    "Best epoch",
    "Final LR",
    "Parameters",
    "Train time (s)",
    "Predict time (s)",
]
results_df = pd.DataFrame(benchmark_results)[ordered_columns]
results_df = results_df.sort_values(
    ["Macro F1", "Accuracy"],
    ascending=False,
).reset_index(drop=True)
display(results_df.style.format({
    "Accuracy": "{:.4f}",
    "Macro Precision": "{:.4f}",
    "Macro Recall": "{:.4f}",
    "Macro F1": "{:.4f}",
    "Train samples": "{:,}",
    "Best epoch": "{:d}",
    "Final LR": "{:.6f}",
    "Parameters": "{:,}",
    "Train time (s)": "{:.2f}",
    "Predict time (s)": "{:.3f}",
}))
```

```
metric_columns = [
    "Accuracy",
    "Macro Precision",
    "Macro Recall",
    "Macro F1",
]
ax = results_df.set_index("Model")[metric_columns].plot(
    kind="bar",
    figsize=(13, 6),
)
ax.set_title(
    f"So sánh ba cách cài đặt CNN trên {DATASET_NAME}"
)
ax.set_ylabel("Score")
ax.set_ylim(0, 1)
plt.xticks(rotation=15, ha="right")
plt.tight_layout()
plt.show()
```

Output của cell:

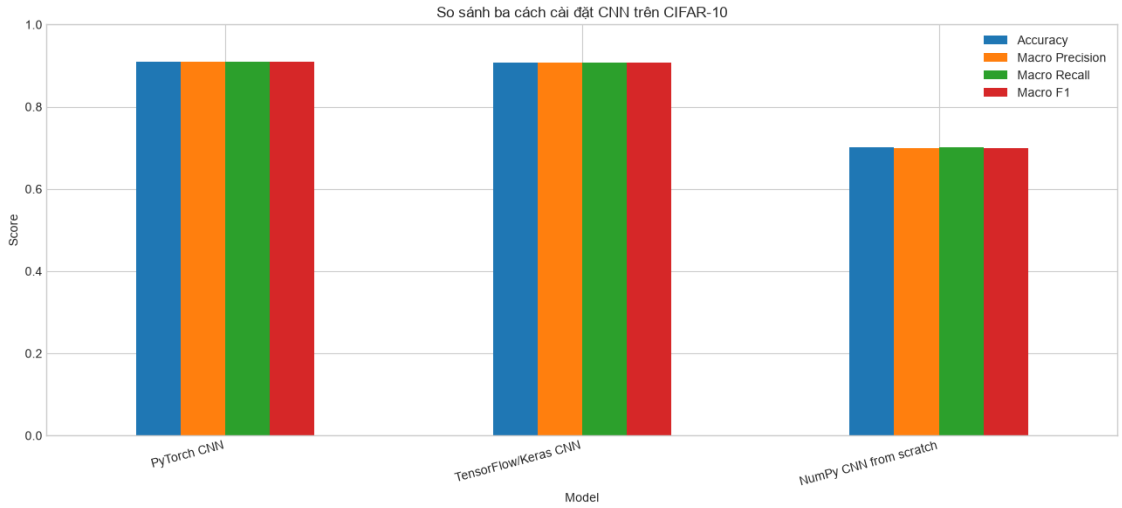
```
index=0; Model=PyTorch CNN; Accuracy=0.9151; Macro Precision=0.9150; Macro
↪  Recall=0.9151; Macro F1=0.9149
  Train samples=45,000; Best epoch=50; Final LR=0.000222;
  ↪  Parameters=272,474; Train time (s)=578.99; Predict time (s)=1.056
index=1; Model=TensorFlow/Keras CNN; Accuracy=0.9078; Macro
↪  Precision=0.9077; Macro Recall=0.9078; Macro F1=0.9075
  Train samples=45,000; Best epoch=49; Final LR=0.000222;
  ↪  Parameters=274,042; Train time (s)=2996.91; Predict time (s)=5.528
index=2; Model=NumPy CNN from scratch; Accuracy=0.7012; Macro
↪  Precision=0.6987; Macro Recall=0.7012; Macro F1=0.6994
  Train samples=15,000; Best epoch=47; Final LR=0.000050;
  ↪  Parameters=200,978; Train time (s)=478.93; Predict time (s)=2.555
```

Output hình của cell: Hình 3.11.

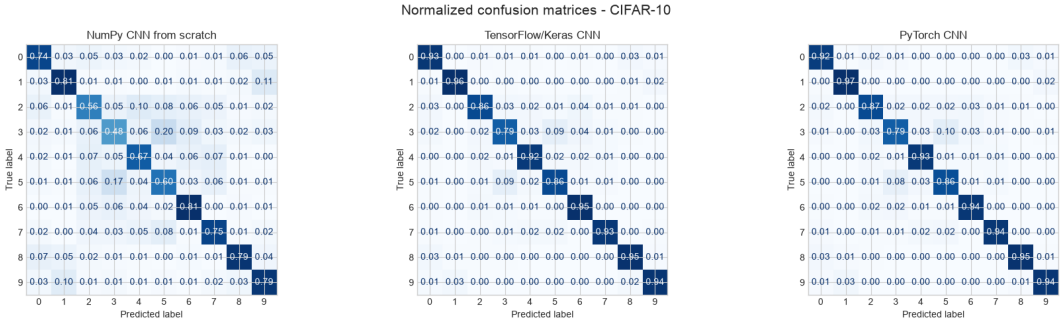
Bảng 3.3: Ba CNN trên full test set CIFAR-10

Mô hình	Accuracy	Macro-P	Macro-R	Macro-F1	Tham số	Train (s)
PyTorch CNN	**0.9151**	**0.9150**	**0.9151**	**0.9149**	272.474	578.99
Keras CNN	0.9078	0.9077	0.9078	0.9075	274.042	2996.91
NumPy CNN	0.7012	0.6987	0.7012	0.6994	**200.978**	**478.93**

Hai ResNet-20 gần nhau về chất lượng: PyTorch hơn Keras 0.0074 Macro-F1 trong lần chạy được lưu. Chênh lệch thời gian lớn hơn, song phụ thuộc thiết bị MPS, backend và điều kiện máy tại thời điểm huấn luyện. Không nên suy ra PyTorch luôn nhanh hơn Keras. NumPy thấp hơn rõ, nhưng đạt hơn 70% accuracy trên bài toán khó hơn MNIST; đó là một kết quả phù hợp cho kiến trúc thủ công nhỏ.



Hình 3.11: So sánh các metric phân loại CIFAR-10 trên cùng full test set.



Hình 3.12: Ma trận nhầm lẫn theo lớp của ba CNN CIFAR-10.

Phân tích lỗi theo lớp là cần thiết dù tập cân bằng. Mèo và chó, ô tô và xe tải có cấu trúc ảnh gần nhau hơn các cặp rất khác như tàu và ếch. Mẫu dự đoán sai với confidence cao có thể là ảnh mờ, vật thể nhỏ hoặc có nhiều vật thể. Khi triển khai ảnh người dùng tải lên, còn cần xử lý resize, kênh màu và chuẩn hóa giống hệt notebook; chỉ cần đổi thứ tự RGB/BGR cũng có thể làm kết quả lệch đáng kể.



Hình 3.13: Giao diện deploy CIFAR-10 với ảnh đầu vào, mô hình và nhãn dự đoán.

3.4 Phân tích lỗi và lựa chọn mô hình khi sử dụng

Đối với một hệ phân loại ảnh, accuracy trung bình chỉ là điểm khởi đầu. Ma trận nhầm lẫn chuẩn hóa cho biết mô hình hay nhầm lớp nào, nhưng mỗi ô chỉ là số lượng tổng hợp. Bước tiếp theo là xem những ảnh sai đại diện: vật thể có bị che khuất không, nhãn có mơ hồ không, ảnh có quá tối hoặc quá nhỏ không. Nếu lỗi tập trung ở một kiểu ảnh, giải pháp có thể là dữ liệu augmentation phù hợp, điều chỉnh cách resize hoặc bổ sung ví dụ, chứ không nhất thiết là tăng số tầng mạng. Tập CIFAR-10 chỉ 32×32 pixel nên nhiều chi tiết nhận dạng vốn đã bị mất trước khi đưa vào mô hình.

Mối quan hệ giữa confidence và độ đúng cũng đáng kiểm tra. Khi softmax cho lớp A giá trị 0.95, người dùng dễ đọc thành xác suất 95% mô hình đúng; cách hiểu đó chỉ hợp lệ khi mô hình được hiệu chỉnh tốt trên phân bố đầu vào tương ứng. Dữ liệu ngoài miền có thể khiến các logit cực đoan dù dự đoán sai. Một giao diện deploy nên hiển thị nhãn cùng confidence như thông tin hỗ trợ, đồng thời cho phép người dùng kiểm tra lại ảnh đầu vào. Đối với bài MNIST, vùng vẽ hoặc ảnh tải lên cần được xử lý giống tiền xử lý train; đối với CIFAR-10, phải giữ ba kênh màu đúng thứ tự.

Việc chọn mô hình cuối cùng phụ thuộc mục tiêu. PyTorch đứng đầu một chút trên cả hai dataset trong lần chạy đã lưu, nhưng Keras gần như tương đương về Macro-F1. Nếu ứng dụng chỉ cần nhận dạng chữ số ở chất lượng đã đủ, CNN NumPy nhỏ hơn có thể hữu ích cho mục đích giáo dục hoặc môi trường rất giới hạn, miễn là chi phí tự bảo trì phù hợp. Nếu ứng dụng phân loại ảnh vật thể tự nhiên, khoảng cách hơn 20 điểm phần trăm giữa NumPy CNN và hai ResNet-20 là đáng kể. Dù vậy, trước khi triển khai ngoài phạm vi bài tập, cần thử trên ảnh từ đúng nguồn sử dụng, không chỉ trên tập test chuẩn.

3.5 Tổng hợp Chương 3

MNIST và CIFAR-10 cùng có mười lớp nhưng độ khó rất khác nhau. Trên MNIST, CNN nhỏ đã gần đạt trần chất lượng và chênh lệch giữa hai framework rất nhỏ. Trên CIFAR-10, lợi ích của kiến trúc sâu, residual connection và toàn bộ tập train thể hiện rõ hơn. Các bảng benchmark cần được đọc cùng Bảng 3.1: NumPy chỉ dùng subset train, nên ba số trong một hàng không đại diện cho ba mô hình được cung cấp điều kiện huấn luyện giống hệt. Giao diện deploy kiểm chứng đường đi từ ảnh tới nhãn, còn bộ test độc lập mới đo được khả năng tổng quát hóa trong phạm vi dữ liệu chuẩn.

Chương 4

Mạng nơ-ron hồi tiếp

4.1 Chuỗi thời gian và trạng thái ẩn

Dữ liệu chuỗi thời gian gồm các quan sát có thứ tự. Dự báo giá phiên kế tiếp từ 60 phiên trước không giống phân loại các dòng độc lập: vị trí của mỗi giá trị và khoảng cách thời gian giữa các giá trị đều có ý nghĩa. RNN dùng một trạng thái ẩn h_t để tóm tắt quá khứ khi đọc từng phần tử x_t:

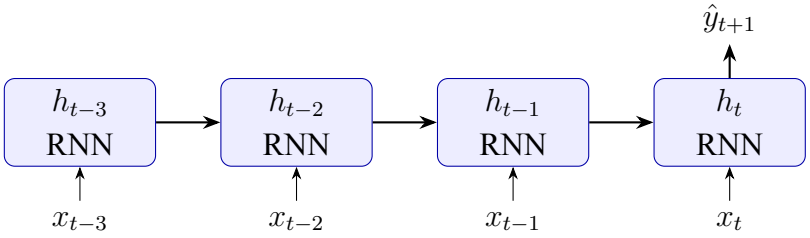
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h), \tag{4.1}$$

$$\hat{y}_{t+1} = W_y h_t + b_y. \tag{4.2}$$

Trọng số được chia sẻ qua mọi bước thời gian, nên cùng một cơ chế cập nhật có thể áp dụng cho chuỗi có độ dài khác nhau. Trong Assignment 06, input là cửa sổ giá một chiều; output là một dự báo giá cho bước kế tiếp. Cửa sổ trượt biến một chuỗi $z_1, \ldots, z_T$ thành các cặp $(z_{t-L:t-1}, z_t)$ với độ dài nhìn lại L.

Ở đây x_t là vector đầu vào tại bước t, h_{t-1} là tóm tắt quá khứ trước khi đọc x_t, còn h_t là trạng thái mới. W_x biến đổi đầu vào, W_h biến đổi trạng thái cũ, b_h là bias; hàm tanh giữ giá trị trạng thái trong khoảng $(-1, 1)$. Với dự báo một bước, $W_y h_t + b_y$ là đầu ra tuyến tính trên thang đã chuẩn hóa. Các ma trận W_x, W_h, W_y được dùng lặp lại cho mọi t chứ không tạo một bộ trọng số riêng cho mỗi ngày. Trạng thái ban đầu thường là vector không hoặc được học; nó ảnh hưởng mạnh ở đầu cửa sổ nhưng tác động có thể yếu dần khi chuỗi dài.

Hình 4.1 cho thấy cùng một khối RNN khi "trải" theo thời gian. Mũi tên ngang mang trạng thái, mũi tên dưới đưa quan sát ở từng bước vào và đầu ra chỉ được lấy sau bước cuối trong bài dự báo many-to-one. Nếu nhìn lại $L = 60$ phiên Amazon, mô hình đọc 60 giá trị theo thứ tự trước khi dự báo phiên thứ 61; nó không được nhìn giá của phiên cần dự báo. Nếu thay L, số tham số của khối RNN không nhất thiết đổi, nhưng lượng ngữ cảnh và số cửa sổ học hợp lệ sẽ đổi.

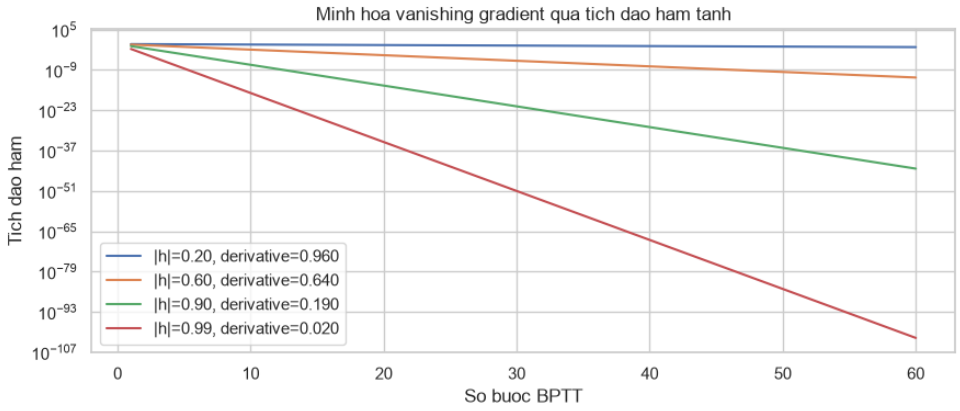


Hình 4.1: Một RNN dùng chung trọng số tại các bước để dự báo sau cửa sổ quan sát.

RNN được huấn luyện bằng Backpropagation Through Time: mạng được "trải" ra theo số bước L, rồi gradient đi ngược qua các trạng thái. Khi nhân liên tiếp nhiều ma trận Jacobian, gradient có thể nhỏ dần hoặc tăng vọt; đây là khó khăn của phụ thuộc dài hạn [8]. Gradient clipping giới hạn bước cập nhật quá lớn. LSTM và GRU thêm các cổng học cách giữ hoặc quên thông tin, nhưng notebook của bài này dùng vanilla RNN để so sánh hai framework trên cùng kiến trúc. Vì vậy các kết luận thực nghiệm dưới đây không được suy rộng sang LSTM, GRU hay Transformer.

Theo quy tắc dây chuyền, ảnh hưởng của trạng thái cũ lên trạng thái mới đi qua tích các đạo hàm $\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$ theo đúng thứ tự thời gian. Mỗi thừa số phụ thuộc W_h và đạo hàm của tanh. Nếu chuẩn của nhiều thừa số liên tiếp nhỏ hơn 1, tín hiệu từ bước xa có thể suy giảm; nếu một số hướng liên tục khuếch đại, gradient có thể bùng nổ. Đó là hiện tượng liên quan đến chuỗi đạo hàm, không có nghĩa mọi RNN đều quên hoàn toàn sau một số bước cố định. Với ngưỡng clipping c, cách cắt theo chuẩn là $\tilde{g} = g \min(1, c/\|g\|_2)$: gradient nhỏ giữ nguyên, gradient lớn được co về chuẩn c.

LSTM đưa vào một đường trạng thái ô nhớ và các cổng quên, ghi, xuất; GRU gộp bớt cổng và dùng trạng thái ẩn trực tiếp. Cổng là giá trị được học trong khoảng 0–1, điều chỉnh tỷ lệ thông tin cũ được giữ và thông tin mới được thêm. Chúng làm việc với phụ thuộc dài dễ hơn vanilla RNN trong nhiều bối cảnh, nhưng tăng số phép tính và tham số. Trong tiểu luận này, chúng chỉ là nền tảng lý thuyết so sánh; các bảng kết quả ở Chương 4 không chứa thử nghiệm LSTM hay GRU.



Hình 4.2: Minh họa sự suy giảm gradient khi truyền qua nhiều bước thời gian.

4.1.1 Thiết kế dự báo và chống rò rỉ tương lai

Tách ngẫu nhiên các cửa sổ time series có thể để những quan sát tương lai xuất hiện trong train trong khi quá khứ của chúng nằm ở test. Notebook vì thế chia theo thời gian, xấp xỉ 80% train, 10% validation và 10% test. StandardScaler chỉ fit trên đoạn train. Với Avocado, scaler được fit riêng cho từng chuỗi region–type để tránh trộn thang giá giữa các thị trường. Validation dùng để chọn checkpoint và dừng sớm; test chỉ dùng để công bố kết quả cuối.

Ranh giới thời gian phải được xác định trên chuỗi gốc trước khi tạo hoặc gán cửa sổ cho các tập. Một cửa sổ test có thể dùng vài quan sát *quá khứ* thuộc đoạn trước test làm đầu vào vì tại thời điểm dự báo chúng đã biết; nhãn của cửa sổ vẫn nằm trong test. Ngược lại, fit StandardScaler trên toàn chuỗi sẽ đưa mean và độ lệch chuẩn của tương lai vào train. Với dữ liệu Avocado gồm nhiều chuỗi region–type, nhóm cũng là một ranh giới: không được ghép tuần của hai nhóm thành một cửa sổ chỉ vì chúng đứng cạnh nhau trong file CSV.

Bảng 4.1: Nhiệm vụ dự báo và cách chia dữ liệu RNN

Dataset	Số dòng	Cửa sổ	Train/Val/Test	Target và bước dự báo
Amazon Stock Price	6.684	60	5.299/662/663	`Close`, phiên giao dịch kế tiếp
Avocado Price	18.249	12	13.498/1.727/1.728	`AveragePrice`, tuần kế tiếp theo nhóm

Ba mô hình được so sánh là persistence baseline, PyTorch RNN và Keras RNN. Persistence dự báo $\hat{y}_{t+1} = y_t$, nghĩa là giá tiếp theo bằng giá mới nhất trong cửa sổ. Baseline không có thời gian train nhưng rất mạnh với chuỗi mức giá biến đổi chậm. Nếu mô hình phức tạp không vượt được baseline này, độ phức tạp tăng thêm chưa đem lại lợi ích dự báo trong phép thử hiện tại. PyTorch và Keras cùng cấu trúc RNN 64 hidden units, activation tanh, dropout 0.2 và đầu ra tuyến tính; tối ưu bằng Adam với MSE loss, early stopping và khôi phục trọng số tốt nhất trên validation.

Các metric được tính sau khi inverse-transform về đơn vị giá gốc. MAE đo sai số tuyệt đối trung bình, RMSE nhấn mạnh dự báo lệch lớn, MAPE là sai số phần trăm trung bình và R^2 so với dự đoán trung bình của test. Với chuỗi giá có xu hướng mạnh, R^2 cao không đảm bảo mô hình dự báo tốt hơn persistence; phải đọc trực tiếp MAE và RMSE của baseline. Các con số thời gian train là số đo của lần chạy notebook, không phải so sánh tuyệt đối giữa hai thư viện.

Với nhãn thật y_i và dự báo $\hat{y}_i$, notebook tính $\text{MAPE}_{\%} = \frac{100}{n} \sum_i \frac{|y_i - \hat{y}_i|}{\max(|y_i|, 10^{-8})}$. Hằng số 10^{-8} tránh chia cho 0, nhưng chỉ số vẫn không ổn định khi giá thật rất gần 0; MAE và RMSE không có phép chia ấy. Ví dụ giá thật 100 và dự báo 110 cho lỗi tuyệt đối 10, lỗi tỷ lệ 10%; nếu giá thật 10 và dự báo 20, lỗi tuyệt đối vẫn 10 nhưng lỗi tỷ lệ là 100%. Vì vậy cần xem đơn vị và vùng giá của từng dataset trước khi so sánh metric.

Persistence dùng $\hat{y}_{t+1} = y_t$. Trên đoạn giá $[10, 11, 9, 12]$, các dự báo một bước lần lượt là 10, 11, 9 cho ba giá thật 11, 9, 12; sai số tuyệt đối là $[1, 2, 3]$, nên MAE bằng 2. Một RNN chỉ có ý nghĩa cải thiện trong phép thử này khi giảm lỗi so với mốc đơn giản trên cùng cửa sổ test và cùng phép đổi ngược thang giá. R^2 so với trung bình test trả lời câu hỏi khác, nên phải đọc cùng MAE/RMSE của persistence. Nếu dữ liệu đổi chế độ thị trường sau thời điểm train,

cả baseline lẫn RNN đều cần được đánh giá lại ở nhiều mốc thời gian.

4.1.2 Ví dụ về cửa sổ, chuẩn hóa và phép đo

Giả sử một chuỗi giá ngắn có các giá trị $[10, 11, 9, 12, 13]$ và lookback bằng 3. Hai cặp học là $([10, 11, 9], 12)$ và $([11, 9, 12], 13)$. Cửa sổ kế tiếp trượt một bước, không bỏ qua thứ tự. Nếu chuẩn hóa bằng mean μ và scale σ học trên train, giá trị đưa vào RNN là $(x - \mu)/\sigma$. Dự báo $\hat{z}$ của mạng phải được đổi ngược thành $\hat{x} = \hat{z}\sigma + \mu$ trước khi so với giá thật. Nếu tính MAE khi một bên ở thang chuẩn hóa còn bên kia ở thang giá, kết quả sẽ vô nghĩa.

Trong chuỗi thời gian, ranh giới train/validation/test có một chi tiết tinh tế. Một cửa sổ test có thể cần vài điểm quá khứ nằm trước ngày bắt đầu test để đủ lookback; việc dùng quá khứ đã biết tại thời điểm dự báo là hợp lệ. Điều không hợp lệ là fit scaler hoặc chọn hyperparameter bằng các giá tương lai của test. Cũng không nên xáo trộn cửa sổ khi xác định split, vì các cửa sổ kề nhau chia sẻ phần lớn đầu vào. Ở Avocado, còn phải giữ nguyên ranh giới nhóm: tuần cuối ở Albany không thể trở thành tuần đầu của một cửa sổ tại Boston.

Persistence baseline có thể được xem là trường hợp đặc biệt của mô hình dự báo không học tham số. Với giá y_t biến động chậm, dự báo $\hat{y}_{t+1} = y_t$ tạo sai số đúng bằng thay đổi một bước $|y_{t+1} - y_t|$. Vì thế độ khó của bài toán phụ thuộc ta dự báo giá mức, sai phân hay phần trăm thay đổi. Một mô hình dự báo mức giá có thể đạt R^2 cao chỉ vì mức giá trên tập test trải rộng; nhưng vẫn thua baseline theo MAE. Đây là lý do Bảng 4.2 không thể bị diễn giải bằng R^2 của RNN tách khỏi hàng persistence.

4.1.3 Đọc biểu đồ dự báo thay vì chỉ đọc bảng

Đường actual–predicted cho thấy mô hình có theo kịp các bước tăng giảm hay bị trễ pha. Residual $e_t = y_t - \hat{y}_t$ cho biết sai số có đổi dấu liên tục hay kéo dài thành một vùng thiên lệch. Nếu nhiều residual dương liên tiếp khi giá tăng, mô hình đang dự báo thấp có hệ thống trong giai đoạn ấy. RMSE sẽ tăng mạnh nếu thỉnh thoảng xuất hiện những đỉnh sai số rất lớn; MAE diễn tả lỗi điển hình ổn định hơn. Biểu đồ theo thời gian vì vậy bổ sung thông tin mà một metric tổng hợp không có.

Phân bố train/test có thể đổi theo thời gian, đặc biệt ở thị trường tài chính. Một mô hình học quan hệ từ nhiều năm cũ rồi dự báo những năm giá ở vùng mới đối mặt với distribution shift. Đánh giá walk-forward sẽ liên tục mở rộng hoặc trượt tập train, dự báo giai đoạn tiếp theo và gom lỗi qua nhiều mốc. Notebook hiện tại dùng một split cố định, nên kết luận chỉ phản ánh đúng split đã lưu. Đề xuất walk-forward là hướng đánh giá tiếp theo chứ không phải thí nghiệm đã hoàn thành.

4.2 Amazon Stock Price: dự báo giá đóng cửa

Mã nguồn GitHub: https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2006/Amazon%20Stock%20Price

Nguồn dữ liệu: https://finance.yahoo.com/quote/AMZN/history/

Tệp được dùng là `AMZN.csv` trong thư mục dữ liệu của Assignment 06; project không ghi lại URL tải chính xác của snapshot CSV này. Link ở trên là trang lịch sử giá AMZN để tham khảo, không được hiểu là bằng chứng về nguồn tải tệp. CSV có 6.684 phiên từ ngày 15/05/1997 đến 05/12/2023, với các cột Date, Open, High, Low, Close, Adj Close và Volume. Dòng đầu có Close bằng 0.097917 và Volume bằng 1.443.120.000; output của một cửa sổ 60 phiên là Close ở phiên kế tiếp. Trong bản CSV, Close thấp nhất khoảng 0.0698, trung vị 7.1048 và cao nhất 186.5705; khoảng biến thiên rất rộng phản ánh nhiều giai đoạn thị trường khác nhau.

Cell 16 — EDA (notebook gốc).

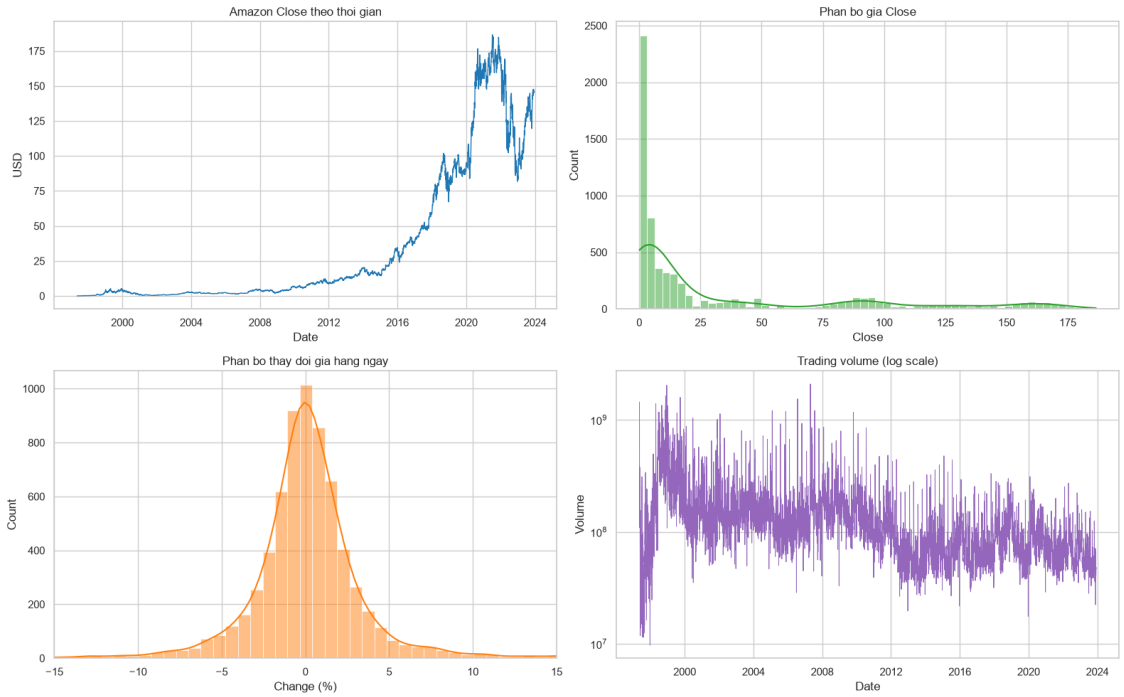
```
df["Close_change_pct"] = df["Close"].pct_change() * 100
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes[0, 0].plot(df["Date"], df["Close"], color="tab:blue", linewidth=1)
axes[0, 0].set(title="Amazon Close theo thoi gian", xlabel="Date",
↪  ylabel="USD")
sns.histplot(df["Close"], bins=60, kde=True, ax=axes[0, 1],
↪  color="tab:green")
axes[0, 1].set(title="Phan bo gia Close", xlabel="Close")
sns.histplot(df["Close_change_pct"].dropna(), bins=80, kde=True,
↪  ax=axes[1, 0], color="tab:orange")
axes[1, 0].set(title="Phan bo thay doi gia hang ngay", xlabel="Change
↪  (%)", xlim=(-15, 15))
axes[1, 1].plot(df["Date"], df["Volume"], color="tab:purple",
↪  linewidth=0.7)
axes[1, 1].set_yscale("log")
axes[1, 1].set(title="Trading volume (log scale)", xlabel="Date",
↪  ylabel="Volume")
plt.tight_layout()
plt.show()

yearly = df.set_index("Date").resample("YE").agg(Close_mean=("Close",
↪  "mean"), Close_std=("Close", "std"))
display(yearly.tail(10))
```

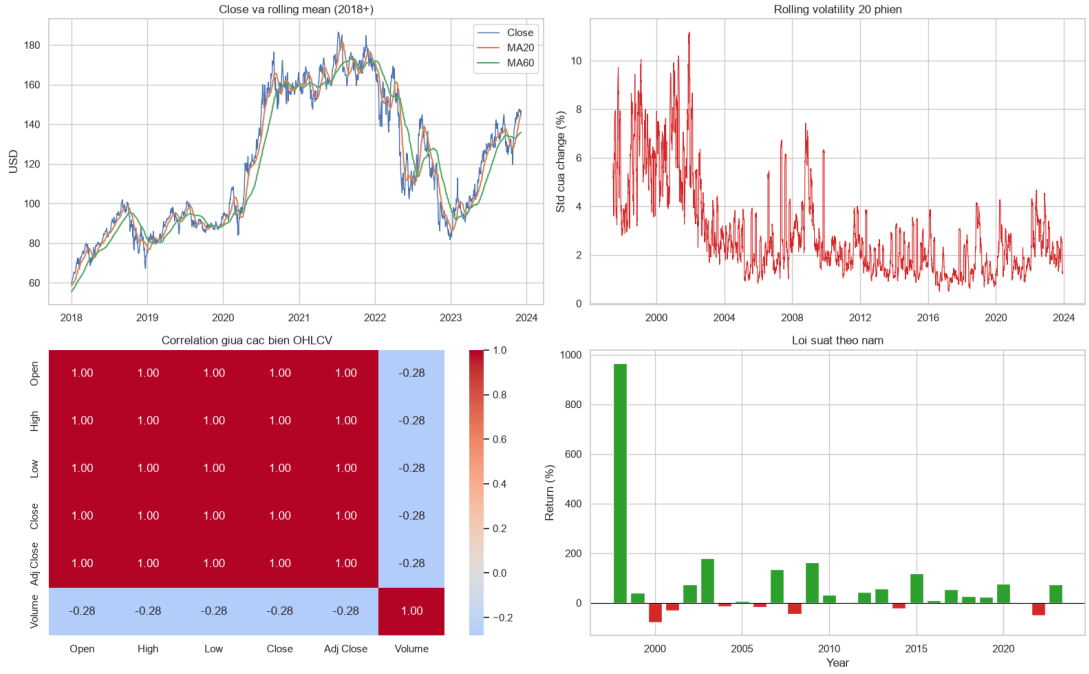
Output của cell:

```
index=2014-12-31; Close_mean=16.627552; Close_std=1.383261
index=2015-12-31; Close_mean=23.906915; Close_std=5.537250
index=2016-12-31; Close_mean=34.976157; Close_std=4.618116
index=2017-12-31; Close_mean=48.408351; Close_std=5.329010
index=2018-12-31; Close_mean=82.086309; Close_std=9.863754
index=2019-12-31; Close_mean=89.459460; Close_std=5.065682
index=2020-12-31; Close_mean=134.042755; Close_std=27.288052
index=2021-12-31; Close_mean=167.193349; Close_std=8.002098
index=2022-12-31; Close_mean=126.098819; Close_std=23.904315
index=2023-12-31; Close_mean=119.250945; Close_std=17.781396
```

Output hình của cell: Hình 4.3.



Hình 4.3: Chuỗi Close, phân phối mức giá, thay đổi ngày và volume trong dữ liệu Amazon.



Hình 4.4: Rolling statistics, tương quan các cột OHLCV và thay đổi theo năm.

Không có giá trị thiếu hay ngày trùng trong tệp Amazon. Các ngày cuối tuần và ngày nghỉ không có dòng; vì thế "bước tiếp theo" là phiên giao dịch kế tiếp, không phải ngày lịch tiếp theo. Các cột Open, High, Low, Close thường tương quan mạnh vì cùng một phiên. Notebook chỉ dùng Close trong mô hình RNN một biến; những biểu đồ OHLCV khác dùng cho khảo sát, không phải đầu vào của mô hình. Giá đóng cửa ở dạng mức có xu hướng dài hạn, còn phần trăm biến động ngày tập trung gần 0 và có đuôi dày. Điều này giải thích vì sao bài toán dự báo mức giá có thể khó ngoại suy khi test nằm ở vùng giá mới.

Sau lookback 60, notebook tạo 6.624 cửa sổ. StandardScaler fit trên phần train; mean khoảng 11.5805 và scale khoảng 17.4447. PyTorch train 107 epoch trong 101.50 giây; Keras train 75 epoch trong 18.88 giây. Trọng số tốt nhất của từng framework được nạp lại trước khi tính metric trên test. Các chi tiết này cần được giữ khi tái lập, bởi nếu fit scaler trên toàn chuỗi hoặc lấy trạng thái epoch cuối, kết quả có thể khác đáng kể.

Cell 24 — huấn luyện PyTorch RNN (notebook gốc).

```
train_loader = make_loader(X_train, y_train, shuffle=True)
val_loader = make_loader(X_val, y_val)

torch_model = TorchPriceRNN().to(DEVICE)
optimizer = torch.optim.Adam(torch_model.parameters(), lr=LEARNING_RATE)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.5, patience=5
)
best_val_loss = float("inf")
best_state = None
epochs_without_improvement = 0
torch_history = {"loss": [], "val_loss": [], "learning_rate": []}

torch_start = time.perf_counter()
for epoch in range(1, MAX_EPOCHS + 1):
    train_loss = run_torch_epoch(torch_model, train_loader, optimizer)
    val_loss = run_torch_epoch(torch_model, val_loader)
    scheduler.step(val_loss)
    current_lr = optimizer.param_groups[0]["lr"]
    torch_history["loss"].append(train_loss)
    torch_history["val_loss"].append(val_loss)
    torch_history["learning_rate"].append(current_lr)

    if val_loss < best_val_loss - 1e-8:
        best_val_loss = val_loss
        best_state = copy.deepcopy(torch_model.state_dict())
        epochs_without_improvement = 0
        torch.save(
            {"state_dict": best_state, "model_config": MODEL_CONFIG},
            PYTORCH_WEIGHTS_PATH,
        )
    else:
        epochs_without_improvement += 1
```

```
    if epoch == 1 or epoch % 10 == 0:
        print(
            f"Epoch {epoch:03d} | train={train_loss:.6f} | "
            f"val={val_loss:.6f} | lr={current_lr:.2e}"
        )
    if epoch >= MIN_EPOCHS and epochs_without_improvement >=
    ↪  EARLY_STOPPING_PATIENCE:
        print(f"Early stopping tai epoch {epoch}.")
        break

torch_train_seconds = time.perf_counter() - torch_start
torch_model.load_state_dict(best_state)
torch_scaled_prediction = predict_torch(torch_model, X_test)
print(f"PyTorch hoan tat trong {torch_train_seconds:.2f}s; best val
↪  loss={best_val_loss:.6f}")
```

Output của cell:

```
Epoch 001 | train=0.178609 | val=5.266870 | lr=1.00e-03

Epoch 010 | train=0.005766 | val=1.987922 | lr=1.00e-03

Epoch 020 | train=0.006129 | val=1.137135 | lr=1.00e-03

Epoch 030 | train=0.005416 | val=0.899365 | lr=1.00e-03

Epoch 040 | train=0.004985 | val=0.696580 | lr=1.00e-03

Epoch 050 | train=0.005175 | val=0.687103 | lr=1.00e-03

Epoch 060 | train=0.005699 | val=0.370009 | lr=1.00e-03

Epoch 070 | train=0.004700 | val=0.311509 | lr=5.00e-04

Epoch 080 | train=0.004698 | val=0.398837 | lr=2.50e-04

Epoch 090 | train=0.004271 | val=0.384622 | lr=1.25e-04

Epoch 100 | train=0.004517 | val=0.367282 | lr=3.13e-05

Early stopping tai epoch 107.
PyTorch hoan tat trong 101.50s; best val loss=0.309872
```

Persistence thắng cả bốn metric. Điều này là một kết quả thực nghiệm quan trọng, không phải lý do để loại baseline khỏi báo cáo. Trên chuỗi mức giá, phiên kề nhau thường gần nhau; dự báo "bằng phiên trước" khai thác đặc tính đó trực tiếp. RNN nhỏ có thể học một vùng giá điển hình của train nhưng khó dự báo khi test dịch lên vùng cao hơn. Hai RNN khá gần nhau, PyTorch nhỉnh hơn Keras một chút về MAE, nhưng cả hai còn cách xa baseline. Nếu nghiên cứu tiếp, có thể dự báo return hoặc difference thay vì mức giá, dùng rolling normalization và đánh giá walk-forward; những thay đổi này chưa nằm trong kết quả đã công bố.

Bảng 4.2: Ba mô hình trên test Amazon Stock Price

Mô hình	MAE	RMSE	MAPE (%)	R^2	Train (s)
Persistence baseline	**2.2720**	**3.1507**	**1.7519**	**0.9874**	0.00
PyTorch RNN	10.4466	13.6738	6.7825	0.7633	101.50
Keras RNN	10.5704	14.1749	6.8377	0.7456	18.88

Cell 28 — đánh giá dự báo (notebook gốc).

```
def inverse_amazon(values):
    values = np.asarray(values).reshape(-1, 1)
    return scaler.inverse_transform(values).reshape(-1)

torch_prediction = inverse_amazon(torch_scaled_prediction)
keras_prediction = inverse_amazon(keras_scaled_prediction)
predictions = {
    "Persistence baseline": baseline_test,
    "PyTorch RNN": torch_prediction,
    "Keras RNN": keras_prediction,
}
results = []
for name, prediction in predictions.items():
    row = {"Model": name, **regression_metrics(y_test_raw, prediction)}
    if name == "PyTorch RNN":
        row["Train time (s)"] = torch_train_seconds
    elif name == "Keras RNN":
        row["Train time (s)"] = keras_train_seconds
    else:
        row["Train time (s)"] = 0.0
    results.append(row)
results_df = pd.DataFrame(results).set_index("Model").sort_values("RMSE")
display(results_df.style.format({column: "{:.4f}" for column in
↪  results_df.columns}))

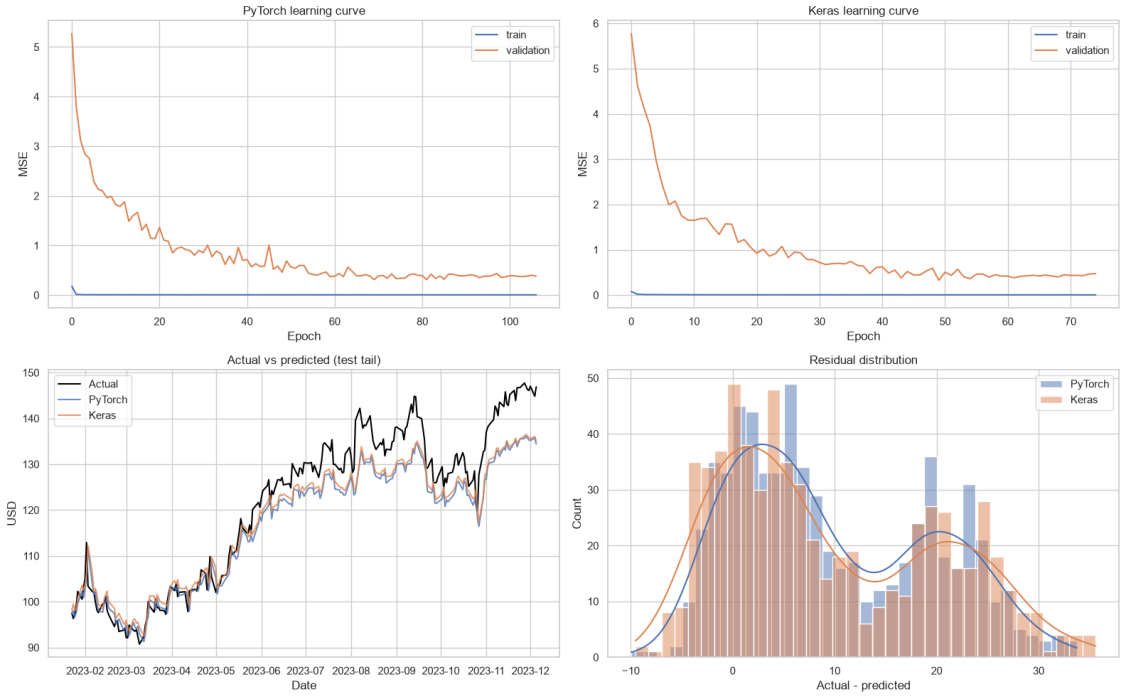
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes[0, 0].plot(torch_history["loss"], label="train")
axes[0, 0].plot(torch_history["val_loss"], label="validation")
axes[0, 0].set(title="PyTorch learning curve", xlabel="Epoch",
↪  ylabel="MSE")
axes[0, 0].legend()
axes[0, 1].plot(keras_history["loss"], label="train")
axes[0, 1].plot(keras_history["val_loss"], label="validation")
axes[0, 1].set(title="Keras learning curve", xlabel="Epoch",
↪  ylabel="MSE")
axes[0, 1].legend()
plot_tail = min(220, len(y_test_raw))
axes[1, 0].plot(test_dates.iloc[-plot_tail:], y_test_raw[-plot_tail:],
↪  label="Actual", color="black")
axes[1, 0].plot(test_dates.iloc[-plot_tail:],
↪  torch_prediction[-plot_tail:], label="PyTorch", alpha=0.8)
```

```
axes[1, 0].plot(test_dates.iloc[-plot_tail:],
↪  keras_prediction[-plot_tail:], label="Keras", alpha=0.8)
axes[1, 0].set(title="Actual vs predicted (test tail)", xlabel="Date",
↪  ylabel="USD")
axes[1, 0].legend()
sns.histplot(y_test_raw - torch_prediction, bins=35, kde=True, ax=axes[1,
↪  1], label="PyTorch", alpha=0.5)
sns.histplot(y_test_raw - keras_prediction, bins=35, kde=True, ax=axes[1,
↪  1], label="Keras", alpha=0.5)
axes[1, 1].set(title="Residual distribution", xlabel="Actual -
↪  predicted")
axes[1, 1].legend()
plt.tight_layout()
plt.show()
```

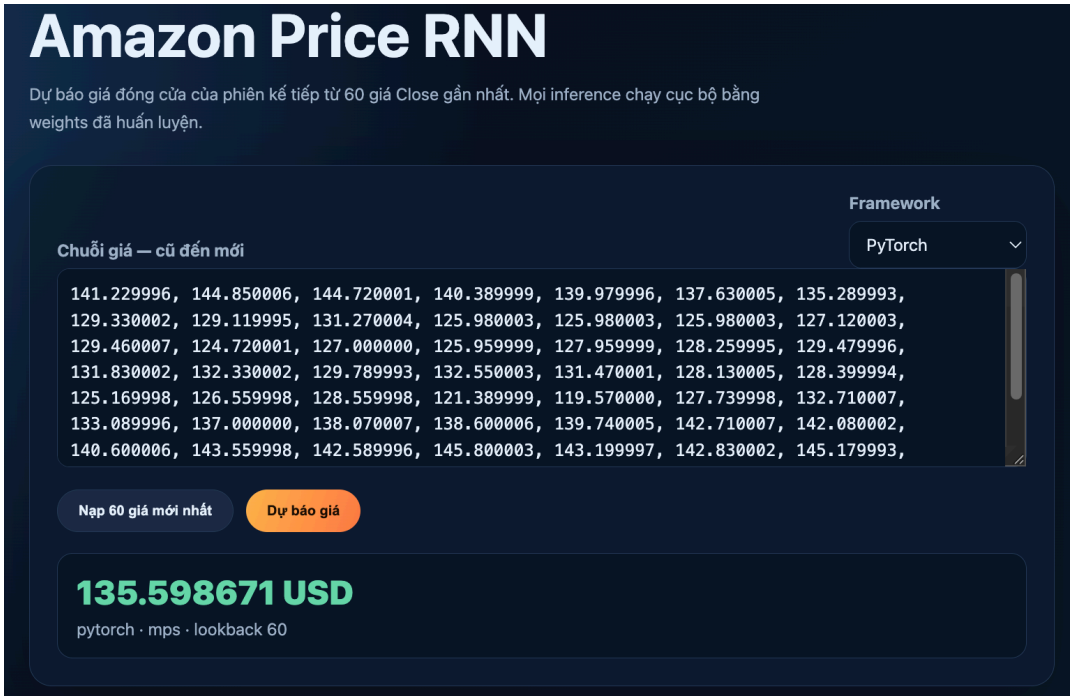
Output của cell:

```
index=Persistence baseline; MAE=2.2720; RMSE=3.1507; MAPE (%)=1.7519;
↪  R2=0.9874; Train time (s)=0.0000
index=PyTorch RNN; MAE=10.4466; RMSE=13.6738; MAPE (%)=6.7825; R2=0.7633;
↪  Train time (s)=101.4997
index=Keras RNN; MAE=10.5704; RMSE=14.1749; MAPE (%)=6.8377; R2=0.7456;
↪  Train time (s)=18.8772
```

Output hình của cell: Hình 4.5.



Hình 4.5: Learning curves, giá thực so với dự báo và residual trên test Amazon.



Hình 4.6: Giao diện deploy Amazon với chuỗi giá Close, framework và giá phiên kế tiếp dự đoán.

4.3 Avocado Price: nhiều chuỗi giá theo thị trường

Mã nguồn GitHub: https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2006/Avocado%20Price

Nguồn dữ liệu: https://www.kaggle.com/datasets/neuromusic/avocado-prices

Tệp `avocado.csv` có 18.249 dòng từ 04/01/2015 đến 25/03/2018. Mỗi dòng là thống kê một tuần cho một tổ hợp vùng và loại bơ, với target `AveragePrice`; các cột khác gồm sản lượng, mã PLU, số lượng túi, `type`, năm và `region`. Dòng mẫu cho Albany, loại conventional vào 27/12/2015 có AveragePrice 1.33 và Total Volume 64.236,62. Trong CSV, giá thấp nhất 0.44, trung vị 1.37 và cao nhất 3.25. Hai loại conventional và organic có lần lượt 9.126 và 9.123 dòng, gần cân bằng về số quan sát.

Cell 16 — EDA (notebook gốc).

```
weekly_by_type = (
    df.groupby(["Date", "type"], as_index=False)["AveragePrice"].mean()
)
top_regions = (
    df.groupby("region")["Total
    ↪  Volume"].sum().nlargest(12).index.tolist()
)
fig, axes = plt.subplots(2, 2, figsize=(17, 11))
sns.histplot(
    data=df, x="AveragePrice", hue="type", bins=45, kde=True,
    stat="density", common_norm=False, element="step", ax=axes[0, 0]
```

```
)
axes[0, 0].set(title="Phan bo AveragePrice theo type")
sns.lineplot(data=weekly_by_type, x="Date", y="AveragePrice", hue="type",
↪  ax=axes[0, 1])
axes[0, 1].set(title="Gia trung binh theo tuan va type")
sns.histplot(np.log1p(df["Total Volume"]), bins=55, kde=True, ax=axes[1,
↪  0], color="tab:purple")
axes[1, 0].set(title="Phan bo log(1 + Total Volume)", xlabel="log
↪  volume")
sns.boxplot(
    data=df[df["region"].isin(top_regions)], x="AveragePrice",
    ↪  y="region",
    hue="type", orient="h", ax=axes[1, 1]
)
axes[1, 1].set(title="Gia tai 12 region co volume lon nhat")
plt.tight_layout()
plt.show()

seasonal = df.assign(month=df["Date"].dt.month).groupby(["month",
↪  "type"])["AveragePrice"].mean().unstack()
display(seasonal.style.format("{:.3f}"))
```

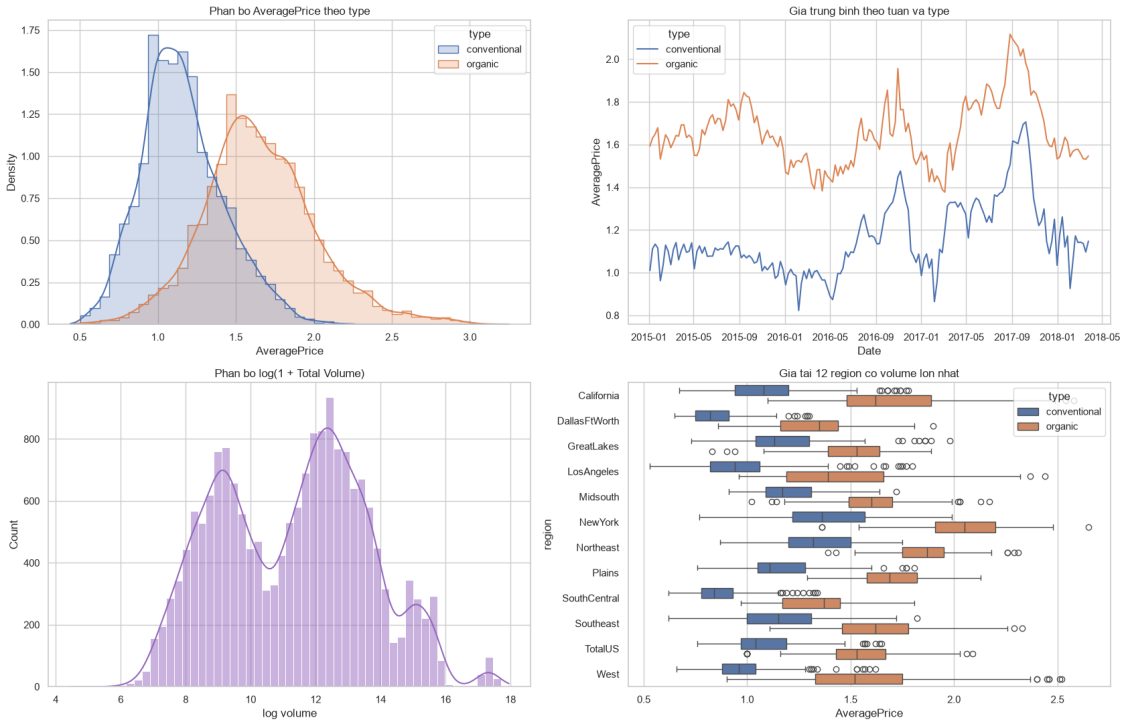
Output của cell:

```
type=1; conventional=1.069; organic=1.553
type=2; conventional=1.017; organic=1.531
type=3; conventional=1.126; organic=1.538
type=4; conventional=1.142; organic=1.606
type=5; conventional=1.090; organic=1.604
type=6; conventional=1.149; organic=1.670
type=7; conventional=1.212; organic=1.712
type=8; conventional=1.235; organic=1.790
type=9; conventional=1.295; organic=1.851
type=10; conventional=1.349; organic=1.810
type=11; conventional=1.206; organic=1.705
type=12; conventional=1.077; organic=1.585
```

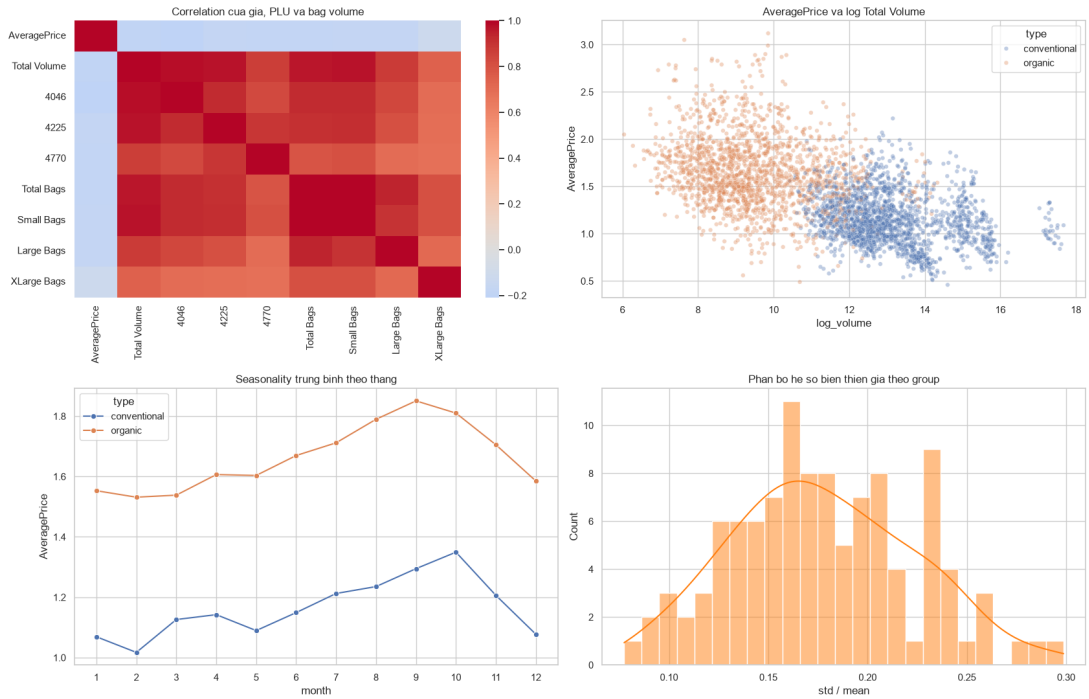
Output hình của cell: Hình 4.7.

Khác Amazon, nhiều dòng có cùng ngày vì 54 region và hai type tạo thành 108 chuỗi độc lập. Do đó không được xếp toàn bộ CSV theo Date rồi tạo cửa sổ xuyên ranh giới region hoặc type. Notebook sắp xếp và tạo cửa sổ riêng trong mỗi nhóm, với lookback 12 tuần, sau đó gộp các cửa sổ để huấn luyện một mô hình chung. Scaler theo nhóm giữ mức giá riêng của từng thị trường. Input của một mẫu là 12 giá trung bình của cùng một region–type; output là AveragePrice tuần kế tiếp của đúng nhóm ấy.

Hai RNN có cùng kiến trúc với bài Amazon nhưng học trên nhiều chuỗi. PyTorch train 32 epoch trong 22.72 giây và Keras mất 10.88 giây. Kết quả tính trên 1.728 cửa sổ test sau khi đổi ngược về thang giá của từng nhóm. Đường học và đồ thị actual–predicted của một nhóm minh họa diễn biến, trong khi bảng metric tổng hợp là phép đánh giá trên toàn bộ các nhóm. Không nên lấy một nhóm tiêu biểu để suy ra chất lượng như nhau ở mọi vùng.



Hình 4.7: Tổng quan mức giá, sản lượng và khác biệt giữa các nhóm Avocado.



Hình 4.8: Mẫu hình mùa vụ, tương quan và biến động giá Avocado.

Cell 24 — huấn luyện PyTorch RNN (notebook gốc).

```
train_loader = make_loader(X_train, y_train, shuffle=True)
val_loader = make_loader(X_val, y_val)

torch_model = TorchPriceRNN().to(DEVICE)
optimizer = torch.optim.Adam(torch_model.parameters(), lr=LEARNING_RATE)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.5, patience=5
)
best_val_loss = float("inf")
best_state = None
epochs_without_improvement = 0
torch_history = {"loss": [], "val_loss": [], "learning_rate": []}

torch_start = time.perf_counter()
for epoch in range(1, MAX_EPOCHS + 1):
    train_loss = run_torch_epoch(torch_model, train_loader, optimizer)
    val_loss = run_torch_epoch(torch_model, val_loader)
    scheduler.step(val_loss)
    current_lr = optimizer.param_groups[0]["lr"]
    torch_history["loss"].append(train_loss)
    torch_history["val_loss"].append(val_loss)
    torch_history["learning_rate"].append(current_lr)

    if val_loss < best_val_loss - 1e-8:
        best_val_loss = val_loss
        best_state = copy.deepcopy(torch_model.state_dict())
        epochs_without_improvement = 0
        torch.save(
            {"state_dict": best_state, "model_config": MODEL_CONFIG},
            PYTORCH_WEIGHTS_PATH,
        )
    else:
        epochs_without_improvement += 1

    if epoch == 1 or epoch % 10 == 0:
        print(
            f"Epoch {epoch:03d} | train={train_loss:.6f} | "
            f"val={val_loss:.6f} | lr={current_lr:.2e}"
        )
    if epoch >= MIN_EPOCHS and epochs_without_improvement >=
    ↪  EARLY_STOPPING_PATIENCE:
        print(f"Early stopping tai epoch {epoch}.")
        break

torch_train_seconds = time.perf_counter() - torch_start
torch_model.load_state_dict(best_state)
torch_scaled_prediction = predict_torch(torch_model, X_test)
print(f"PyTorch hoan tat trong {torch_train_seconds:.2f}s; best val
↪  loss={best_val_loss:.6f}")
```

Output của cell:

```
Epoch 001 | train=0.506085 | val=0.853458 | lr=1.00e-03

Epoch 010 | train=0.442515 | val=0.857946 | lr=1.00e-03

Epoch 020 | train=0.438074 | val=0.958861 | lr=2.50e-04

Epoch 030 | train=0.435213 | val=0.840648 | lr=1.25e-04

Early stopping tai epoch 32.
PyTorch hoan tat trong 22.72s; best val loss=0.773740
```

Bảng 4.3: Ba mô hình trên test Avocado Price

Mô hình	MAE	RMSE	MAPE (%)	R^2	Train (s)
Keras RNN	**0.1099**	**0.1534**	**8.7926**	**0.7490**	10.88
PyTorch RNN	0.1122	0.1562	9.0260	0.7398	22.72
Persistence baseline	0.1199	0.1742	9.5478	0.6762	0.00

Trên Avocado, cả hai RNN vượt persistence ở cả bốn metric, trái với Amazon. Keras dẫn đầu với MAE 0.1099 và R^2 0.7490. Mức chênh lệch giữa hai framework nhỏ hơn khoảng cách giữa chúng với baseline theo RMSE. Một lời giải thích hợp lý là chuỗi giá theo tuần và các nhóm thị trường có mẫu hình mà RNN học được tốt hơn quy tắc giữ nguyên giá tuần trước; đây là suy luận từ kết quả, không phải chứng minh nhân quả. Cần đánh giá riêng theo region và type để biết lợi ích có đồng đều không.

Cell 28 — đánh giá dự báo (notebook gốc).

```
def inverse_by_group(scaled_values, group_keys):
    output = np.empty(len(scaled_values), dtype=np.float64)
    for index, (value, key) in enumerate(zip(scaled_values, group_keys)):
        scaler_info = group_scalers[key]
        output[index] = value * scaler_info["scale"] +
        ↪  scaler_info["mean"]
    return output

torch_prediction = inverse_by_group(torch_scaled_prediction,
↪  test_group_keys)
keras_prediction = inverse_by_group(keras_scaled_prediction,
↪  test_group_keys)
predictions = {
    "Persistence baseline": baseline_test,
    "PyTorch RNN": torch_prediction,
    "Keras RNN": keras_prediction,
}
results = []
for name, prediction in predictions.items():
```

```
    row = {"Model": name, **regression_metrics(y_test_raw, prediction)}
    if name == "PyTorch RNN":
        row["Train time (s)"] = torch_train_seconds
    elif name == "Keras RNN":
        row["Train time (s)"] = keras_train_seconds
    else:
        row["Train time (s)"] = 0.0
    results.append(row)
results_df = pd.DataFrame(results).set_index("Model").sort_values("RMSE")
display(results_df.style.format({column: "{:.4f}" for column in
↪  results_df.columns}))

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes[0, 0].plot(torch_history["loss"], label="train")
axes[0, 0].plot(torch_history["val_loss"], label="validation")
axes[0, 0].set(title="PyTorch learning curve", xlabel="Epoch",
↪  ylabel="MSE")
axes[0, 0].legend()
axes[0, 1].plot(keras_history["loss"], label="train")
axes[0, 1].plot(keras_history["val_loss"], label="validation")
axes[0, 1].set(title="Keras learning curve", xlabel="Epoch",
↪  ylabel="MSE")
axes[0, 1].legend()

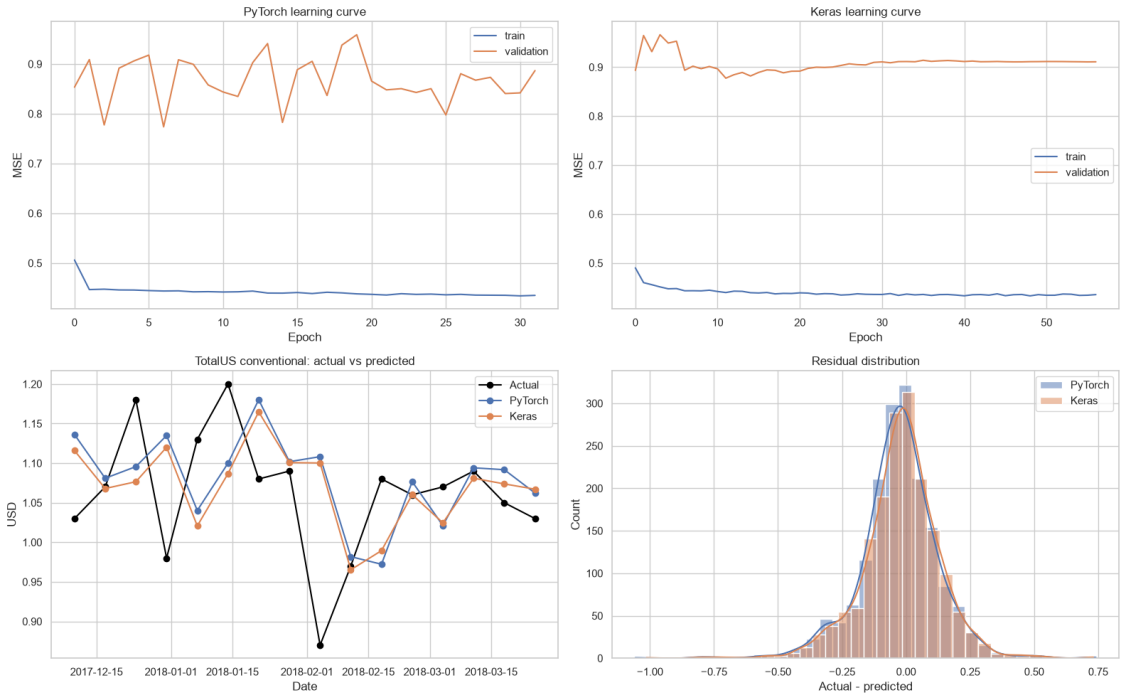
focus_key = f"TotalUS{GROUP_SEPARATOR}conventional"
focus = test_group_keys == focus_key
focus_order = np.argsort(test_dates[focus])
axes[1, 0].plot(test_dates[focus][focus_order],
↪  y_test_raw[focus][focus_order], marker="o", label="Actual",
↪  color="black")
axes[1, 0].plot(test_dates[focus][focus_order],
↪  torch_prediction[focus][focus_order], marker="o", label="PyTorch")
axes[1, 0].plot(test_dates[focus][focus_order],
↪  keras_prediction[focus][focus_order], marker="o", label="Keras")
axes[1, 0].set(title="TotalUS conventional: actual vs predicted",
↪  xlabel="Date", ylabel="USD")
axes[1, 0].legend()
sns.histplot(y_test_raw - torch_prediction, bins=35, kde=True, ax=axes[1,
↪  1], label="PyTorch", alpha=0.5)
sns.histplot(y_test_raw - keras_prediction, bins=35, kde=True, ax=axes[1,
↪  1], label="Keras", alpha=0.5)
axes[1, 1].set(title="Residual distribution", xlabel="Actual -
↪  predicted")
axes[1, 1].legend()
plt.tight_layout()
plt.show()
```

Output của cell:

```
index=Keras RNN; MAE=0.1099; RMSE=0.1534; MAPE (%)=8.7926; R2=0.7490;
↪  Train time (s)=10.8845
```

```
index=PyTorch RNN; MAE=0.1122; RMSE=0.1562; MAPE (%)=9.0260; R2=0.7398;
↪  Train time (s)=22.7235
index=Persistence baseline; MAE=0.1199; RMSE=0.1742; MAPE (%)=9.5478;
↪  R2=0.6762; Train time (s)=0.0000
```

Output hình của cell: Hình 4.9.



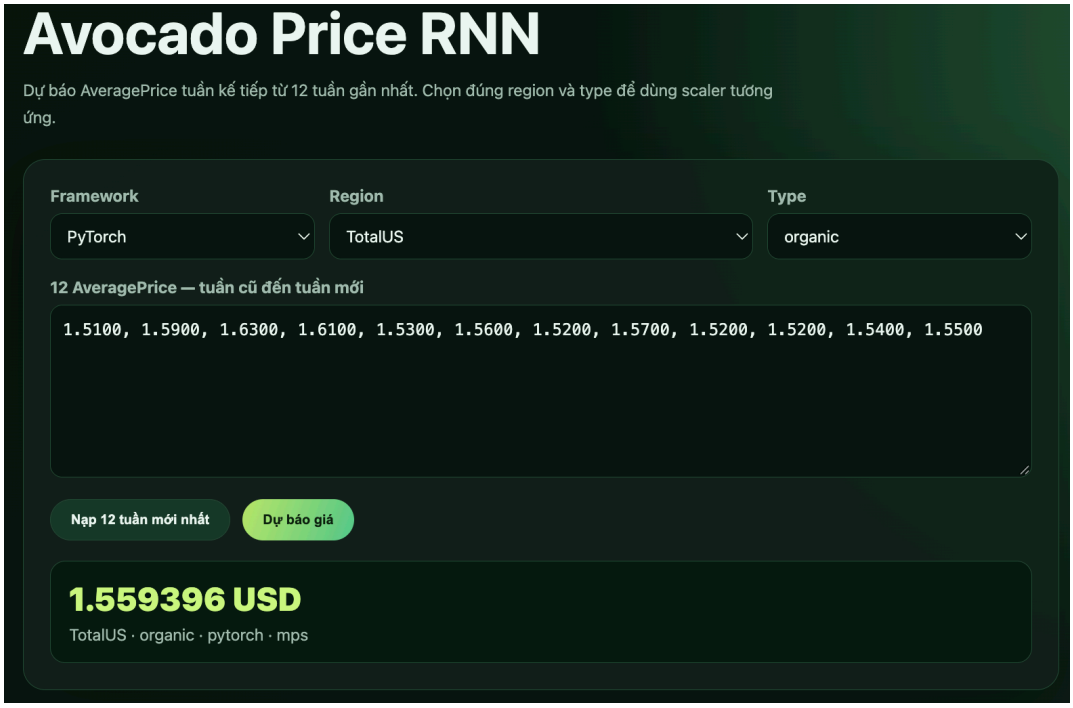
Hình 4.9: Đường học, actual–predicted và residual của Avocado trong notebook.

4.4 Kiểm tra luồng deploy trên localhost

Hai ứng dụng của Assignment 06 đã lưu weights PyTorch, Keras và metadata trong thư mục `models` của từng dataset. Người dùng không cần huấn luyện lại trước khi mở web. Tại localhost, ứng dụng Amazon mặc định dùng cổng 8001; Avocado dùng cổng 8002. Giao diện Amazon cho phép nạp 60 giá Close mới nhất từ CSV hoặc nhập 60 số theo thứ tự cũ đến mới. Giao diện Avocado cho phép chọn region, type rồi nạp 12 giá AveragePrice tuần gần nhất của đúng nhóm. Cả hai cho phép chọn PyTorch hoặc Keras trước khi nhấn nút dự báo.

Luồng inference cần giữ đúng thứ tự: kiểm tra đầu vào, chọn scaler từ metadata, chuẩn hóa chuỗi, nạp mô hình và checkpoint tương ứng, chạy dự báo rồi đổi ngược sang đơn vị giá. Việc nạp một bộ weights khác nhưng giữ scaler cũ có thể trả về một con số hợp lệ về mặt kiểu dữ liệu mà sai về ý nghĩa. Với Avocado, còn phải dùng scaler của region–type đã chọn. Đây là một điểm dễ lỗi khi chuyển code từ notebook sang ứng dụng web, vì notebook thường giữ sẵn các biến trong bộ nhớ, còn web phải tái tạo toàn bộ trạng thái từ artifact đã lưu.

Ứng dụng cần từ chối chuỗi không đủ độ dài, giá âm hoặc không hữu hạn, framework không được hỗ trợ và nhóm Avocado không có trong metadata. Việc từ chối dữ liệu xấu giúp người dùng hiểu lý do không có dự báo và ngăn dữ liệu sai đi vào mô hình. Tuy nhiên, một



Hình 4.10: Giao diện deploy Avocado với region, type, chuỗi giá, framework và kết quả dự đoán.

chuỗi có đúng 60 số dương vẫn có thể vô lý về kinh tế, ví dụ các giá nhập nhầm đơn vị hoặc đảo thứ tự thời gian. Kiểm tra đầu vào vì vậy giảm lỗi cơ học nhưng không đảm bảo giá trị dự báo có thể sử dụng.

Để tạo minh chứng deploy cho báo cáo, ảnh chụp nên được thực hiện sau khi nạp chuỗi và bấm dự báo. Ảnh Amazon phải thấy lựa chọn framework, 60 giá gần nhất hoặc phần nhập tương ứng và giá Close dự đoán. Ảnh Avocado phải thấy thêm region, type và 12 tuần giá; cả hai cần chụp cả kết quả xuất ra. Nếu ảnh chỉ hiển thị trang trống trước dự báo, nó chứng minh giao diện khởi động được nhưng chưa chứng minh mô hình đã chạy. Đó là lý do hai vị trí ảnh ở Hình 4.6 và Hình 4.10 được đặt sau phần kết quả benchmark.

4.5 Giới hạn của hai bài toán dự báo

Thứ nhất, cả hai notebook dự báo một bước, không đánh giá dự báo nhiều bước. Nếu cần giá sau bảy phiên hoặc bốn tuần, không thể lấy MAE một bước rồi suy ra sai số ở chân trời dài hơn. Một cách đơn giản là dự báo lặp: dùng kết quả bước trước làm đầu vào cho bước sau, nhưng lỗi sẽ tích lũy. Cách khác là huấn luyện mô hình dự báo trực tiếp từng chân trời. Hai cách này cần thiết kế tập train và baseline tương ứng, vì mục tiêu vận hành đã đổi.

Thứ hai, dữ liệu Amazon là một chuỗi đơn và chỉ dùng giá Close. Dữ liệu Avocado có nhiều chuỗi nhưng notebook cũng chỉ dùng giá quá khứ của cùng nhóm làm input. Các biến khác trong CSV, như Volume của Amazon hay Total Volume của Avocado, xuất hiện trong EDA chứ không đi vào RNN được báo cáo. Nếu bổ sung biến ngoại sinh, phải kiểm tra nó có thực sự biết được tại thời điểm dự báo hay không. Chẳng hạn doanh số tuần kế tiếp chưa thể dùng để

dự báo giá tuần kế tiếp nếu doanh số ấy chỉ xuất hiện sau khi tuần kết thúc. Dùng biến không sẵn có lúc inference là một dạng rò rỉ dữ liệu dù metric test rất đẹp.

Thứ ba, metric tổng hợp trên Avocado trộn 108 chuỗi theo số cửa sổ. Nếu một số nhóm có nhiều quan sát hơn, chúng đóng góp nhiều hơn vào MAE chung. Để hiểu chất lượng theo thị trường, cần thêm bảng MAE theo region–type và xem phân phối lỗi giữa các nhóm. Một model chung có thể cải thiện trung bình nhưng kém baseline ở vài thị trường cụ thể. Tương tự, Amazon cần xem lỗi theo giai đoạn: một năm biến động mạnh có thể tạo phần lớn RMSE. Bảng tổng hợp ở tiểu luận phản ánh đúng notebook hiện có nhưng không trả lời hết những câu hỏi phân tầng này.

Thứ tư, dự báo giá không tương đương quyết định mua bán hay định giá giao dịch. Một mô hình có MAE thấp vẫn có thể không tạo lợi thế sau khi tính chi phí giao dịch và rủi ro; với Avocado, giá trung bình quan sát cũng không mô tả từng đơn hàng. Vì vậy phần ứng dụng nên trình bày kết quả như một ước lượng từ lịch sử và nêu phạm vi dữ liệu. Điểm mạnh của hai bài thực nghiệm là chúng buộc mô hình được đối chiếu với một quy tắc rất đơn giản. Khi RNN thua persistence trên Amazon, báo cáo giữ nguyên kết luận ấy; khi RNN thắng trên Avocado, báo cáo vẫn không suy rộng sang mọi chuỗi thời gian.

4.6 Tổng hợp Chương 4 và ý nghĩa triển khai

Hai dataset minh họa vì sao baseline cần xuất hiện trong mọi nghiên cứu dự báo chuỗi. Cùng là RNN một tầng với 64 hidden units, kết luận ở Amazon là "không vượt được giá phiên trước", còn ở Avocado là "vượt baseline một mức vừa phải". Sự khác biệt liên quan đến cấu trúc dữ liệu, thang thời gian, phân bố train/test và cách nhóm chuỗi. Không thể kết luận RNN luôn tốt hoặc luôn kém từ một dataset. PyTorch và Keras cũng không có thứ hạng cố định: PyTorch nhỉnh hơn ở Amazon, Keras nhỉnh hơn ở Avocado trong các lần chạy đã lưu.

Ứng dụng localhost cho phép nhập hoặc nạp sẵn cửa sổ gần nhất, chọn framework, chạy inference từ trọng số đã lưu và hiển thị dự báo. Ảnh deploy cần chứng minh đúng thao tác end-to-end, gồm đầu vào, lựa chọn mô hình và output. Việc website trả một số hợp lệ chỉ kiểm tra đường đi dữ liệu; chất lượng dự báo vẫn được quyết định bằng test theo thời gian và so sánh baseline. Nếu dữ liệu mới vượt ra ngoài khoảng thời gian của CSV, cần theo dõi sai số trên những cửa sổ gần đây và cân nhắc huấn luyện lại.

Chương 5

Knowledge Graph, RAG và Agent

5.1 Từ cơ sở tri thức đến hệ hỏi đáp có nguồn

Cơ sở tri thức (knowledge base, KB) lưu các sự kiện theo cấu trúc cho phép truy vấn. Knowledge Graph (KG) là một cách xây KB: thực thể là nút, quan hệ là cạnh có hướng, còn thuộc tính ghi giá trị gắn với nút hoặc cạnh. Mô hình bộ ba (s, p, o) phát biểu chủ thể s liên hệ với đối tượng o qua vị từ p. RDF của W3C là một chuẩn biểu diễn dữ liệu dựa trên bộ ba; graph trong bài thực nghiệm này được cài bằng `networkx.DiGraph`, không phải kho RDF hay hệ truy vấn SPARQL [11]. Google đưa thuật ngữ Knowledge Graph vào tìm kiếm năm 2012 để nhấn mạnh việc nhận diện thực thể và quan hệ thay vì chỉ khớp chuỗi ký tự [12].

Retrieval-Augmented Generation (RAG) kết hợp truy xuất tri thức bên ngoài với mô hình sinh văn bản. Thay vì yêu cầu mô hình nhớ mọi số liệu, hệ thống tìm tài liệu liên quan đến câu hỏi và đưa chúng vào ngữ cảnh trước khi sinh câu trả lời. Bài báo của Lewis và cộng sự năm 2020 là một mốc quan trọng của hướng này [13]. Trong Assignment 07, KB và RAG cùng lấy dữ kiện từ CSV, nhưng xử lý những loại câu hỏi khác nhau: graph trả lời theo khóa ngày hoặc nhóm, phép tính CSV trả kết quả số học, còn RAG hỗ trợ câu hỏi tóm tắt và mô tả. ID nguồn cho phép truy lại tài liệu hoặc node đã dùng; nó không tự xác nhận mọi câu do mô hình sinh ra đều đúng.

KB là khái niệm rộng cho một kho dữ kiện có tổ chức; KG nhấn mạnh cách biểu diễn dưới dạng nút và quan hệ. Một bảng CSV cũng có thể chứa tri thức, nhưng graph khiến đường đi giữa các thực thể trở nên tường minh. Chẳng hạn từ node cổ phiếu có thể lần theo năm, tháng rồi tới phiên 05/12/2023. Dạng này phù hợp khi câu hỏi có khóa rõ ràng và cần giải thích nguồn của một thuộc tính. Ngược lại, thống kê trung bình trên một khoảng gồm nhiều phiên là phép tính trên tập hàng; cất sẵn mọi khoảng khả dĩ trong graph sẽ dư thừa và khó cập nhật.

KG chỉ đáng tin trong phạm vi schema và dữ liệu đã nạp. Nếu một node nhóm diabetes ghi 4.461 nhãn dương trong 58.552 bản ghi nữ, tỷ lệ $4.461/58.552 \approx 7{,}62\%$ là thống kê mô tả của đúng nhóm ấy. Cạnh `has_cohort` có nghĩa "dataset chứa nhóm thống kê", không có nghĩa giới tính gây ra bệnh. Sự phân biệt giữa quan hệ cấu trúc dữ liệu và quan hệ nhân quả là điều kiện quan trọng khi đọc graph y sinh.

5.2 Biểu diễn graph và luồng RAG

5.2.1 Schema, truy vấn và nguồn dữ liệu

Với Amazon, đường đi trong graph là cổ phiếu → năm → tháng → phiên. Ví dụ cạnh `has_month` nối năm 2023 với tháng 2023-12; node phiên lưu Open, High, Low, Close, Adj Close và Volume. Với diabetes, node dataset nối đến các node nhóm thống kê bằng `has_cohort`; mỗi nhóm lưu số bản ghi, số nhãn `diabetes=1` và tỷ lệ. Graph còn nối nhóm đến hai node kết quả nhãn. Không có node cho từng người bệnh, quan hệ lâm sàng hay suy luận nhân quả. Khóa node nhất quán giúp truy vấn `day:2023-12-05` hoặc `cohort:gender:Female` không nhập nhằng.

Về hình thức, đặt $G = (V, E)$ với V là tập node và $E \subseteq V \times R \times V$ là tập cạnh có nhãn thuộc tập quan hệ R. Bộ ba (s, p, o) là một cạnh từ chủ thể s tới đối tượng o qua vị từ p. Ví dụ $(\texttt{year:2023}, \texttt{has_month}, \texttt{month:2023-12})$ nêu một quan hệ; thuộc tính `Close=146.880005` lại là giá trị gắn với node phiên. Schema quy định kiểu node nào được nối bằng quan hệ nào, kiểu dữ liệu của thuộc tính và cách tạo ID. Nếu trộn ID `2023-12-05` với `05/12/2023` mà không chuẩn hóa, cùng một phiên có thể thành hai node khác nhau.

Graph trong code là `networkx.DiGraph` với thuộc tính trên node/cạnh; nó không dùng RDF hay SPARQL. RDF của W3C cung cấp một cách chuẩn hóa biểu diễn bộ ba và định danh tài nguyên [11], nhưng ví dụ RDF ở phần lý thuyết không phải mô tả định dạng lưu trữ thực tế của bài tập. Truy vấn trong ứng dụng lấy node theo ID và đọc thuộc tính đã lưu. Nó không tự suy ra một ngày giao dịch mới từ ngày lân cận và cũng không suy luận rằng hai nhóm có quan hệ khi CSV không cung cấp quan hệ ấy.

Một dữ kiện có trong graph không thay thế phép tính cho hàng loạt bản ghi. Chẳng hạn, câu hỏi Close của một phiên lấy thuộc tính node; câu hỏi trung bình Close trong một khoảng ngày lọc CSV và tính trung bình trực tiếp. Nếu ngày hoặc nhóm không có trong dữ liệu, agent thông báo thiếu dữ liệu thay vì tạo ra giá trị mới. Ranh giới này đặc biệt quan trọng với dữ liệu y tế: tỷ lệ của một nhóm trong CSV chỉ mô tả tập quan sát, không chẩn đoán cho một cá nhân.

5.2.2 Tạo tài liệu, truy xuất và sinh câu trả lời

RAG của hai ứng dụng có ba bước. Thứ nhất, CSV được chuyển thành các tài liệu ngắn có ID ổn định: Amazon có tài liệu tổng quan, từng phiên và từng tháng; diabetes có tài liệu tổng quan và từng nhóm.

Đơn vị tài liệu được chọn theo đơn vị câu hỏi. Tài liệu phiên trả lời về một ngày; tài liệu tháng hỗ trợ tóm tắt tháng; tài liệu nhóm diabetes mô tả số lượng và tỷ lệ của nhóm. Mỗi tài liệu gồm ID ổn định và đoạn văn bản sinh từ CSV. ID đóng vai trò dấu vết để quay lại hàng hoặc nhóm nguồn, không phải đặc trưng ngữ nghĩa tự thân. Vì các tài liệu vốn ngắn, ứng dụng không cần chia trang dài thành các chunk chồng lấn. Khi dữ liệu nguồn đổi, corpus và chỉ mục phải được tạo lại để tránh trả lời từ bản cũ. Thứ hai, `TfidfVectorizer` biến tài liệu thành ma trận thưa với n-gram ký tự dài 3–5. Nó dùng `analyzer="char_wb"` để nhận diện một

phần các cách viết tiếng Việt, tên cột tiếng Anh và mã nguồn trộn nhau. Trọng số TF-IDF giảm với các n-gram xuất hiện ở nhiều tài liệu; cosine similarity xếp hạng độ gần của câu hỏi q và tài liệu d:

$$w_{t,d} = \mathrm{tf}(t,d)\left[\log\left(\frac{N+1}{\mathrm{df}(t)+1}\right)+1\right]. \tag{5.1}$$

Ở đây t là n-gram, d là tài liệu, N là số tài liệu trong corpus, $\mathrm{tf}(t,d)$ là số lần t xuất hiện ở d và $\mathrm{df}(t)$ là số tài liệu có t. Công thức trong ngoặc là IDF có làm trơn theo mặc định của `TfidfVectorizer` [16]. Nếu $N = 4$, n-gram xuất hiện ở cả bốn tài liệu có IDF bằng 1; n-gram chỉ xuất hiện ở một tài liệu có IDF $\log(5/2)+1 \approx 1{,}916$. Với cùng tần suất trong một tài liệu, n-gram hiếm được trọng số cao hơn vì phân biệt tài liệu tốt hơn. Đây vẫn là so khớp hình thức ký tự, không phải hiểu nghĩa từ.

$$\mathrm{sim}(q,d) = \frac{q \cdot d}{\|q\|_2 \|d\|_2}. \tag{5.2}$$

Agent xét tối đa bốn tài liệu đầu có điểm ít nhất $0{,}10$. Đây là điều kiện khớp văn bản, không phải xác suất câu trả lời đúng.

Trong công thức cosine, q và d là hai vector TF-IDF trong cùng không gian đặc trưng; tử số là tích vô hướng, mẫu số chuẩn hóa độ dài. Nếu hai vector đã có chuẩn L_2 bằng 1, điểm cosine đúng bằng tích vô hướng. Ví dụ hai vector $(1,1)$ và $(1,0)$ có điểm $1/\sqrt{2} \approx 0{,}707$, còn hai vector cùng hướng có điểm 1. Khi câu hỏi dùng từ không tạo n-gram trùng với corpus, vector truy vấn có thể thiếu tín hiệu và điểm thấp dù ý định người hỏi liên quan về mặt ngữ nghĩa.

Hệ sắp tài liệu theo điểm giảm dần, xét tối đa $k = 4$ kết quả và bỏ những tài liệu dưới 0,10. Top-k giới hạn lượng ngữ cảnh đưa vào mô hình, còn ngưỡng loại các khớp quá yếu. Cả k lẫn ngưỡng là lựa chọn kỹ thuật của bài thực nghiệm, không phải bảo đảm rằng bốn tài liệu chứa đủ dữ kiện. Nếu không còn tài liệu sau lọc, ứng dụng báo thiếu bằng chứng và không sinh câu trả lời; nếu có tài liệu nhưng thiếu chi tiết cần hỏi, mô hình vẫn phải được yêu cầu thừa nhận giới hạn.

Thứ ba, ứng dụng ghép những tài liệu được chọn theo dạng ID–nội dung vào prompt và yêu cầu Qwen3-0.6B trả lời ngắn bằng tiếng Việt từ ngữ cảnh đó [14]. Mô hình chạy cục bộ, được nạp khi nhánh RAG cần sinh câu trả lời. Nếu không tìm thấy tài liệu đạt ngưỡng, agent báo thiếu bằng chứng và không gọi mô hình. Câu trả lời trả kèm ID các tài liệu đã truy xuất; người đọc vẫn cần đối chiếu nội dung vì mô hình có thể lược bỏ hoặc diễn đạt sai chi tiết.

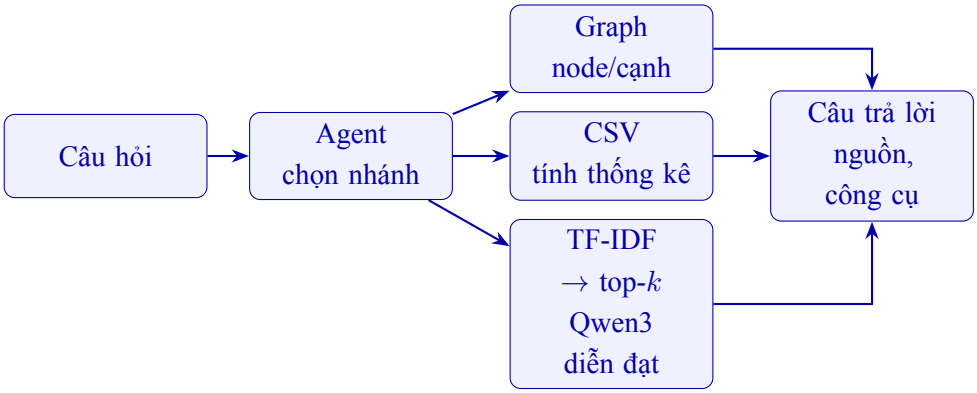
Prompt ghép mỗi đoạn bằng dạng `[ID] nội dung`, kèm câu hỏi và chỉ dẫn trả lời từ ngữ cảnh. Danh sách `sources` được chương trình lấy từ ID tài liệu truy xuất; LLM chỉ tạo phần diễn đạt. Cách tách trách nhiệm này cho phép người đọc kiểm tra lại bằng chứng mà không cần tin vào khả năng trích dẫn tự phát của mô hình. Tuy vậy, một ID nguồn có mặt chỉ chứng minh tài liệu đã được đưa vào prompt, chưa chứng minh mọi mệnh đề trong câu trả lời đều được tài liệu ấy hỗ trợ. Với câu hỏi số học chính xác, agent chọn graph hoặc CSV để con số do code tính ra.

5.3 Agent chọn công cụ và giới hạn của GraphRAG

Agent định tuyến theo dấu hiệu trong câu hỏi. Câu có ngày cụ thể hoặc tên nhóm đi tới graph; câu yêu cầu tổng hợp trên nhiều hàng đi tới phép tính CSV; câu mô tả trong phạm vi dataset mới đi vào RAG. Đầu ra luôn có ba trường `answer`, `sources` và `tools`. Công cụ được ghi nhận theo nhánh thực sự chạy, ví dụ `knowledge_graph`, `csv_statistics` hoặc `rag_retrieval` cùng `local_llm`.

Thứ tự luật định tuyến có ý nghĩa. Ở Amazon, hai ngày được nhận diện trước một ngày để tính khoảng trên CSV; một ngày lấy node phiên; truy vấn theo tháng hoặc năm kiểm tra node rồi tính trên các hàng tương ứng; câu hỏi mô tả mới dùng RAG. Ở diabetes, các yêu cầu chẩn đoán hoặc điều trị bị chặn trước; câu hỏi schema trả tên cột; thống kê theo nhóm đọc graph, thống kê toàn dataset tính CSV; câu hỏi mô tả dùng RAG. Nếu câu hỏi nằm ngoài phạm vi, agent báo ngoài phạm vi. Đây là agent điều phối bằng luật xác định, không phải mô hình ngôn ngữ tự quyết định công cụ.

Hình 5.1 mô tả các đường đi. Câu hỏi "Close ngày 05/12/2023?" lấy thuộc tính một node; "Close từ 01 đến 05/12/2023?" lọc và tính trên CSV; "Tóm tắt dữ liệu Amazon" truy xuất tài liệu rồi để Qwen3 diễn đạt. Cả ba đường đi cùng trả lời, nguồn và danh sách công cụ, nhưng ý nghĩa của nguồn khác nhau theo nhánh: ID node, phần CSV được thống kê hoặc ID tài liệu RAG.



Hình 5.1: Agent chọn một đường xử lý phù hợp rồi trả câu trả lời kèm nguồn và công cụ.

Bảng 5.1: Ranh giới giữa ba nhánh hỏi đáp Assignment 07

Nhánh	Câu hỏi phù hợp	Cách tạo kết quả
Graph	Một ngày giao dịch hoặc một nhóm diabetes	Đọc node/cạnh đã dựng từ CSV
CSV	Khoảng ngày, thống kê toàn dataset	Tính trên các hàng dữ liệu
RAG + LLM	Tóm tắt hoặc mô tả tập dữ liệu	Truy xuất tài liệu và sinh câu trả lời

Một số nghiên cứu GraphRAG đưa cả cấu trúc đồ thị vào khâu truy xuất và tổng hợp ngữ cảnh cho mô hình sinh [15]. Hệ thống ở đây sử dụng *agent chọn nhánh*; nó không trộn bộ ba

graph với tài liệu RAG trong một prompt và không có suy luận đa bước trên graph. Cách phân biệt này giúp đọc đúng kết quả thực nghiệm, thay vì gán cho ứng dụng một khả năng chưa được cài đặt.

5.4 Amazon Stock Price: hỏi đáp lịch sử giao dịch

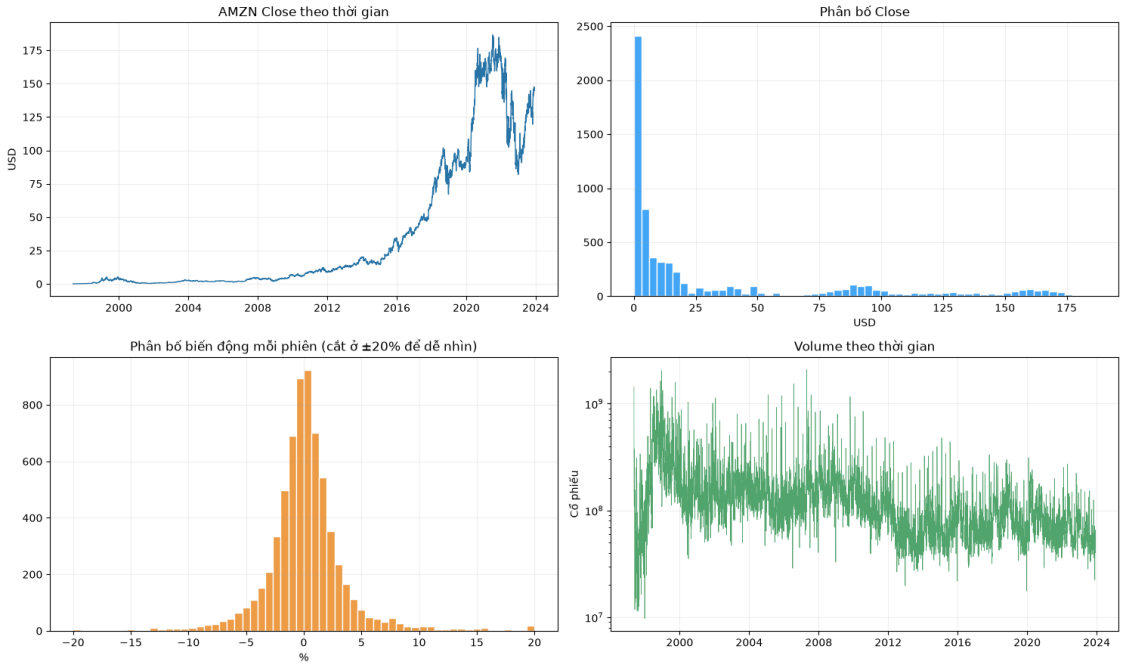
Mã nguồn GitHub: https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2007/Amazon%20Stock%20Price

CSV Amazon có 6.684 phiên và bảy cột, từ 15/05/1997 đến 05/12/2023. Notebook kiểm tra thứ tự ngày, dữ liệu thiếu và phân bố Close, biến động mỗi phiên, Volume. Hình 5.2 chỉ mô tả giá lịch sử; phiên cuối không phải giá hiện tại.

Cell 8 — EDA (notebook gốc).

```
returns = df["Close"].pct_change().dropna() * 100
fig, axes = plt.subplots(2, 2, figsize=(15, 9))
axes[0,0].plot(df["Date"], df["Close"], lw=1, color="#2874a6")
axes[0,0].set(title="AMZN Close theo thời gian", ylabel="USD")
axes[0,1].hist(df["Close"], bins=60, color="#42a5f5", edgecolor="white")
axes[0,1].set(title="Phân bố Close", xlabel="USD")
axes[1,0].hist(returns.clip(-20,20), bins=60, color="#ed9b45",
 ↪  edgecolor="white")
axes[1,0].set(title="Phân bố biến động mỗi phiên (cắt ở ±20% để dễ
 ↪  nhìn)", xlabel="%")
axes[1,1].plot(df["Date"], df["Volume"], lw=.5, color="#52a46d")
axes[1,1].set(title="Volume theo thời gian", ylabel="Cổ phiếu",
 ↪  yscale="log")
for ax in axes.flat: ax.grid(alpha=.2)
plt.tight_layout(); plt.show()
```

Output hình của cell: Hình 5.2.



Hình 5.2: Khảo sát chuỗi giá, phân bố Close, biến động và Volume của Amazon.

Cell 10 — xây và truy vấn Knowledge Graph (notebook gốc).

```
graph = a07.build_graph(df)
print("Số nút:", graph.number_of_nodes(), "| Số cạnh:",
↪  graph.number_of_edges())
print("Bộ ba mẫu:")
for triple in a07.graph_triples(graph): print(triple)
node = a07.query_graph(graph, "2023-12-05")
print("Graph Close:", node["Close"])
assert node["Close"] == float(df.loc[df["Date"].eq("2023-12-05"),
↪  "Close"].iloc[0])
```

Output của cell:

```
Số nút: 7032 | Số cạnh: 7031
Bộ ba mẫu:
('AMZN', 'has_year', 'year:1997')
('year:1997', 'has_month', 'month:1997-05')
('month:1997-05', 'has_session', 'day:1997-05-15')
('AMZN', 'has_year', 'year:1998')
('AMZN', 'has_year', 'year:1999')
('AMZN', 'has_year', 'year:2000')
Graph Close: 146.880005
```

Cell 15 — lập chỉ mục và truy xuất RAG (notebook gốc).

```
documents = a07.build_documents(df)
vectorizer, matrix = a07.build_index(documents)
```

```
print("Tài liệu:", len(documents), "| Ma trận TF-IDF:", matrix.shape)
for item in a07.retrieve("Amazon tháng 2023-12 Close và Volume",
 ↪  documents, vectorizer, matrix):
    print(item["id"], "score=", item["score"], "|", item["text"][:110])
```

Output của cell:

```
Tài liệu: 7005 | Ma trận TF-IDF: (7005, 58345)
AMZN:month:2023-12 score= 0.2643 | Amazon AMZN tháng 2023-12: 3 phiên; giá
 ↪  Close trung bình 146.2500 USD, thấp nhất 144.8400 USD, cao nhất 147.03
AMZN:month:2023-11 score= 0.2117 | Amazon AMZN tháng 2023-11: 21 phiên;
 ↪  giá Close trung bình 143.4576 USD, thấp nhất 137.0000 USD, cao nhất
 ↪  147.7
AMZN:month:2023-10 score= 0.2104 | Amazon AMZN tháng 2023-10: 22 phiên;
 ↪  giá Close trung bình 128.2786 USD, thấp nhất 119.5700 USD, cao nhất
 ↪  133.0
AMZN:month:2023-02 score= 0.1867 | Amazon AMZN tháng 2023-02: 19 phiên;
 ↪  giá Close trung bình 99.2142 USD, thấp nhất 93.5000 USD, cao nhất
 ↪  112.910
```

Cell 17 — agent chọn nhánh công cụ (notebook gốc).

```
agent = a07.AmazonAgent(df)
for question in ["Giá Close ngày 2023-12-05 là bao nhiêu?", "Giá Close từ
 ↪  2023-12-01 đến 2023-12-05?", "Giá mới nhất trong dữ liệu là gì?"]:
    print("\nHỏi:", question)
    print(agent.ask(question, generate=False))
print("\nRAG không sinh:", agent.ask("Tóm tắt dữ liệu Amazon AMZN",
 ↪  generate=False))
```

Output của cell:

```

Hỏi: Giá Close ngày 2023-12-05 là bao nhiêu?
{'answer': 'Ngày 2023-12-05, Close của AMZN là 146.8800 USD.', 'sources':
 ↪  ['AMZN:2023-12-05'], 'tools': ['knowledge_graph']}

Hỏi: Giá Close từ 2023-12-01 đến 2023-12-05?
{'answer': 'Từ 2023-12-01 đến 2023-12-05 có 3 phiên. Close: trung bình
 ↪  146.2500 USD; thấp nhất 144.8400 USD; cao nhất 147.0300 USD.',
 ↪  'sources': ['AMZN.csv#Date=2023-12-01..2023-12-05'], 'tools':
 ↪  ['csv_statistics']}

Hỏi: Giá mới nhất trong dữ liệu là gì?
{'answer': 'Phiên cuối trong CSV là 2023-12-05; Close 146.8800 USD. Đây
 ↪  không phải giá hiện tại.', 'sources': ['AMZN:2023-12-05'], 'tools':
 ↪  ['knowledge_graph']}
```

```
RAG không sinh: {'answer': 'Tóm tắt dữ liệu Amazon AMZN: 6684 phiên giao
↪  dịch từ 1997-05-15 đến 2023-12-05. Dataset gồm Date, Open, High, Low,
↪  Close, Adj Close và Volume. Đây là giá lịch sử trong CSV, không phải
↪  giá hiện tại hoặc dự báo.', 'sources': ['AMZN:overview'], 'tools':
↪  ['rag_retrieval']}
```

Cell 19 — sinh câu trả lời RAG bằng Qwen3 cục bộ (notebook gốc).

```
RUN_LOCAL_LLM = True
if RUN_LOCAL_LLM:
    print(agent.ask("Tóm tắt dữ liệu Amazon AMZN", generate=True))
else:
    print("Đặt RUN_LOCAL_LLM=True để kiểm tra lượt sinh bằng Qwen3 cục
    ↪  bộ.")
```

Output của cell:

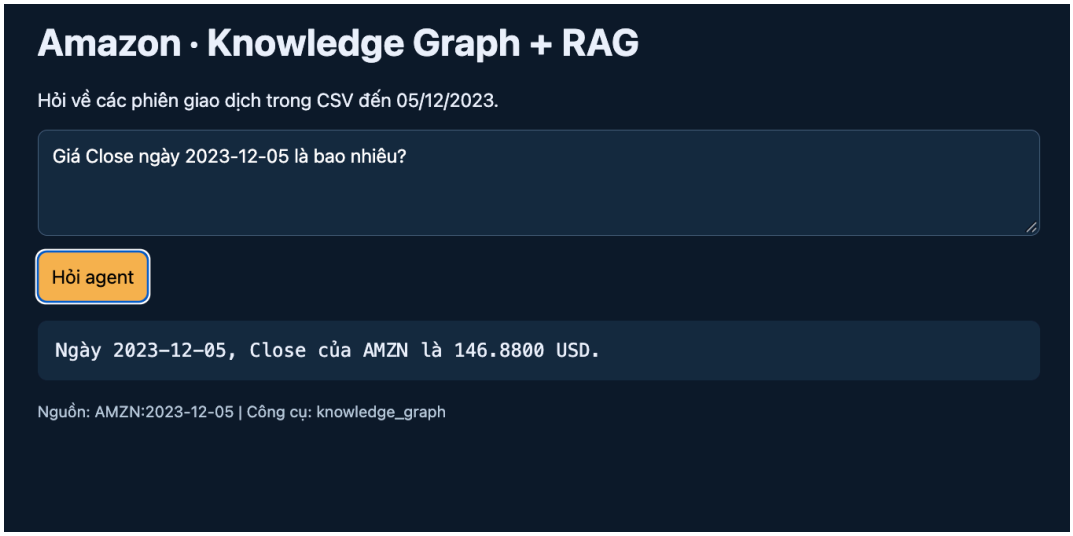
```
{'answer': 'Tóm tắt dữ liệu Amazon AMZN: 6684 phiên giao dịch từ
↪  1997-05-15 đến 2023-12-05.', 'sources': ['AMZN:overview'], 'tools':
↪  ['rag_retrieval', 'local_llm']}
```

Graph có 7.032 node và 7.031 cạnh. Node phiên 05/12/2023 lưu Close 146,880005 USD, được hiển thị làm tròn thành 146,88 USD. Corpus RAG có 7.005 tài liệu, ma trận TF-IDF kích thước 7.005×58.345. Trong ví dụ truy xuất theo tháng, tài liệu `AMZN:month:2023-12` được xếp đầu với điểm cosine 0,2643. Điểm này nói về độ giống văn bản, không phải độ tin cậy của giá.

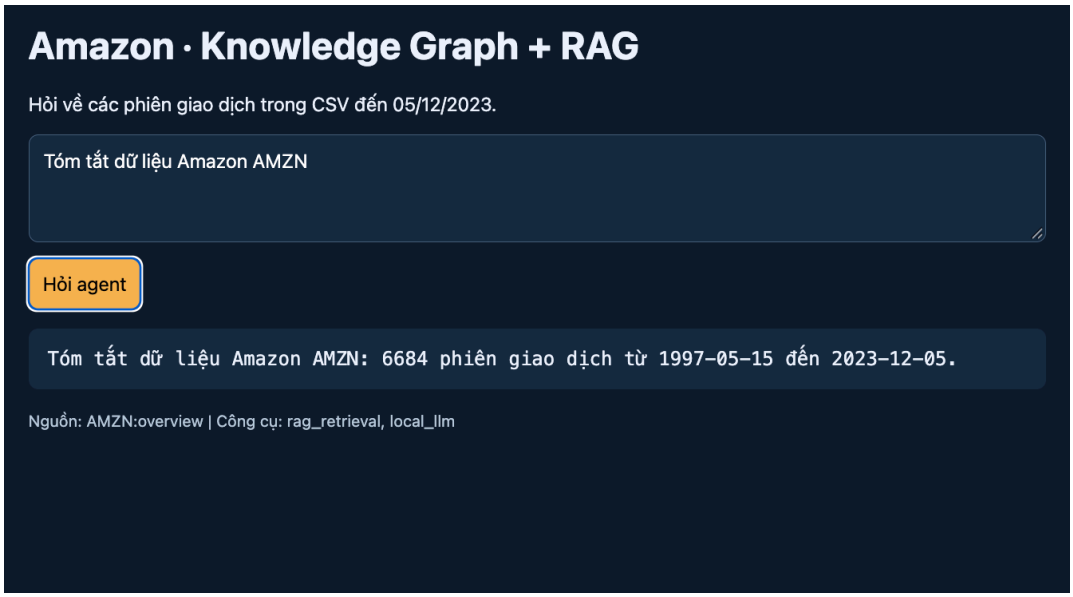
Bảng 5.2: Câu hỏi minh họa trên Amazon Stock Price

Câu hỏi	Trả lời từ dữ liệu	Nhánh
Close ngày 05/12/2023?	146,8800 USD; nguồn `AMZN:2023-12-05`	Graph
Close từ 01 đến 05/12/2023?	3 phiên; trung bình 146,2500 USD; min 144,8400; max 147,0300	CSV
Tóm tắt dữ liệu Amazon?	6.684 phiên; nguồn `AMZN:overview`	RAG + LLM

Giao diện web minh họa hai lối trả lời khác nhau. Hình 5.3 đọc giá của ngày chỉ định từ graph; Hình 5.4 dùng tài liệu tổng quan và Qwen3. Cả hai hiển thị nguồn và công cụ bên dưới câu trả lời.



Hình 5.3: Amazon: hỏi Close ngày 05/12/2023 qua Knowledge Graph.



Hình 5.4: Amazon: câu hỏi tóm tắt đi qua RAG và Qwen3.

5.5 diabetes_large: hỏi đáp thống kê theo nhóm

Mã nguồn GitHub: `https://github.com/KhoiF/PTHTTM/tree/main/src/Assignment%2007/diabetes_large`

CSV gốc có 100.000 bản ghi với chín cột; `age_group` là cột được tạo thêm khi xử lý, không phải cột thứ mười của CSV. Notebook thấy 3.854 dòng trùng toàn bộ thuộc tính và 8.500 bản ghi nhãn 1, bằng 8,50%. Các đồ thị EDA mô tả nhãn, tuổi, BMI, HbA1c, glucose cùng một số nhóm danh mục. Dữ liệu quan sát này không thiết lập nguyên nhân gây bệnh.

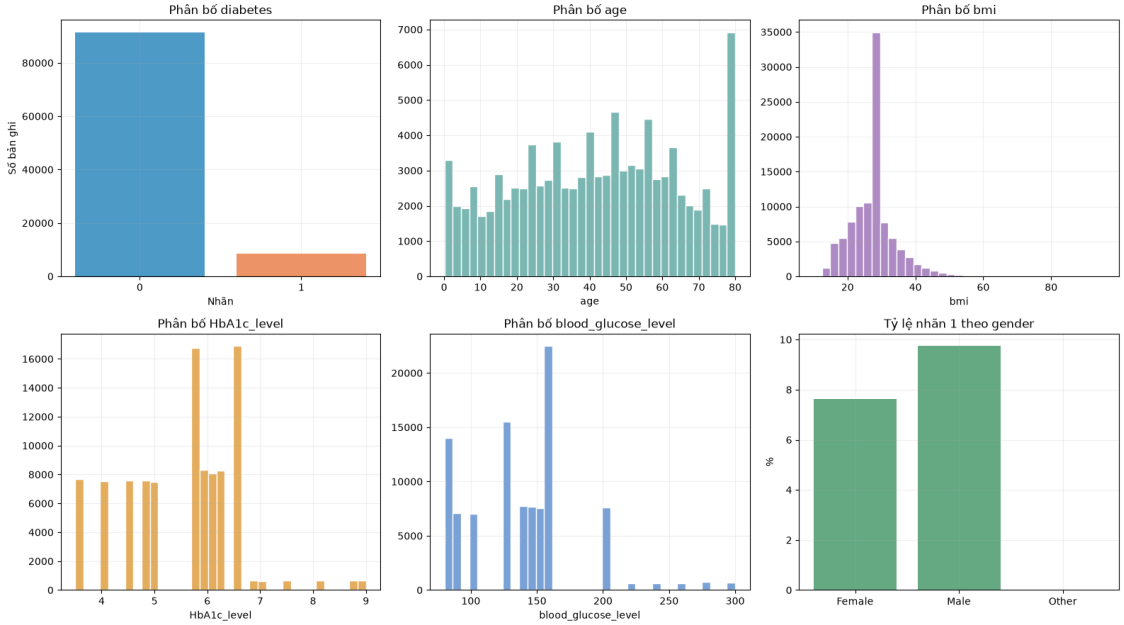
Cell 8 — EDA (notebook gốc).

```
fig, axes = plt.subplots(2, 3, figsize=(16, 9))
counts = df["diabetes"].value_counts().sort_index()
axes[0,0].bar(counts.index.astype(str), counts.values, color=["#4e9ac7",
↪  "#ec9467"])
axes[0,0].set(title="Phân bố diabetes", xlabel="Nhãn", ylabel="Số bản
↪  ghi")
for ax, col, color in
↪  [(axes[0,1],"age","#7bb7b2"),(axes[0,2],"bmi","#ae8ac4"),
                        (axes[1,0],"HbA1c_level","#e4ad5d"),(axes[1,1],"b
                        ↪  lood_glucose_level","#79a1d6")]:
    ax.hist(df[col], bins=35, color=color, edgecolor="white")
    ax.set(title=f"Phân bố {col}", xlabel=col)
by_gender = df.groupby("gender")["diabetes"].agg(["size","mean"])
axes[1,2].bar(by_gender.index, by_gender["mean"]*100, color="#65a982")
axes[1,2].set(title="Tỷ lệ nhãn 1 theo gender", ylabel="%")
for ax in axes.flat: ax.grid(alpha=.2)
plt.tight_layout(); plt.show()
display(df.groupby("smoking_history")["diabetes"].agg(["size","mean"]).r
↪  ename(columns={"size":"count","mean":"rate"}))
```

Output của cell:

```
index=No Info; count=35816; rate=0.040596
index=current; count=9286; rate=0.102089
index=ever; count=4004; rate=0.117882
index=former; count=9352; rate=0.170017
index=never; count=35095; rate=0.095341
index=not current; count=6447; rate=0.107027
```

Output hình của cell: Hình 5.5.



Hình 5.5: Phân bố nhãn và các đặc trưng trong diabetes_large.

Cell 10 — xây và truy vấn Knowledge Graph (notebook gốc).

```
graph = a07.build_graph(df)
print("Số nút:", graph.number_of_nodes(), "| Số cạnh:",
↪  graph.number_of_edges())
for triple in a07.graph_triples(graph): print(triple)
node = a07.query_graph(graph, "gender", "Female")
print("Nhóm Female:", node)
raw = df.loc[df["gender"].eq("Female"), "diabetes"]
assert node["count"] == len(raw)
assert node["positive"] == int(raw.sum())
```

Output của cell:

```
Số nút: 20 | Số cạnh: 51
('dataset:diabetes_large', 'has_cohort', 'cohort:gender:Female')
('cohort:gender:Female', 'positive_count', 'outcome:1')
('cohort:gender:Female', 'negative_count', 'outcome:0')
('dataset:diabetes_large', 'has_cohort', 'cohort:gender:Male')
('dataset:diabetes_large', 'has_cohort', 'cohort:gender:Other')
('dataset:diabetes_large', 'has_cohort', 'cohort:smoking_history:No Info')
('dataset:diabetes_large', 'has_cohort', 'cohort:smoking_history:current')
('dataset:diabetes_large', 'has_cohort', 'cohort:smoking_history:ever')
Nhóm Female: {'kind': 'cohort', 'dimension': 'gender', 'value': 'Female',
↪  'count': 58552, 'positive': 4461, 'rate': 0.0761886869790955}
```

Cell 15 — lập chỉ mục và truy xuất RAG (notebook gốc).

```
documents = a07.build_documents(df)
vectorizer, matrix = a07.build_index(documents)
print("Tài liệu:", len(documents), "| Ma trận TF-IDF:", matrix.shape)
for item in a07.retrieve("Tóm tắt diabetes gender Female", documents,
↪  vectorizer, matrix):
    print(item["id"], "score=", item["score"], "|", item["text"][:120])
```

Output của cell:

```
Tài liệu: 18 | Ma trận TF-IDF: (18, 1351)
DIABETES:gender:Female score= 0.4447 | Nhóm gender=Female: 58552 bản ghi,
↪  4461 nhãn diabetes=1, tỷ lệ 7.62%. Tỷ lệ mô tả dataset, không chứng
↪  minh quan hệ nhân
DIABETES:gender:Male score= 0.2921 | Nhóm gender=Male: 41430 bản ghi, 4039
↪  nhãn diabetes=1, tỷ lệ 9.75%. Tỷ lệ mô tả dataset, không chứng minh
↪  quan hệ nhân q
DIABETES:gender:Other score= 0.2535 | Nhóm gender=Other: 18 bản ghi, 0
↪  nhãn diabetes=1, tỷ lệ 0.00%. Tỷ lệ mô tả dataset, không chứng minh
↪  quan hệ nhân quả.
DIABETES:overall score= 0.15 | Dataset diabetes_large có 100000 bản ghi,
↪  8500 nhãn diabetes=1, 91500 nhãn diabetes=0; tỷ lệ nhãn 1 là 8.50%.
↪  Các trường
```

Cell 17 — agent chọn nhánh công cụ (notebook gốc).

```
agent = a07.DiabetesAgent(df)
for question in ["Tỷ lệ diabetes ở nữ là bao nhiêu?", "Tỷ lệ diabetes
↪  toàn dataset là bao nhiêu?", "Tỷ lệ diabetes nhóm 60+ là bao
↪  nhiêu?"]:
    print("\nHỏi:", question)
    print(agent.ask(question, generate=False))
print("\nRAG không sinh:", agent.ask("Tóm tắt dữ liệu diabetes_large",
↪  generate=False))
```

Output của cell:

```
Hỏi: Tỷ lệ diabetes ở nữ là bao nhiêu?
{'answer': 'Nhóm gender=Female có 58,552 bản ghi; 4,461 nhãn diabetes=1;
↪  tỷ lệ 7.62%.', 'sources': ['DIABETES:gender:Female'], 'tools':
↪  ['knowledge_graph']}

Hỏi: Tỷ lệ diabetes toàn dataset là bao nhiêu?
{'answer': 'CSV có 100,000 bản ghi; 8,500 nhãn diabetes=1; tỷ lệ 8.50%.',
↪  'sources': ['DIABETES:overall'], 'tools': ['csv_statistics']}

Hỏi: Tỷ lệ diabetes nhóm 60+ là bao nhiêu?
{'answer': 'Nhóm age_group=60+ có 25,055 bản ghi; 4,932 nhãn diabetes=1;
↪  tỷ lệ 19.68%.', 'sources': ['DIABETES:age_group:60+'], 'tools':
↪  ['knowledge_graph']}

RAG không sinh: {'answer': 'Dataset diabetes_large có 100000 bản ghi, 8500
↪  nhãn diabetes=1, 91500 nhãn diabetes=0; tỷ lệ nhãn 1 là 8.50%. Các
↪  trường: gender, age, hypertension, heart_disease, smoking_history,
↪  bmi, HbA1c_level, blood_glucose_level, diabetes. Đây là dữ liệu quan
↪  sát, không phải chẩn đoán.', 'sources': ['DIABETES:overall'], 'tools':
↪  ['rag_retrieval']}
```

Cell 19 — sinh câu trả lời RAG bằng Qwen3 cục bộ (notebook gốc).

```
RUN_LOCAL_LLM = True
if RUN_LOCAL_LLM:
    print(agent.ask("Tóm tắt dữ liệu diabetes_large", generate=True))
else:
    print("Đặt RUN_LOCAL_LLM=True để kiểm tra lượt sinh bằng Qwen3 cục
    ↪  bộ.")
```

Output của cell:

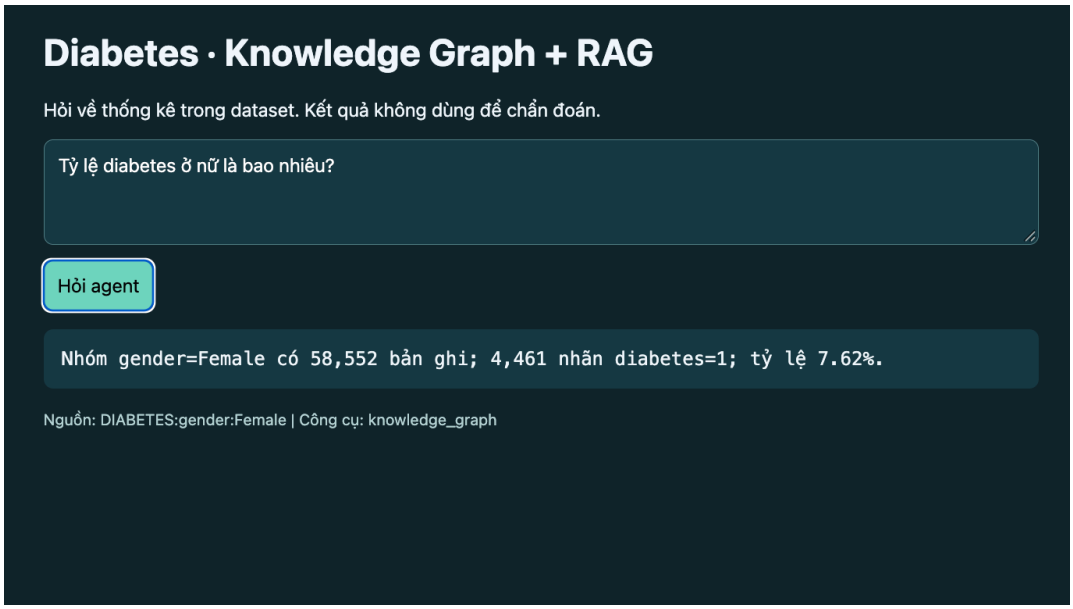
```
{'answer': 'Tóm tắt dữ liệu diabetes_large: có 100,000 bản ghi, 8500 nhãn
↪  1 và 91500 nhãn 0, tỷ lệ nhãn 1 là 8.5%.', 'sources':
↪  ['DIABETES:overall'], 'tools': ['rag_retrieval', 'local_llm']}
```

Graph thống kê có 20 node và 51 cạnh; corpus gồm 18 tài liệu, ma trận TF-IDF kích thước 18×1.351. Nhóm nữ có 58.552 bản ghi, 4.461 nhãn `diabetes=1`, tỷ lệ 7,62%. Tài liệu `DIABETES:gender:Female` đứng đầu trong ví dụ truy xuất nhóm nữ với điểm $0{,}4447$. Tỷ lệ này là số liệu của tập được cung cấp, không phải kết quả chẩn đoán cho người dùng.

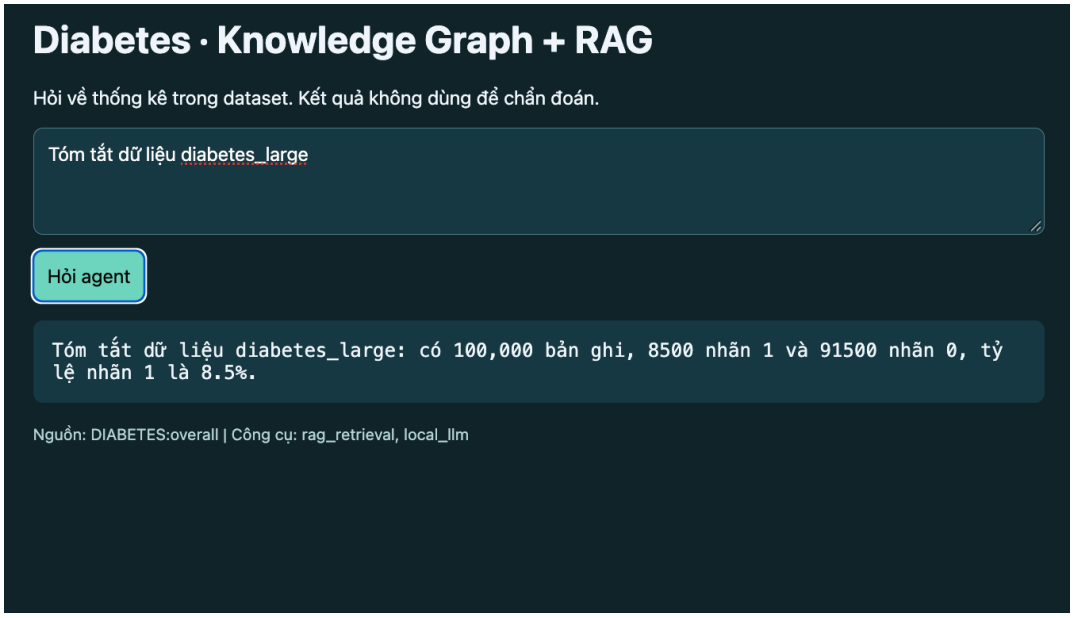
Bảng 5.3: Câu hỏi minh họa trên diabetes_large

Câu hỏi	**Trả lời từ dữ liệu**	**Nhánh**
Tỷ lệ diabetes ở nữ?	58.552 bản ghi; 4.461 nhãn 1; 7,62%	Graph
Tỷ lệ toàn dataset?	100.000 bản ghi; 8.500 nhãn 1; 8,50%	CSV
Tỷ lệ nhóm 60+?	25.055 bản ghi; 4.932 nhãn 1; 19,68%	Graph
Tóm tắt dataset?	100.000 bản ghi; nguồn `DIABETES:overall`	RAG + LLM

Hình 5.6 minh họa câu hỏi theo nhóm được graph xử lý; Hình 5.7 minh họa câu hỏi tóm tắt từ tài liệu tổng quan. Giao diện nhắc rõ rằng thống kê không dùng để chẩn đoán.



Hình 5.6: Diabetes: tỷ lệ nhãn 1 ở nữ được trả từ Knowledge Graph.



Hình 5.7: Diabetes: câu hỏi tóm tắt đi qua RAG và Qwen3.

5.6 Triển khai và giới hạn

Hai ứng dụng có lệnh CLI `--question` và giao diện Flask `--web` trên localhost; Amazon dùng cổng mặc định 8007, diabetes dùng 8008. API nhận câu hỏi và trả câu trả lời, nguồn cùng công cụ đã dùng. Graph và phép tính CSV không cần nạp Qwen3; nhánh RAG mới nạp mô hình khi cần. Việc đưa LLM vào câu hỏi mô tả không biến hệ thành công cụ dự báo giá hay phân loại bệnh của các Assignment trước.

TF-IDF ký tự phụ thuộc mức giống bề mặt giữa câu hỏi và tài liệu. Nếu cách diễn đạt khác xa corpus, tài liệu đúng có thể không được tìm thấy; nếu truy xuất được tài liệu gần từ vựng, mô hình vẫn có thể lược bỏ chi tiết. ID nguồn giúp kiểm tra lại nhưng không tự bảo đảm groundedness. Giá Amazon kết thúc ở 05/12/2023; tỷ lệ diabetes chỉ mô tả dữ liệu quan sát. Ứng dụng do đó thích hợp để minh họa hỏi đáp và thống kê trên hai CSV, không dùng cho quyết định đầu tư hoặc chẩn đoán y khoa.

Chương 6

Agent học tăng cường và agent dựa trên luật

6.1 Giới thiệu và mục tiêu

Thí nghiệm Agent 02 so sánh hai cách xây dựng agent ra quyết định trong cùng một môi trường: **agent học tăng cường** (RL) dùng Q-learning dạng bảng, và **agent dựa trên luật** (rule-based) dùng chính sách if–else do người viết sẵn [17, 18]. Hai người bán cùng bán một loại sản phẩm; ở mỗi bước thời gian, mỗi người quyết định *giảm giá* hoặc *không giảm giá*. Giảm giá làm tăng xác suất khách mua nhưng làm giảm lợi nhuận trên mỗi sản phẩm, nên agent phải cân bằng hai tác động này.

Mục tiêu của báo cáo:

1. Mô hình hóa bài toán theo các khái niệm agent, môi trường, trạng thái, hành động, phần thưởng và chính sách; giải thích vì sao đây là hệ nhiều agent.
2. Cài đặt môi trường, agent Q-learning và agent dựa trên luật bằng Python, tách riêng giai đoạn huấn luyện và giai đoạn đánh giá.
3. Thực hiện năm thí nghiệm A–E của đề bài: chạy baseline, xem quá trình học, thay đổi tốc độ học, thay đổi mức khám phá và so sánh công bằng trên nhiều seed.
4. Trả lời mười câu hỏi bằng số liệu đo được; cài đặt và đánh giá một thay đổi môi trường (câu 9) và một agent cải tiến (câu 10).

Mã nguồn trên GitHub: `https://github.com/KhoiF/PTHTTM`. Chương trình nằm tại `src/Agent02/agent02_experiment.py`; toàn bộ kết quả (CSV, PNG, bảng Q và nhật ký chạy `run_log.txt`) nằm trong `src/Agent02/agent02_results/`. Mọi con số trong báo cáo được lấy từ các tệp này, không có số liệu ước đoán. Các đoạn code trong báo cáo được trích tự động từ tệp nguồn khi biên dịch LaTeX, số dòng ở lề trái là số dòng trong tệp.

Kết luận chính. Trong môi trường tham chiếu (giảm giá 20%), chính sách tối ưu khi đối thủ là agent dựa trên luật là *không bao giờ giảm giá*, đạt lợi nhuận kỳ vọng 35,00 mỗi bước. Agent dựa trên luật cũng không bao giờ giảm giá trong tình huống này nên đã đạt đúng mức tối ưu. Agent RL học được chính sách tối ưu ở 9/10 seed, đạt trung bình $34{,}72 \pm 0{,}75$; chênh lệch so với agent dựa trên luật không có ý nghĩa thống kê. Khi đổi mức giảm giá xuống 5%, luật cố

định không còn tối ưu và RL vượt luật (36,72 so với 34,99). Agent dựa trên mô hình đề xuất ở câu 10 gần đạt tối ưu trong cả sáu kịch bản thử.

6.2 Môi trường và hai agent

6.2.1 Trạng thái, hành động và động lực môi trường

Với agent $i \in \{1, 2\}$, trạng thái cục bộ tại thời điểm t là

$$s_i(t) = (d_t,\ c_i(t)), \qquad d_t \in \{0, 1\},\ c_i(t) = a_{-i}(t-1) \in \{0, 1\}, \tag{6.1}$$

trong đó d_t là nhu cầu thị trường (0: thấp, 1: cao) và $c_i(t)$ là hành động của đối thủ ở bước trước. Mỗi agent có 4 trạng thái và 2 hành động $a_i(t) \in \{0, 1\}$ (0: không giảm giá, 1: giảm giá). Hai agent chọn hành động *đồng thời*, nên không agent nào biết hành động hiện tại của đối thủ.

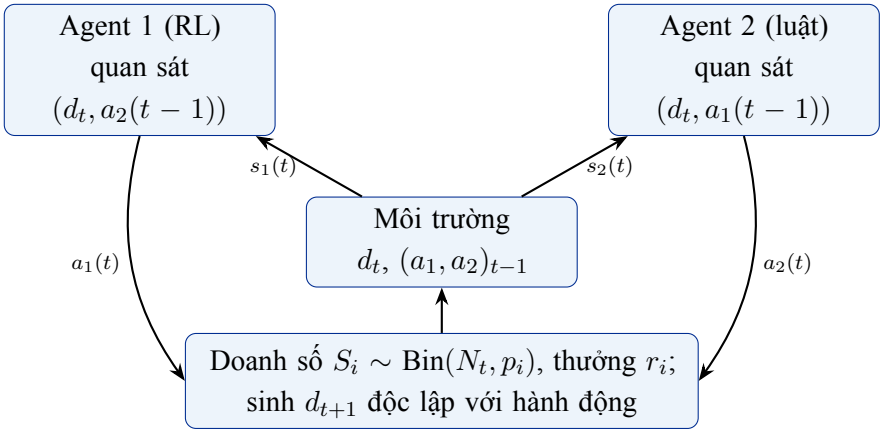
Xác suất một khách tiềm năng mua hàng của agent i là

$$p_i(t) = \text{clip}\big(0{,}30 + 0{,}25 d_t + 0{,}15 a_i(t) - 0{,}10 a_{-i}(t),\ 0{,}05,\ 0{,}95\big). \tag{6.2}$$

Số khách tiềm năng là $N_t = 10$ khi $d_t = 0$ và $N_t = 20$ khi $d_t = 1$. Số đơn bán được $S_i(t) \sim \text{Binomial}(N_t, p_i(t))$. Với giá gốc $P = 10$, giá vốn $C = 5$ và mức giảm $\delta = 0{,}20$, phần thưởng là lợi nhuận

$$r_i(t) = S_i(t)\big[P(1 - \delta a_i(t)) - C\big]. \tag{6.3}$$

Sau khi trả thưởng, môi trường sinh d_{t+1} ngẫu nhiên đều, độc lập với hành động, và hành động hiện tại trở thành "hành động trước" của bước sau. Mỗi episode có 20 bước; đầu episode, nhu cầu được rút ngẫu nhiên và hành động trước của cả hai agent được đặt bằng 0.



Hình 6.1: Vòng tương tác của hai agent trong một bước thời gian.

6.2.2 Agent A: Q-learning dạng bảng

Agent RL giữ bảng $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ gồm $4 \times 2 = 8$ giá trị. Sau mỗi bước, giá trị của cặp trạng thái–hành động vừa dùng được cập nhật

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big], \tag{6.4}$$

với $\alpha = 0{,}10$, $\gamma = 0{,}90$; ở bước cuối episode, mục tiêu chỉ còn r_t. Hành động được chọn theo ε-greedy: ngẫu nhiên với xác suất ε, ngược lại chọn $\arg\max_a Q(s_t, a)$ (hòa thì chọn ngẫu nhiên). ε giảm theo $\varepsilon_e = \max(0{,}02;\ 0{,}20 \cdot 0{,}995^e)$ ở episode e.

6.2.3 Agent B: chính sách dựa trên luật

$$\pi_{\text{rule}}(d, c) = \begin{cases} 0, & d = 0, \\ 1, & d = 1 \text{ và } c = 1, \\ 0, & d = 1 \text{ và } c = 0. \end{cases} \tag{6.5}$$

Nghĩa là: nhu cầu thấp thì không giảm giá; nhu cầu cao thì giảm giá chỉ khi đối thủ vừa giảm giá ở bước trước. Agent này không dùng phần thưởng để thay đổi hành vi.

6.2.4 Phân tích lợi nhuận kỳ vọng một bước

Trước khi chạy thí nghiệm, có thể tính lợi nhuận kỳ vọng $\mathbb{E}[r_i] = N_t\, p_i\, (P(1 - \delta a_i) - C)$ cho mọi tổ hợp. Lợi nhuận mỗi sản phẩm là $10 - 5 = 5$ khi không giảm giá và $8 - 5 = 3$ khi giảm giá 20%.

Bảng 6.1: Lợi nhuận kỳ vọng một bước của agent i theo nhu cầu d và hành động hiện tại b của đối thủ (output của chương trình).

d	$b = a_{-i}(t)$	Không giảm giá	Giảm giá	Hành động tốt hơn
0	0	$10 \cdot 0{,}30 \cdot 5 = 15{,}0$	$10 \cdot 0{,}45 \cdot 3 = 13{,}5$	không giảm
0	1	$10 \cdot 0{,}20 \cdot 5 = 10{,}0$	$10 \cdot 0{,}35 \cdot 3 = 10{,}5$	giảm
1	0	$20 \cdot 0{,}55 \cdot 5 = 55{,}0$	$20 \cdot 0{,}70 \cdot 3 = 42{,}0$	không giảm
1	1	$20 \cdot 0{,}45 \cdot 5 = 45{,}0$	$20 \cdot 0{,}60 \cdot 3 = 36{,}0$	không giảm

Agent dựa trên luật không bao giờ giảm giá khi nhu cầu thấp, nên dòng thứ hai (nơi giảm giá có lợi hơn) không xảy ra khi đối thủ là agent luật. Ngoài ra, nếu agent RL giảm giá ở bước t và nhu cầu ở bước $t + 1$ cao, agent luật sẽ giảm giá ở bước $t + 1$, khiến lợi nhuận của RL giảm từ 55 xuống 45. Vì vậy không giảm giá vừa có lợi trước mắt vừa không kích hoạt phản ứng của đối thủ. Khi đối thủ là agent luật, chính sách tối ưu là *luôn không giảm giá*, cho lợi nhuận kỳ vọng $0{,}5 \cdot 15 + 0{,}5 \cdot 55 = 35{,}0$ mỗi bước. Hàm `best_response` trong chương trình xác nhận điều này bằng cách liệt kê toàn bộ 16 chính sách tất định trên trạng thái (d, c) (Mục 6.3.4.5).

Một hệ quả quan trọng: hai agent luật đối đầu nhau cũng không bao giờ giảm giá (vì hành động trước ban đầu là 0), nên agent luật đặt vào ghế của RL đạt đúng 35,0. Trong môi trường tham chiếu, RL *tốt nhất chỉ có thể hòa* agent dựa trên luật.

6.3 Cài đặt và thiết lập thí nghiệm

6.3.1 Kiến trúc chương trình

Chương trình phát triển từ mã tham khảo `agent02_experiment.py` của đề bài [18], giữ nguyên mô hình thị trường và các tham số, và thêm các phần cần cho thí nghiệm A–E và câu 8–10.

Bảng 6.2: Các thành phần chính của chương trình.

Thành phần	Vai trò
`Config`	Tham số mô phỏng, seed, số episode, kích thước cửa sổ
`ECommerceEnvironment`	Sinh nhu cầu, doanh số, phần thưởng và trạng thái tiếp theo
`QLearningAgent`	Q-learning với trạng thái (d, c) như đề bài
`ExtendedQLearningAgent`	Q-learning với trạng thái $(d, c, a_i(t-1))$ (so sánh ở câu 10)
`ModelBasedAgent`	Agent đề xuất ở câu 10: học mô hình rồi lập kế hoạch
`RuleBasedAgent`, `TitForTatAgent`	Agent luật của đề bài và đối thủ thay thế cho câu 9–10
`run_training`, `train_rl_against_rule`	Vòng huấn luyện chung cho mọi cặp agent
`evaluate_pair`	Đánh giá tách biệt: đóng băng bảng Q, $\varepsilon = 0$, seed riêng
`exact_evaluation`, `best_response`	Giá trị kỳ vọng chính xác của chính sách và chính sách tối ưu
`run_baseline`, `run_*_experiment`	Thí nghiệm A–E, câu 8, câu 9–10; ghi CSV và hình

6.3.2 Các thay đổi so với mã tham khảo

Đề bài cho phép sửa mã nếu ghi lại thay đổi và giữ so sánh hợp lệ, tái lập được [17]. Các thay đổi gồm:

M1. Số ngẫu nhiên chung. Môi trường dùng một bộ sinh số cho nhu cầu và một bộ riêng cho doanh số của từng agent, cùng tách từ một seed bằng `SeedSequence`. Khi đánh giá RL–luật và luật–luật cùng seed, hai lần chạy thấy *đúng cùng một chuỗi nhu cầu*; nếu hai agent chọn cùng hành động thì doanh số cũng giống hệt. Nhờ vậy chênh lệch đo được đến từ chính sách, không phải từ may rủi của chuỗi nhu cầu.

M2. Giao diện agent thống nhất (`observe`, `act`, `learn`). Cùng một vòng huấn luyện và đánh giá dùng được cho RL–luật, RL–RL và các agent mới. Mỗi agent tự tạo quan sát từ trạng thái chung $(d_t, a_1(t-1), a_2(t-1))$.

M3. Đánh giá chính xác. Hàm `exact_evaluation` lan truyền phân phối của trạng thái chung qua 20 bước và tính lợi nhuận kỳ vọng của chính sách tham lam mà không cần lấy mẫu. Hàm `best_response` liệt kê mọi chính sách tất định để tìm mức tối ưu cho từng cách biểu diễn trạng thái.

M4. Thí nghiệm bổ sung: đường học theo cửa sổ, ε cố định, 10 seed độc lập, hai agent cùng học, thay đổi mức giảm giá, hàm thưởng và đối thủ, agent cải tiến.

6.3.3 Tham số và quy trình đánh giá

Bảng 6.3: Tham số tham chiếu và cấu hình tái lập.

Tham số	Giá trị
Số agent	2
Số episode huấn luyện $\times$ số bước	$5.000 \times 20 = 100.000$ bước
Tốc độ học α, hệ số chiết khấu γ	0,10; 0,90
Khám phá ε	0,20 $\rightarrow$ 0,02, hệ số giảm 0,995 mỗi episode
Giá gốc, giá vốn, mức giảm	10; 5; 20%
Seed baseline (môi trường)	42; seed của agent RL $= 42 + 10.000$
Seed đánh giá baseline	$42 + 500.000$
Seed nhiều lần chạy	11, 22, 33, 44, 55, 66, 77, 88, 99, 110
Seed đánh giá nhiều lần chạy	seed huấn luyện $+800.000$
Đánh giá	500 episode $\times$ 20 bước $= 10.000$ bước, $\varepsilon = 0$, bảng Q đóng băng
Môi trường phần mềm	Python 3.12.13, NumPy 2.5.2, Matplotlib 3.11.1
Thời gian chạy toàn bộ	khoảng 40 giây trên MacBook (CPU)

Các chỉ số đo: tổng thưởng, thưởng trung bình mỗi bước $\bar{R}_i = \frac{1}{T}\sum_{t=1}^{T} r_i(t)$, số lần giảm/không giảm giá, tỷ lệ giảm giá ρ_i, bảng Q và chính sách tham lam. Với nhiều seed, báo cáo dùng trung bình và độ lệch chuẩn mẫu $(n-1)$. Chạy lại chương trình hai lần cho nhật ký giống hệt nhau, nghĩa là kết quả tái lập hoàn toàn với cùng môi trường phần mềm.

Chuỗi chính sách như `0100` liệt kê hành động tham lam ở các trạng thái theo thứ tự $(0,0), (0,1), (1,0), (1,1)$ của (d, c); 1 là giảm giá.

6.3.4 Trích đoạn mã nguồn chính

6.3.4.1 Mô hình thị trường và môi trường

Hàm xác suất chuyển đổi cài đặt công thức (6.2); `step` nhận hai hành động đồng thời, trả phần thưởng (6.3) rồi sinh nhu cầu mới độc lập.

```
def conversion_probability(demand, own_action, competitor_action):
    """p_i(t) = clip(0.30 + 0.25 d + 0.15 a_i - 0.10 a_-i, 0.05, 0.95)."""
    p = (
        0.30
        + 0.25 * demand
        + 0.15 * own_action
        - 0.10 * competitor_action
    )
    return min(max(p, 0.05), 0.95)


def effective_price(config, action):
```

```
    return config.price * (1.0 - config.discount_rate * action)


def profit_per_sale(config, action):
    return effective_price(config, action) - config.unit_cost


def reward_per_sale(config, action):
    if config.reward_mode == "revenue":
        return effective_price(config, action)
    return profit_per_sale(config, action)


def customers_for(config, demand):
    if demand == 0:
        return config.low_demand_customers
    return config.high_demand_customers
```

```
    def __init__(self, config, seed=None):
        self.config = config
        actual_seed = config.seed if seed is None else seed

        # M1: separate streams for demand and for each agent's sales.
        streams = np.random.SeedSequence(actual_seed).spawn(3)
        self.demand_rng = np.random.default_rng(streams[0])
        self.sales_rngs = [
            np.random.default_rng(stream) for stream in streams[1:]
        ]

        self.demand = 0
        self.previous_actions = (0, 0)

    def reset(self):
        self.demand = int(self.demand_rng.integers(0, 2))
        self.previous_actions = (0, 0)

    def step(self, actions):
        if len(actions) != 2:
            raise ValueError("Exactly two actions are required.")

        if any(action not in (0, 1) for action in actions):
            raise ValueError("Actions must be 0 or 1.")

        customers = customers_for(self.config, self.demand)

        rewards = []
        profits = []
        sales_list = []

        # Both actions are selected before this function is called.
        # Neither agent observes the other's current action beforehand.
        for i in range(2):
            conversion = conversion_probability(
                self.demand, actions[i], actions[1 - i]
            )

            sales = int(self.sales_rngs[i].binomial(customers, conversion))

            rewards.append(sales * reward_per_sale(self.config, actions[i]))
            profits.append(sales * profit_per_sale(self.config, actions[i]))
            sales_list.append(sales)

        # Store current actions for the next observation.
        self.previous_actions = tuple(actions)

        # Demand evolves independently of current actions.
        self.demand = int(self.demand_rng.integers(0, 2))

        info = {
            "sales": sales_list,
            "profits": profits,
            "customers": customers
        }

```

```
        return rewards, info
```

6.3.4.2 Agent Q-learning

`act` cài đặt ε-greedy với phá hòa ngẫu nhiên; `learn` cài đặt công thức (6.4).

```
    def observe(self, demand, previous_actions, i):
        return (demand, previous_actions[1 - i])

    def set_episode(self, episode):
        # Decrease exploration as training progresses.
        self.epsilon = max(
            self.epsilon_min,
            self.epsilon_start * self.epsilon_decay ** episode
        )

    def finalize(self):
        pass

    def act(self, state, explore=True):
        # Exploration.
        if explore and self.rng.random() < self.epsilon:
            return int(self.rng.integers(0, 2))

        # Exploitation with random tie-breaking.
        q0, q1 = self.q[state]

        if math.isclose(q0, q1, rel_tol=1e-9, abs_tol=1e-12):
            return int(self.rng.integers(0, 2))

        return int(q1 > q0)

    def discount_probability(self, state):
        """Probability of action 1 under the greedy evaluation policy."""
        q0, q1 = self.q[state]

        if math.isclose(q0, q1, rel_tol=1e-9, abs_tol=1e-12):
            return 0.5

        return float(q1 > q0)

    def learn(self, state, action, reward, next_state, terminal=False):
        old_value = self.q[state + (action,)]

        if terminal:
            target = reward
        else:
            target = reward + self.gamma * np.max(self.q[next_state])

        # Q-learning update.
        self.q[state + (action,)] = old_value + self.alpha * (
            target - old_value
        )
```

6.3.4.3 Agent dựa trên luật

```
class RuleBasedAgent:
    """
    Fixed policy:

    Rule 1: Low demand -> no discount.
    Rule 2: High demand and competitor previously discounted
            -> offer a discount.
    Rule 3: High demand and competitor did not previously discount
            -> no discount.

    This agent does not learn from rewards.
    """
```

```
    learns = False

    def observe(self, demand, previous_actions, i):
        return (demand, previous_actions[1 - i])

    def set_episode(self, episode):
        pass

    def finalize(self):
        pass

    def act(self, state, explore=False):
        demand, competitor_previous_action = state

        if demand == 0:
            return 0

        elif demand == 1 and competitor_previous_action == 1:
            return 1

        else:
            return 0

    def discount_probability(self, state):
        return float(self.act(state))
```

6.3.4.4 Vòng huấn luyện

Hai agent quan sát và chọn hành động trước khi môi trường thay đổi; chỉ agent có `learns = True` cập nhật bảng Q.

```
    for episode in range(config.episodes):

        for agent in agents:
            agent.set_episode(episode)

        env.reset()

        episode_rewards = [0.0, 0.0]
        episode_discounts = [0, 0]

        for step in range(T):

            # Both agents choose actions before the environment changes.
            states = [
                agent.observe(env.demand, env.previous_actions, i)
                for i, agent in enumerate(agents)
            ]

            actions = [
                agent.act(states[i], explore=True)
                for i, agent in enumerate(agents)
            ]

            rewards, info = env.step(actions)

            terminal = step == T - 1

            for i, agent in enumerate(agents):
                if agent.learns:
                    next_state = agent.observe(
                        env.demand, env.previous_actions, i
                    )
                    agent.learn(
                        states[i], actions[i], rewards[i],
                        next_state, terminal
                    )

                episode_rewards[i] += rewards[i]
```

```
                episode_discounts[i] += actions[i]
```

6.3.4.5 Đánh giá tách biệt và đánh giá chính xác

`evaluate_pair` chạy chính sách đã đóng băng với $\varepsilon = 0$ trên seed riêng. `exact_evaluation` tính kỳ vọng chính xác: ở mỗi bước, với mỗi trạng thái chung và mỗi tổ hợp hành động, cộng thưởng kỳ vọng $N_t p_i \cdot m_i$ (m_i là biên lợi nhuận) có trọng số xác suất, rồi chuyển khối xác suất sang trạng thái kế tiếp.

```
def evaluate_pair(config, agents, episodes=None, seed=900_000):
    """
    Evaluate two frozen policies.

    Important:
        - No Q-table updates occur.
        - RL exploration is disabled.
        - Evaluation uses a separate environment random seed.
    """

    if episodes is None:
        episodes = config.evaluation_episodes

    env = ECommerceEnvironment(config, seed=seed)
    T = config.steps_per_episode

    total_rewards = np.zeros(2)
    total_profits = np.zeros(2)
    action_counts = np.zeros((2, 2), dtype=int)

    episode_results = []

    for episode in range(episodes):

        env.reset()
        rewards_this_episode = [0.0, 0.0]

        for _ in range(T):
            states = [
                agent.observe(env.demand, env.previous_actions, i)
                for i, agent in enumerate(agents)
            ]

            # Greedy policy: no exploration.
            actions = [
                agent.act(states[i], explore=False)
                for i, agent in enumerate(agents)
            ]

            rewards, info = env.step(actions)

            for i in range(2):
                total_rewards[i] += rewards[i]
                total_profits[i] += info["profits"][i]
                rewards_this_episode[i] += rewards[i]
                action_counts[i, actions[i]] += 1

        episode_results.append({
            "episode": episode + 1,
            "agent1_reward": rewards_this_episode[0],
            "agent2_reward": rewards_this_episode[1],
            "agent1_mean_reward_per_step": rewards_this_episode[0] / T,
            "agent2_mean_reward_per_step": rewards_this_episode[1] / T
        })

    total_steps = episodes * T

    summary = []

    for i in range(2):
```

```
        summary.append({
            "seat": AGENT_NAMES[i],
            "total_reward": total_rewards[i],
            "mean_reward_per_step": total_rewards[i] / total_steps,
            "mean_profit_per_step": total_profits[i] / total_steps,
            "no_discount_actions": int(action_counts[i, 0]),
            "discount_actions": int(action_counts[i, 1]),
            "discount_action_rate": action_counts[i, 1] / total_steps
        })

    return summary, episode_results
```

```
def exact_evaluation(config, agents):
    """
    Expected mean reward/profit per step and discount rate of two
    frozen greedy policies under the evaluation protocol:
    T-step episodes, uniform initial demand, previous actions (0, 0).

    The joint state (d, a_1(t-1), a_2(t-1)) is Markov, so its
    distribution is propagated step by step without sampling.
    """

    T = config.steps_per_episode

    distribution = {(0, 0, 0): 0.5, (1, 0, 0): 0.5}

    reward = np.zeros(2)
    profit = np.zeros(2)
    discount = np.zeros(2)

    for _ in range(T):
        next_distribution = defaultdict(float)

        for (demand, p0, p1), mass in distribution.items():
            previous = (p0, p1)
            customers = customers_for(config, demand)

            probs = [
                agent.discount_probability(
                    agent.observe(demand, previous, i)
                )
                for i, agent in enumerate(agents)
            ]

            for actions in product((0, 1), repeat=2):
                weight = mass
                for i in range(2):
                    weight *= probs[i] if actions[i] else 1.0 - probs[i]

                if weight == 0.0:
                    continue

                for i in range(2):
                    sales = customers * conversion_probability(
                        demand, actions[i], actions[1 - i]
                    )
                    reward[i] += weight * sales * reward_per_sale(
                        config, actions[i]
                    )
                    profit[i] += weight * sales * profit_per_sale(
                        config, actions[i]
                    )
                    discount[i] += weight * actions[i]

                for next_demand in (0, 1):
                    next_distribution[
                        (next_demand,) + actions
                    ] += 0.5 * weight

        distribution = next_distribution

    return {
        "reward": reward / T,
        "profit": profit / T,
```

```
        "discount_rate": discount / T
    }


def best_response(config, opponent, agent_class=QLearningAgent):
    """
    Best deterministic stationary policy for seat 0 over the state
    representation of agent_class, found by exhaustive enumeration
    (16 policies for the reference state, 256 for the extended one).
    """

    prototype = agent_class()
    states = prototype.states()

    best_value = -np.inf
    best_table = None

    for bits in product((0, 1), repeat=len(states)):
        table = dict(zip(states, bits))
        policy = TablePolicyAgent(table, prototype.observe)

        value = exact_evaluation(config, [policy, opponent])["reward"][0]

        if value > best_value + 1e-9:
            best_value = value
            best_table = table

    return best_table, best_value
```

6.4 Kết quả thí nghiệm

6.4.1 Thí nghiệm A: baseline

Agent RL được huấn luyện 5.000 episode với seed 42 trước agent luật, sau đó được đánh giá riêng trong 500 episode. Để so sánh công bằng, agent luật cũng được đặt vào đúng ghế của RL và chạy với cùng seed đánh giá.

```
Separate evaluation results (500 episodes, seed 500042):
RL vs Rule    | RL (Q-learning)  | total =  345565.0 | mean reward/step = 34.5565 | discount
↪  =     0 | no discount = 10000
RL vs Rule    | Rule-based       | total =  346720.0 | mean reward/step = 34.6720 | discount
↪  =     0 | no discount = 10000
Rule vs Rule  | Rule-based       | total =  345565.0 | mean reward/step = 34.5565 | discount
↪  =     0 | no discount = 10000
Rule vs Rule  | Rule-based       | total =  346720.0 | mean reward/step = 34.6720 | discount
↪  =     0 | no discount = 10000

Expected reward/step (exact): RL greedy = 35.0000, rule in seat 1 = 35.0000,
best (d, c) policy = 35.0000 {(0, 0): 0, (0, 1): 0, (1, 0): 0, (1, 1): 0}
```

Bảng 6.4: Kết quả đánh giá baseline (seed 42, 10.000 bước đánh giá).

Lần chạy	Agent	Tổng thưởng	TB/bước	Giảm giá	Không giảm
RL vs Luật	RL (ghế 1)	345.565	34,5565	0	10.000
RL vs Luật	Luật (ghế 2)	346.720	34,6720	0	10.000
Luật vs Luật	Luật (ghế 1)	345.565	34,5565	0	10.000
Luật vs Luật	Luật (ghế 2)	346.720	34,6720	0	10.000

Chính sách tham lam học được là *không giảm giá ở cả bốn trạng thái*. Agent luật cũng không giảm giá lần nào, nên hai lần chạy cho kết quả trùng đến từng đơn vị nhờ dùng chung số ngẫu nhiên (M1). Chênh lệch giữa ghế 1 (34,56) và ghế 2 (34,67) chỉ do doanh số ngẫu nhiên khác nhau; giá trị kỳ vọng của cả hai là 35,00.

```
Q-table: Q[d, c, action]; Action 0 = no discount; Action 1 = discount
State=(0, 0) | Q0=209.034622 | Q1=200.012813 | Greedy action=0
State=(0, 1) | Q0=227.101684 | Q1=181.167889 | Greedy action=0
State=(1, 0) | Q0=266.318271 | Q1=228.504950 | Greedy action=0
State=(1, 1) | Q0=228.334394 | Q1=219.318896 | Greedy action=0
```

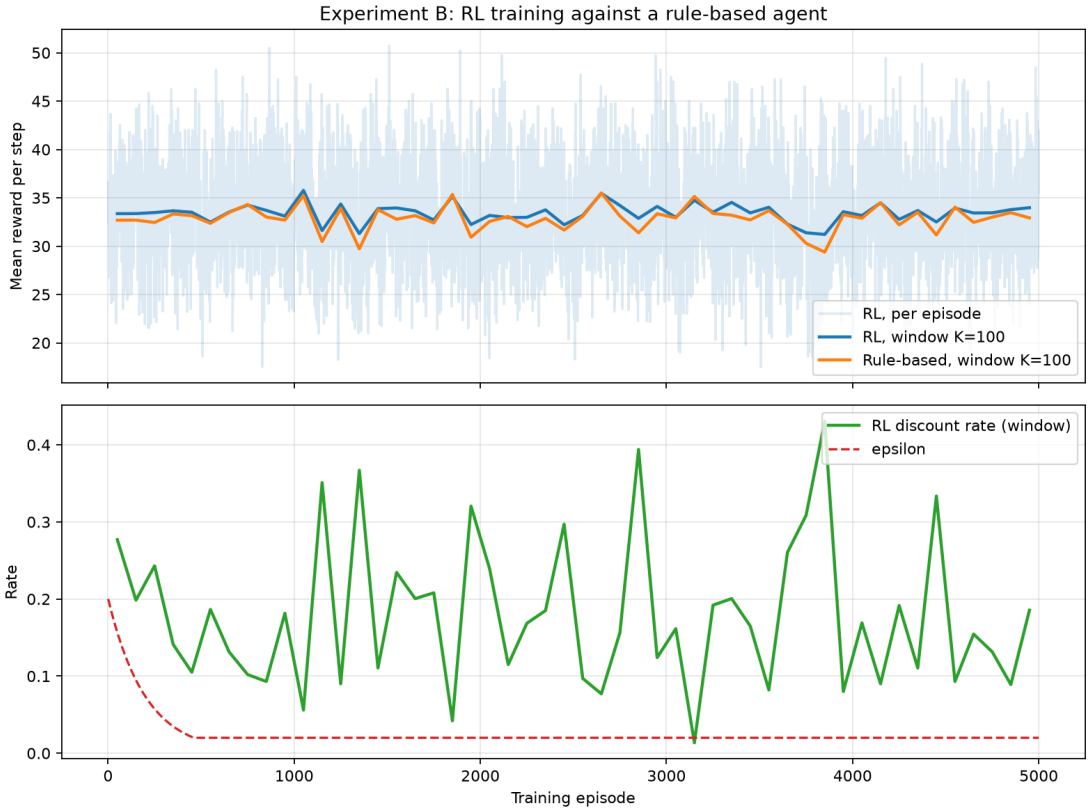
Mọi trạng thái có $Q_0 > Q_1$ nên chính sách tham lam chọn không giảm giá. Độ chênh nhỏ nhất (khoảng 9) nằm ở $(0,0)$ và $(1,1)$. Ở $(0,0)$, điều này khớp với Bảng 6.1: khi nhu cầu thấp, giảm giá chỉ thiệt 1,5 mỗi bước. Trạng thái $(1,1)$ ít được thăm (đối thủ chỉ giảm giá sau khi RL giảm giá) nên ước lượng của nó dựa trên ít dữ liệu. Giá trị Q vào khoảng 200–270 vì nó ước lượng tổng thưởng chiết khấu của nhiều bước còn lại, gần $35/(1-0{,}9) = 350$ ở đầu episode và chỉ còn khoảng 35 ở bước cuối.

6.4.2 Thí nghiệm B: quá trình học

Bảng 6.5 tính $\bar{R}_{i,k} = \sum_{t \in W_k} r_i(t)/(20K)$ với cửa sổ $K = 500$ episode; Hình 6.2 dùng $K = 100$.

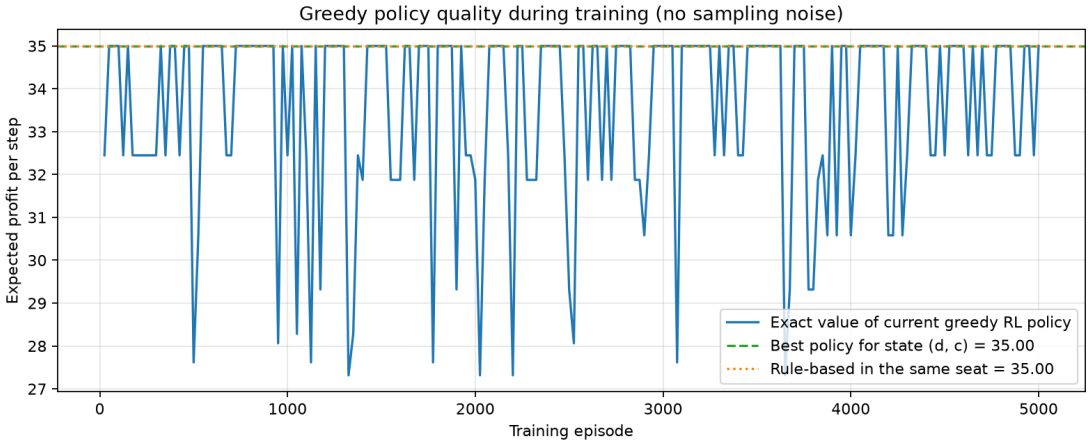
Bảng 6.5: Thưởng trung bình mỗi bước trong huấn luyện theo cửa sổ 500 episode (seed 42).

Episode	RL	Luật	Tỷ lệ giảm giá của RL	ε đầu cửa sổ
1–500	33,474	32,862	0,193	0,200
501–1000	33,412	33,172	0,139	0,020
1001–1500	33,382	32,620	0,195	0,020
1501–2000	33,541	32,922	0,201	0,020
2001–2500	33,015	32,439	0,201	0,020
2501–3000	33,965	33,284	0,170	0,020
3001–3500	33,841	33,472	0,146	0,020
3501–4000	32,492	31,774	0,232	0,020
4001–4500	33,321	32,849	0,179	0,020
4501–5000	33,713	33,177	0,131	0,020

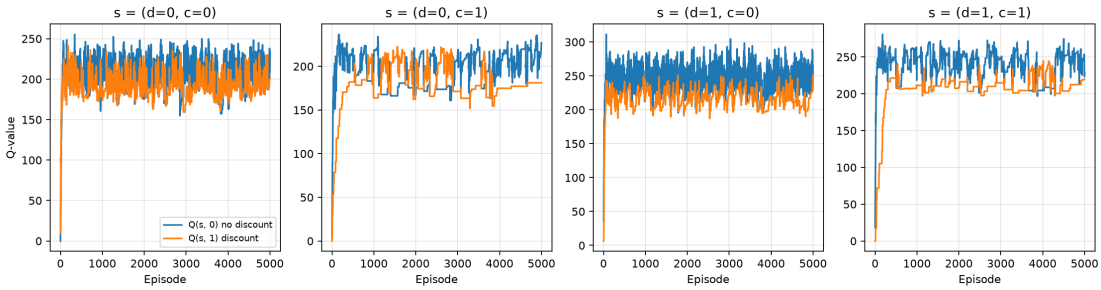


Hình 6.2: Thí nghiệm B: thưởng mỗi bước theo episode, trung bình cửa sổ $K = 100$, tỷ lệ giảm giá của RL và ε (seed 42).

Quan sát. Thưởng huấn luyện của RL dao động quanh 33–34 suốt 5.000 episode và không tăng rõ rệt sau vài trăm episode đầu. Tỷ lệ giảm giá trong huấn luyện vẫn ở mức 0,13–0,23 dù ε đã chạm sàn 0,02 từ khoảng episode 460. Phần khám phá ngẫu nhiên chỉ giải thích được khoảng 0,01 trong tỷ lệ này, nên phần lớn các lần giảm giá đến từ chính sách tham lam tạm thời chọn giảm giá. Hình 6.3 xác nhận điều đó: giá trị kỳ vọng chính xác của chính sách tham lam, đo mỗi 25 episode, liên tục nhảy giữa 35,0 (tối ưu) và các mức 27–33.



Hình 6.3: Giá trị kỳ vọng chính xác của chính sách tham lam trong lúc huấn luyện (seed 42).



Hình 6.4: Quỹ đạo tám giá trị Q cuối mỗi episode (seed 42).

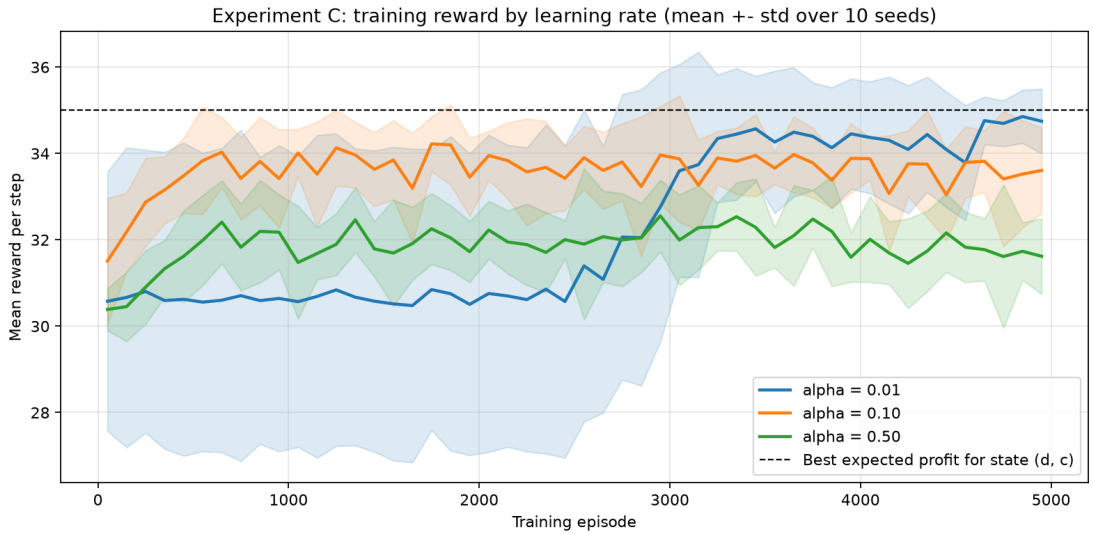
Giải thích. Hình 6.4 cho thấy các giá trị Q không hội tụ mà dao động với biên độ vài chục đơn vị, lớn hơn khoảng cách giữa Q_0 và Q_1 ở trạng thái $(0, 0)$. Có ba nguyên nhân hợp lý: (i) doanh số là biến nhị thức nên thưởng mỗi bước có độ lệch chuẩn lớn (khoảng 11 khi nhu cầu cao); (ii) $\alpha = 0{,}1$ cố định nên mỗi cập nhật chỉ làm trung bình trên khoảng 10 mẫu gần nhất; (iii) trạng thái không chứa vị trí bước trong episode, nên cùng một trạng thái có mục tiêu khoảng 300 ở đầu episode nhưng chỉ $r_t \approx 35$ ở bước cuối. Vì vậy, thưởng huấn luyện tăng hay giảm *không* đủ để kết luận về hội tụ: trong lần chạy này, thưởng huấn luyện gần như không đổi, nhưng chính sách cuối cùng vẫn tối ưu. Chính sách cuối cùng tối ưu một phần nhờ may mắn: nếu dừng ở episode khác, chính sách có thể kém hơn.

6.4.3 Thí nghiệm C: tốc độ học

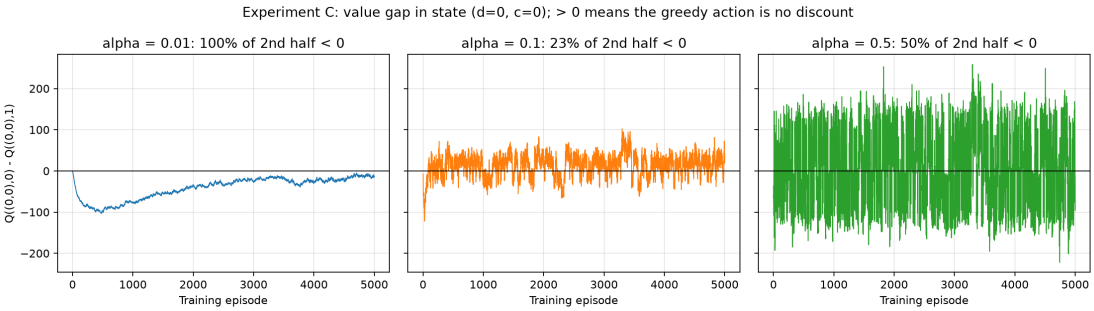
Ba cấu hình $\alpha \in \{0{,}01; 0{,}10; 0{,}50\}$ được chạy với 10 seed; các tham số khác giữ nguyên.

Bảng 6.6: Ảnh hưởng của tốc độ học (10 seed). "Tối ưu" là số seed có chính sách cuối đạt 35,00 kỳ vọng; "Đổi chính sách" là số lần chính sách tham lam thay đổi trong 2.500 episode cuối.

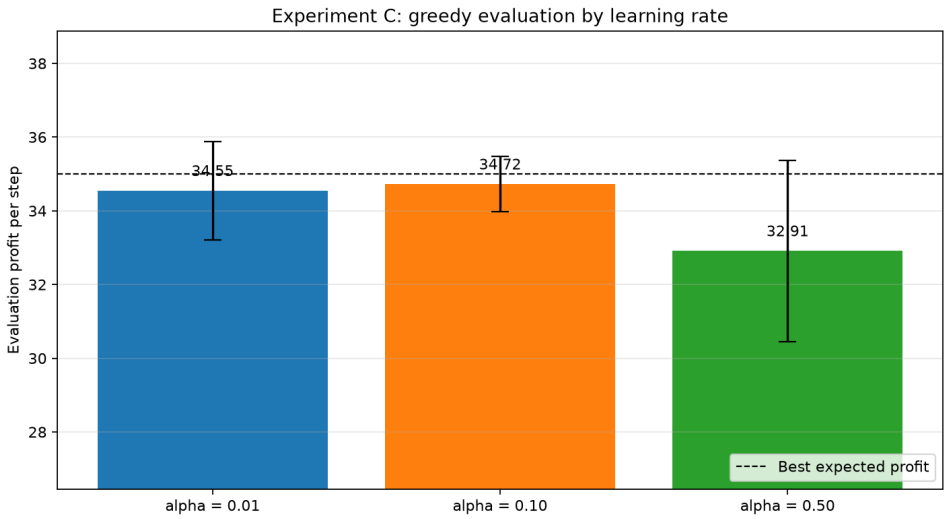
α	Đánh giá (TB $\pm$ SD)	Giảm giá	Huấn luyện	1.000 ep. cuối	Đổi chính sách	Tối ưu
0,01	$34{,}546 \pm 1{,}328$	0,049	32,245	34,414	17,1	9/10
0,10	$34{,}724 \pm 0{,}747$	0,040	33,600	33,566	256,3	9/10
0,50	$32{,}912 \pm 2{,}457$	0,259	31,876	31,763	827,7	5/10



Hình 6.5: Thí nghiệm C: đường học (TB $\pm$ SD trên 10 seed, cửa sổ 100 episode).



Hình 6.6: Thí nghiệm C: hiệu $Q((0,0),0) - Q((0,0),1)$ theo episode, một lần chạy với seed 1042 cho mỗi α.



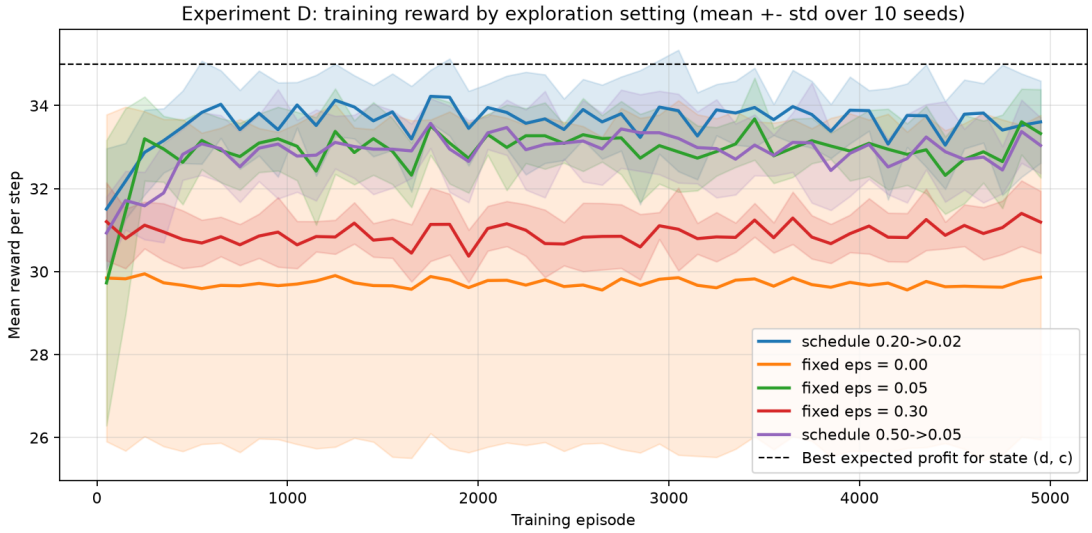
Hình 6.7: Thí nghiệm C: thưởng đánh giá theo α.

6.4.4 Thí nghiệm D: mức khám phá

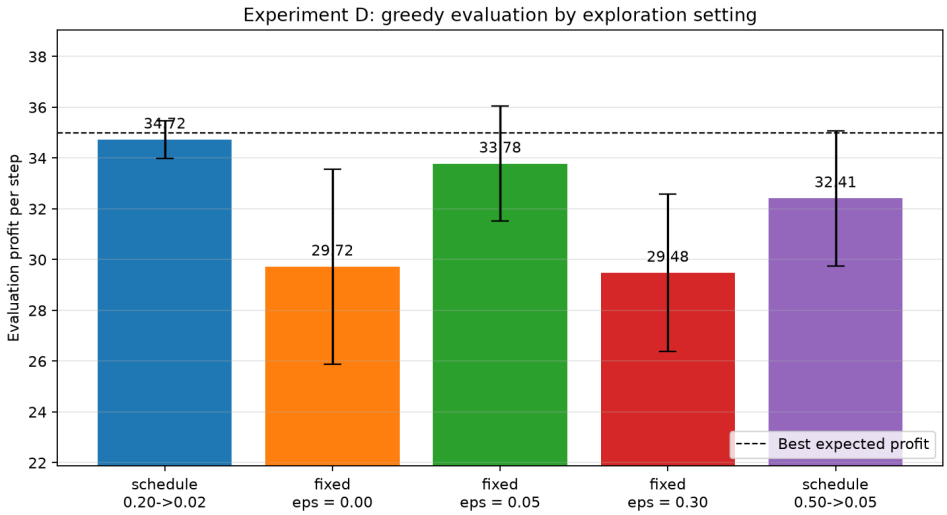
Năm cấu hình được so sánh: lịch tham chiếu $0{,}20 \to 0{,}02$; ε cố định 0; 0,05; 0,30; và lịch cao $0{,}50 \to 0{,}05$ của tài liệu code [18]. Cấu hình $\varepsilon = 0$ được thêm để xem điều gì xảy ra khi agent ngừng khám phá ngay từ đầu.

Bảng 6.7: Ảnh hưởng của khám phá (10 seed). Thưởng huấn luyện là trung bình 5.000 episode.

Cấu hình	Đánh giá (TB $\pm$ SD)	Giảm giá (ĐG)	Huấn luyện	Giảm giá (HL)	Đổi CS	Tối ưu
Lịch $0{,}20 \to 0{,}02$	$34{,}724 \pm 0{,}747$	0,040	33,600	0,167	256,3	9/10
Cố định 0,00	$29{,}719 \pm 3{,}852$	0,415	29,720	0,414	0,0	3/10
Cố định 0,05	$33{,}775 \pm 2{,}267$	0,130	32,907	0,231	502,3	7/10
Cố định 0,30	$29{,}476 \pm 3{,}104$	0,461	30,910	0,400	1.217,0	2/10
Lịch $0{,}50 \to 0{,}05$	$32{,}411 \pm 2{,}656$	0,300	32,867	0,236	505,2	4/10



Hình 6.8: Thí nghiệm D: đường học theo cấu hình khám phá (TB $\pm$ SD trên 10 seed).



Hình 6.9: Thí nghiệm D: thưởng đánh giá theo cấu hình khám phá.

6.4.5 Thí nghiệm E: so sánh công bằng trên nhiều seed

RL (lịch tham chiếu) và agent luật được đánh giá trong các lần chạy tách biệt, cùng ghế 1, cùng đối thủ là agent luật và cùng seed đánh giá (seed huấn luyện $+800.000$).

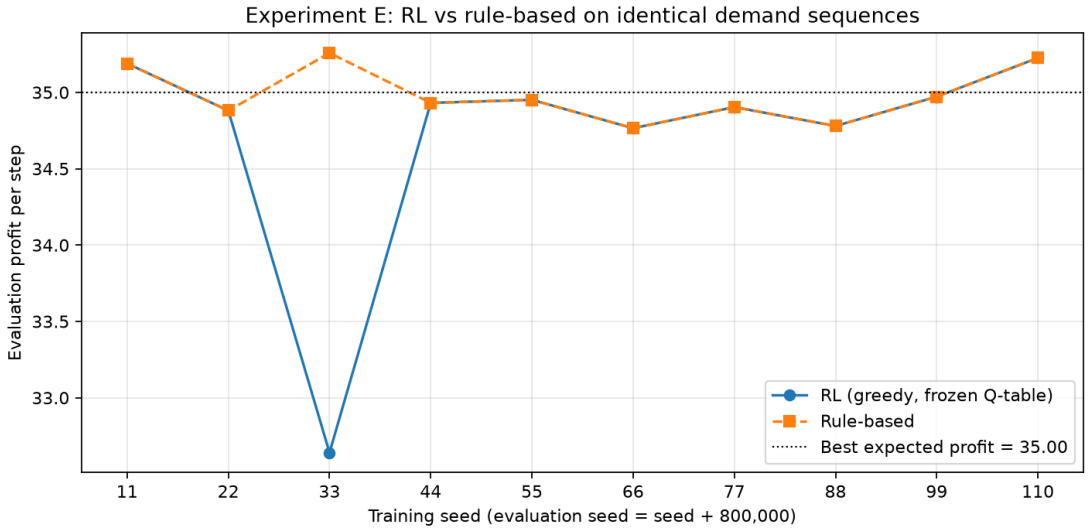
Bảng 6.8: Bảng kết quả theo gợi ý của đề bài (đánh giá tham lam, 10 seed, 10.000 bước mỗi seed).

Agent / cấu hình	Thưởng TB	Độ lệch chuẩn	Tỷ lệ giảm giá
RL, baseline ($\alpha = 0{,}10$, lịch $0{,}20 \to 0{,}02$)	34,724	0,747	0,040
RL, tốc độ học thấp ($\alpha = 0{,}01$)	34,546	1,328	0,049
RL, tốc độ học cao ($\alpha = 0{,}50$)	32,912	2,457	0,259
RL, khám phá khác ($\varepsilon = 0{,}05$ cố định)	33,775	2,267	0,130
RL, khám phá khác ($\varepsilon = 0{,}30$ cố định)	29,476	3,104	0,461
Agent dựa trên luật	34,985	0,178	0,000

Bảng 6.9: So sánh từng cặp RL–luật trên cùng chuỗi nhu cầu (`multi_seed_summary.csv`).

Seed	RL	Luật	RL − Luật	Chính sách RL
11	35,189	35,189	0,000	0100
22	34,881	34,881	0,000	0100
33	32,641	35,258	−2,617	1000
44	34,931	34,931	0,000	0000
55	34,951	34,951	0,000	0000
66	34,765	34,765	0,000	0100
77	34,904	34,904	0,000	0100
88	34,780	34,780	0,000	0000
99	34,972	34,972	0,000	0000
110	35,226	35,226	0,000	0100

Ở 9/10 seed, RL và luật cho kết quả *giống hệt*. Các chính sách `0100` chọn giảm giá ở trạng thái $(0, 1)$, nhưng trạng thái này không bao giờ xuất hiện khi RL không giảm giá: agent luật chỉ giảm giá sau khi RL giảm giá. Vì vậy hành vi thực tế của chúng vẫn là không bao giờ giảm giá. Seed 33 học sai ở trạng thái $(0, 0)$: giảm giá khi nhu cầu thấp, làm lợi nhuận giảm còn 32,64. Hiệu cặp RL−luật có trung bình $-0{,}262$, độ lệch chuẩn $0{,}828$, khoảng tin cậy 95% (phân phối t, 9 bậc tự do) là $[-0{,}854;\ 0{,}330]$, có chứa 0.



Hình 6.10: Thí nghiệm E: RL và luật trên cùng chuỗi nhu cầu theo từng seed.

6.5 Trả lời mười câu hỏi

Câu 1. Mô hình hóa agent và môi trường

- **Agent:** hai người bán. Agent 1 là agent Q-learning, agent 2 là agent dựa trên luật (ở câu 8, cả hai đều là agent Q-learning).
- **Môi trường:** thị trường thương mại điện tử gồm nhu cầu d_t, số khách N_t, xác suất mua (6.2) và cơ chế bán theo phân phối nhị thức.
- **Không gian trạng thái** của mỗi agent: $s_i = (d, c) \in \{0, 1\}^2$, gồm 4 trạng thái. Trạng thái đầy đủ của môi trường là $(d_t, a_1(t-1), a_2(t-1))$, gồm 8 trạng thái; mỗi agent chỉ thấy một phần.
- **Không gian hành động:** $\mathcal{A} = \{0, 1\}$ (không giảm/giảm giá), chọn đồng thời.
- **Hàm thưởng:** lợi nhuận (6.3), ngẫu nhiên vì S_i là biến nhị thức.
- **Chuyển trạng thái:** d_{t+1} đều trên $\{0, 1\}$, độc lập với hành động; $c_i(t+1) = a_{-i}(t)$. Mỗi episode có 20 bước và đặt lại hành động trước về 0.

Đây là hệ *nhiều agent* vì phần thưởng và trạng thái của mỗi agent phụ thuộc vào hành động của agent kia: p_i giảm 0,10 khi đối thủ giảm giá, và $c_i(t+1)$ chính là hành động của đối thủ. Nếu coi đối thủ là một phần của môi trường thì môi trường không còn cố định với agent RL khi đối thủ cũng học (câu 8), và ngay cả với đối thủ luật, môi trường phản ứng với hành động của RL (giảm giá kéo theo đối thủ giảm giá). Mô hình một agent sẽ bỏ qua cả tương tác chiến lược này lẫn câu hỏi về lợi ích của từng bên.

Câu 2. Quyết định dựa trên học và dựa trên luật

Bảng 6.10: So sánh agent RL và agent dựa trên luật.

Tiêu chí	Agent RL (Q-learning)	Agent dựa trên luật
Ra quyết định	$\arg\max_a Q(s, a)$, có khám phá khi huấn luyện	Tra bảng if–else cố định
Dùng kinh nghiệm	Cập nhật Q sau mỗi bước từ thưởng quan sát được	Không dùng thưởng; tri thức nằm trong luật do người viết
Khả năng thích ứng	Tự đổi chính sách khi thị trường đổi; ở mức giảm 5% đạt 36,72 so với 34,99 của luật	Phải viết lại luật; ở mức giảm 5% vẫn giữ nguyên hành vi
Độ ổn định	Phụ thuộc seed, α, ε; SD 0,75 (baseline)	Hoàn toàn nhất quán; SD 0,18 chỉ do nhiễu doanh số
Tính toán	100.000 bước huấn luyện ($\approx$ 0,5 s), bảng 8 giá trị	Không cần huấn luyện, $O(1)$ mỗi quyết định
Giải thích được	Cần đọc bảng Q; khó hơn khi trạng thái lớn	Đọc trực tiếp từ luật

Khi nên dùng agent luật: khi có chính sách được kiểm chứng hoặc ràng buộc pháp lý rõ ràng, ví dụ quy định không được bán dưới giá vốn hay chương trình khuyến mãi theo lịch cố định trong dịp lễ. Khi đó tính nhất quán và khả năng giải thích quan trọng hơn vài phần trăm lợi nhuận. Trong thí nghiệm này, luật tham chiếu tình cờ là chính sách tối ưu nên RL không thể làm tốt hơn. **Khi nên dùng agent RL:** khi điều kiện thị trường thay đổi và không ai biết trước luật tốt nhất, ví dụ chi phí, độ nhạy giá hoặc mức giảm giá thay đổi theo mùa. Kịch bản giảm 5% ở câu 9 cho thấy RL tự phát hiện rằng luôn giảm giá có lợi hơn, trong khi luật cũ mất khoảng 2,35 lợi nhuận mỗi bước.

Câu 3. Cập nhật Q-learning

Công thức (6.4) dịch $Q(s,a)$ một phần α về phía *mục tiêu TD* $r + \gamma \max_{a'} Q(s',a')$. Mục tiêu này là ước lượng một bước của tổng thưởng chiết khấu: thưởng nhận ngay cộng giá trị tốt nhất có thể đạt từ trạng thái kế tiếp. Hiệu $\delta = \text{mục tiêu} - Q(s,a)$ là *sai số TD*: dương thì giá trị cũ đang bị đánh giá thấp. α quyết định mức tin vào mẫu mới; γ quyết định trọng số của tương lai. Toán tử max làm Q-learning học *off-policy*: nó ước lượng giá trị của chính sách tham lam dù hành động đang thực hiện có thể là khám phá.

Với $Q(s,a) = 4$, $\alpha = 0{,}1$, $r = 6$, $\gamma = 0{,}9$, $\max_{a'} Q(s',a') = 5$:

$$\begin{aligned}
\text{Mục tiêu TD} &= r + \gamma \max_{a'} Q(s',a') = 6 + 0{,}9 \times 5 = 6 + 4{,}5 = 10{,}5,\\
\text{Sai số TD } \delta &= 10{,}5 - 4 = 6{,}5,\\
\alpha\,\delta &= 0{,}1 \times 6{,}5 = 0{,}65,\\
Q_{\text{mới}}(s,a) &= 4 + 0{,}65 = 4{,}65.
\end{aligned}$$

Chương trình in cùng kết quả: `Q3: target = 6.0 + 0.9 * 5.0 = 10.50; TD error = 6.50; new Q = 4.0 + 0.1 * 6.50 = 4.65`. **Diễn giải:** kinh nghiệm mới cho thấy hành động a tốt hơn mức agent đang tin (10,5 so với 4), nhưng với $\alpha = 0{,}1$ agent chỉ dịch 10% quãng đường, từ 4 lên 4,65. Nếu cùng mục tiêu lặp lại, Q tiến dần về 10,5 theo cấp số nhân với hệ số $(1-\alpha)$; một mẫu nhiễu đơn lẻ không làm thay đổi đánh giá quá mạnh.

Câu 4. Khám phá và khai thác

ε-greedy chọn hành động ngẫu nhiên với xác suất ε để tiếp tục thu thập thông tin về những hành động có giá trị chưa chắc chắn; phần còn lại khai thác hành động có Q cao nhất. Bảng 6.7 và Hình 6.8 cho thấy:

- **Ngừng khám phá quá sớm** ($\varepsilon = 0$): đánh giá $29{,}72 \pm 3{,}85$, chỉ 3/10 seed tối ưu, chính sách không đổi lần nào trong nửa sau. Lý do: mọi lợi nhuận đều dương và Q khởi tạo bằng 0, nên hành động được thử đầu tiên ở mỗi trạng thái (chọn ngẫu nhiên do hòa) có ngay $Q > 0$ và được chọn mãi mãi. Agent bị khóa vào lựa chọn ngẫu nhiên ban đầu; tỷ lệ giảm giá 0,415 gần với kết quả tung đồng xu.
- **Khám phá quá nhiều** ($\varepsilon = 0{,}30$ cố định): thưởng huấn luyện chỉ 30,91 vì khoảng 15% bước là giảm giá ngẫu nhiên, mỗi lần làm mất lợi nhuận và kích hoạt phản ứng giảm giá của đối thủ. Đáng chú ý, chính sách tham lam cuối cùng cũng kém (29,48, 2/10 tối ưu). Một giải thích phù hợp với dữ liệu: trạng thái (d, c) không chứa hành động trước của chính RL, nên giá trị Q mà Q-learning hội tụ tới phụ thuộc vào tần suất RL tự giảm giá trong lúc huấn luyện. Khám phá nhiều làm thay đổi phân phối đó. Thí nghiệm hiện tại chưa đủ để chứng minh cơ chế này; cần một thí nghiệm riêng với trạng thái đầy đủ.
- **Lịch giảm dần** $0{,}20 \to 0{,}02$ cho kết quả tốt nhất (34,72, 9/10), vì khám phá nhiều lúc bảng Q còn trống, rồi khai thác khi đã có ước lượng. $\varepsilon = 0{,}05$ cố định đứng thứ hai (33,78, 7/10); lịch cao $0{,}50 \to 0{,}05$ kém hơn (32,41, 4/10).

Thưởng huấn luyện và thưởng đánh giá đo hai thứ khác nhau: với $\varepsilon = 0{,}30$, thưởng huấn luyện (30,91) cao hơn thưởng đánh giá (29,48) dù đánh giá không có hành động ngẫu nhiên. Vì vậy chỉ thưởng đánh giá mới phản ánh chất lượng chính sách.

Câu 5. Phần thưởng và mục tiêu kinh doanh

Giảm giá làm tăng xác suất mua thêm 0,15 (công thức (6.2)), nhưng giá bán giảm từ 10 xuống $10 \times (1 - 0{,}20) = 8$ trong khi giá vốn vẫn là 5:

$$\text{Không giảm giá: } 10 - 5 = 5 \text{ trên mỗi sản phẩm,}$$
$$\text{Giảm giá 20\%: } 10 \times 0{,}8 - 5 = 8 - 5 = 3 \text{ trên mỗi sản phẩm.}$$

Biên lợi nhuận giảm 40%, nên giảm giá chỉ có lợi khi số đơn tăng hơn $5/3 \approx 1{,}67$ lần. Khi nhu cầu thấp, tỷ lệ tăng là $0{,}45/0{,}30 = 1{,}5$; khi nhu cầu cao là $0{,}70/0{,}55 \approx 1{,}27$. Cả hai đều dưới 1,67, nên giảm giá làm giảm lợi nhuận kỳ vọng (Bảng 6.1).

Thay đổi hàm thưởng sẽ thay đổi hành vi học được. Chương trình chạy thêm kịch bản phần thưởng là *doanh thu* $S_i \cdot P(1 - \delta a_i)$ thay cho lợi nhuận (Bảng 6.13). Với doanh thu, giảm giá khi nhu cầu thấp cho $10 \times 0{,}45 \times 8 = 36$ so với $10 \times 0{,}30 \times 10 = 30$, tức có lợi trước mắt. Agent RL tham chiếu nâng tỷ lệ giảm giá từ 0,04 lên 0,50, và lợi nhuận thực tế giảm từ 34,72 xuống $29{,}07 \pm 3{,}20$. Agent tối ưu doanh thu nhưng làm doanh nghiệp mất lợi nhuận. Một quan sát phụ: lợi nhuận mỗi đơn với mức giảm 10% là $(5; 4)$, đúng bằng một nửa doanh thu mỗi đơn

với mức giảm 20% là $(10; 8)$. Vì Q-learning không đổi khi nhân thưởng với hằng số dương, hai kịch bản này cho cùng chính sách ở mọi seed (cùng tỷ lệ giảm giá 0,500 và cùng 338,5 lần đổi chính sách). Các thay đổi khác cũng làm đổi hành vi: phạt mỗi lần giảm giá sẽ đẩy agent về phía không giảm giá; thêm thưởng cho thị phần hay khách hàng trung thành có thể làm giảm giá trở nên có lợi. Hàm thưởng phải phản ánh đúng mục tiêu kinh doanh, vì agent sẽ tối ưu chính xác thứ được đo.

Câu 6. Đánh giá hiệu năng

Thưởng trung bình mỗi bước $\bar{R}_i = \frac{1}{T}\sum_t r_i(t)$ trong đánh giá tách biệt:

- Baseline seed 42, $T = 10.000$: RL $345.565/10.000 = 34{,}557$; agent luật ở cùng ghế cũng $34{,}557$; agent luật ở ghế đối thủ $34{,}672$ (Bảng 6.4).
- 10 seed: RL $34{,}724 \pm 0{,}747$; luật $34{,}985 \pm 0{,}178$ (Bảng 6.8); giá trị kỳ vọng chính xác lần lượt $34{,}745 \pm 0{,}807$ và $35{,}000$.

Một lần chạy không đủ để kết luận một phương pháp tốt hơn vì: (i) *nhiễu môi trường*: ngay cả khi chính sách giống hệt, ghế 1 và ghế 2 lệch nhau 0,12 chỉ do doanh số ngẫu nhiên, và giữa các seed đánh giá, cùng một chính sách "không giảm giá" cho kết quả từ 34,77 đến 35,26; (ii) *nhiễu huấn luyện*: chính sách cuối phụ thuộc seed; 9 seed hòa với luật nhưng seed 33 thua 2,62. Nếu chỉ chạy seed 33, ta sẽ kết luận RL kém hẳn; nếu chỉ chạy seed 11 thì kết luận ngược lại; (iii) *phụ thuộc cấu hình*: kết quả đổi theo α, ε và tham số thị trường (câu 9). Với 10 seed và so sánh từng cặp, khoảng tin cậy 95% của hiệu RL$-$luật là $[-0{,}854; 0{,}330]$, tức không có bằng chứng một bên tốt hơn. Đây cũng chỉ là kết luận trong mô hình mô phỏng này, không phải cho thị trường thật.

Câu 7. Tốc độ học và độ ổn định

Theo Bảng 6.6 và các Hình 6.5, 6.6:

- $\alpha = 0{,}01$ **(chậm, ổn định):** mỗi mẫu chỉ đóng góp 1%, nên ước lượng ít nhiễu (chỉ 17,1 lần đổi chính sách trong nửa sau, so với 256,3 ở $\alpha = 0{,}1$). Đổi lại, agent học rất chậm: đường học trung bình đứng quanh 30,6 trong khoảng 2.500 episode đầu, sau đó mới tăng lên khoảng 34,8. Vì Q khởi tạo bằng 0 và mọi thưởng đều dương, hành động được thử nhiều hơn lúc đầu sẽ có Q cao hơn trong thời gian dài. Hình 6.6 cho thấy ở seed 1042, hiệu $Q_0 - Q_1$ tại $(0, 0)$ vẫn âm suốt 5.000 episode (dù đang tăng dần về 0), tức agent chưa kịp sửa lựa chọn ban đầu. Đánh giá $34{,}55 \pm 1{,}33$; 9/10 seed tối ưu.
- $\alpha = 0{,}10$ **(tham chiếu):** học nhanh trong vài trăm episode, kết quả tốt nhất ($34{,}72 \pm 0{,}75$; 9/10), nhưng hiệu $Q_0 - Q_1$ vẫn đổi dấu 23% thời gian nửa sau ở seed 1042.
- $\alpha = 0{,}50$ **(nhanh, dao động):** mỗi cập nhật thay một nửa giá trị cũ bằng một mẫu nhiễu, nên Q gần như chỉ phản ánh vài mẫu gần nhất. Hiệu $Q_0 - Q_1$ dao động trong khoảng ± 200 và âm 50% thời gian; chính sách đổi 827,7 lần trong nửa sau. Thưởng đánh giá thấp nhất ($32{,}91 \pm 2{,}46$, chỉ 5/10 tối ưu).

Tóm lại, α lớn làm agent nhạy với quan sát mới nhưng ước lượng nhiễu và chính sách không ổn định; α nhỏ làm ước lượng mượt nhưng cần nhiều dữ liệu hơn. Với thưởng nhiều nhiễu như ở đây, α giảm dần theo số lần thăm (ví dụ $1/N(s,a)$) hoặc ước lượng mô hình bằng trung bình mẫu (câu 10) là lựa chọn tốt hơn α cố định.

Câu 8. Tương tác nhiều agent và tính không dừng

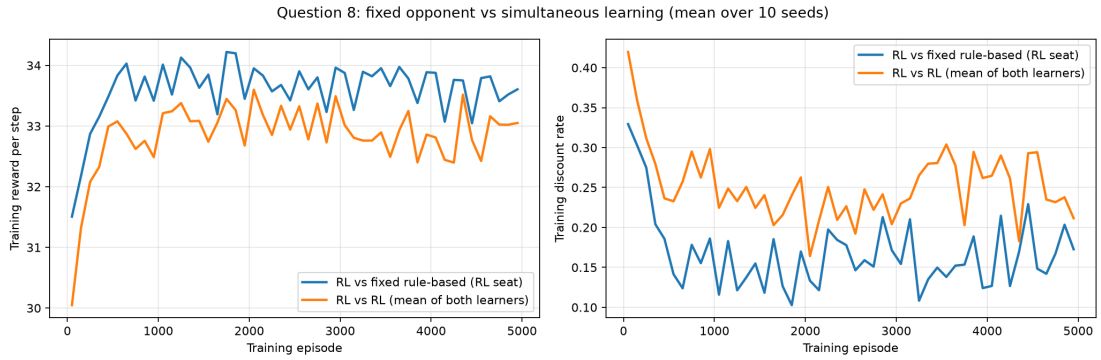
Hành động của đối thủ ảnh hưởng đến agent theo hai đường: (i) *phần thưởng* – khi đối thủ giảm giá, p_i giảm 0,10, ví dụ từ 55 xuống 45 khi nhu cầu cao; (ii) *quan sát* – $c_i(t+1) = a_{-i}(t)$ quyết định agent ở trạng thái nào tiếp theo. Khi đối thủ là agent luật, quan hệ này cố định, nên với agent RL môi trường là dừng (dù không quan sát đầy đủ). Khi *cả hai cùng học*, chính sách đối thủ thay đổi theo thời gian; cùng một cặp (s,a) có phân phối thưởng và trạng thái kế tiếp khác nhau ở các giai đoạn khác nhau. Giả định Markov dừng mà Q-learning cần để hội tụ bị vi phạm: mỗi agent đuổi theo một mục tiêu đang dịch chuyển, và các lần khám phá của agent này trở thành nhiễu trong dữ liệu của agent kia.

Chương trình kiểm tra điều này bằng cách cho hai agent Q-learning độc lập cùng học (10 seed, cùng tham số baseline), rồi so sánh với RL học trước đối thủ cố định:

Bảng 6.11: Đối thủ cố định và hai agent cùng học (đánh giá tham lam, 10 seed).

Thiết lập	Thưởng ĐG (TB $\pm$ SD)	Tỷ lệ giảm giá	Đổi CS (nửa sau)
RL vs luật cố định (ghế RL)	34,724 $\pm$ 0,747	0,040	256,3
RL vs RL (20 agent)	33,266 $\pm$ 2,469	0,187	284,4

```
seed  11 | policies 0100 / 0000 | eval = 35.19 / 35.26 | changes = 271 / 271
seed  44 | policies 1010 / 1101 | eval = 28.73 / 29.90 | changes = 272 / 271
seed  66 | policies 0000 / 1000 | eval = 32.27 / 34.20 | changes = 288 / 297
seed  77 | policies 1100 / 0100 | eval = 33.42 / 32.64 | changes = 247 / 262
seed  99 | policies 0011 / 0100 | eval = 27.31 / 29.62 | changes = 294 / 291
seed 110 | policies 0100 / 1100 | eval = 32.89 / 33.76 | changes = 275 / 296
```



Hình 6.11: Câu 8: thưởng và tỷ lệ giảm giá trong huấn luyện khi đối thủ cố định và khi hai agent cùng học (TB 10 seed).

Khi cùng học, 5/10 seed kết thúc ở cặp chính sách kém (seed 44, 66, 77, 99, 110), so với 1/10 khi đối thủ cố định. Lợi nhuận đánh giá giảm 1,46 mỗi bước, độ lệch chuẩn tăng hơn ba lần, và tỷ lệ giảm giá trong huấn luyện luôn cao hơn (0,20–0,30 so với 0,10–0,20). Ở seed 44 và 99 cả hai agent cùng giảm giá ở nhiều trạng thái, một dạng "chiến tranh giá" nhỏ làm cả hai mất lợi nhuận. Số lần đổi chính sách chỉ tăng nhẹ (284 so với 256); phần lớn dao động đã có sẵn do nhiễu (Thí nghiệm B), nên thí nghiệm này chưa tách riêng được phần do tính không dừng.

Đề xuất thí nghiệm để nghiên cứu sâu hơn: *học luân phiên* – đóng băng agent 2 trong M episode trong khi agent 1 học, rồi đổi vai. Mỗi pha khi đó là bài toán dừng. So sánh với học đồng thời (cùng tổng số bước, cùng seed) về số seed đạt cặp chính sách tốt và tần suất đổi chính sách sẽ cho biết phần suy giảm nào do tính không dừng. Một biến thể khác là đưa hành động trước của chính agent vào trạng thái, hoặc cho agent học trên hành động chung (joint-action learner).

Câu 9. Hạn chế và độ bền vững

Hạn chế của mô phỏng tham chiếu:

1. **Trạng thái không Markov với agent RL.** Agent luật phản ứng với hành động trước của RL, nhưng trạng thái (d, c) của RL không chứa hành động trước của chính nó. Hậu quả của việc giảm giá (đối thủ giảm giá ở bước sau) bị gán cho một trạng thái không liên quan. Trạng thái cũng không chứa số bước còn lại trong episode, trong khi bước cuối dùng mục tiêu khác.
2. **Mô hình chuyển đổi tuyến tính và minh họa:** hệ số 0,15 và 0,10 không được ước lượng từ dữ liệu; nhu cầu chỉ có hai mức và độc lập với hành động (giảm giá không kéo khách quay lại sau đó, không có hiệu ứng mùa vụ).
3. **Chỉ hai hành động và một mức giảm giá;** không có tồn kho, chi phí vận hành, khách hàng trung thành hay khác biệt sản phẩm.
4. **Đối thủ cố định và đối xứng:** RL học trước một luật duy nhất; chính sách học được có thể kém khi đối thủ đổi cách chơi.
5. **Kết luận nhạy với tham số:** chính sách tối ưu đổi hẳn khi mức giảm giá đổi từ 20% xuống 5% (Bảng 6.12).

Thay đổi được cài đặt. Hàm `scenario_list` thay đổi *mức giảm giá* (5%, 10%, 30%), *hàm thưởng* (doanh thu) và *hành vi đối thủ* (tit-for-tat: lặp lại hành động trước của RL bất kể nhu cầu). Mỗi kịch bản được huấn luyện và đánh giá trên 10 seed như Thí nghiệm E; mức tối ưu được tính chính xác.

```
def scenario_list(config):
    return [
        ("discount 20% (reference)", config, RuleBasedAgent),
        ("discount 5%", replace(config, discount_rate=0.05), RuleBasedAgent),
        ("discount 10%", replace(config, discount_rate=0.10), RuleBasedAgent),
        ("discount 30%", replace(config, discount_rate=0.30), RuleBasedAgent),
        ("revenue reward, 20%", replace(config, reward_mode="revenue"),
         RuleBasedAgent),
        ("discount 5%, tit-for-tat", replace(config, discount_rate=0.05),
```

```
        TitForTatAgent),
    ]

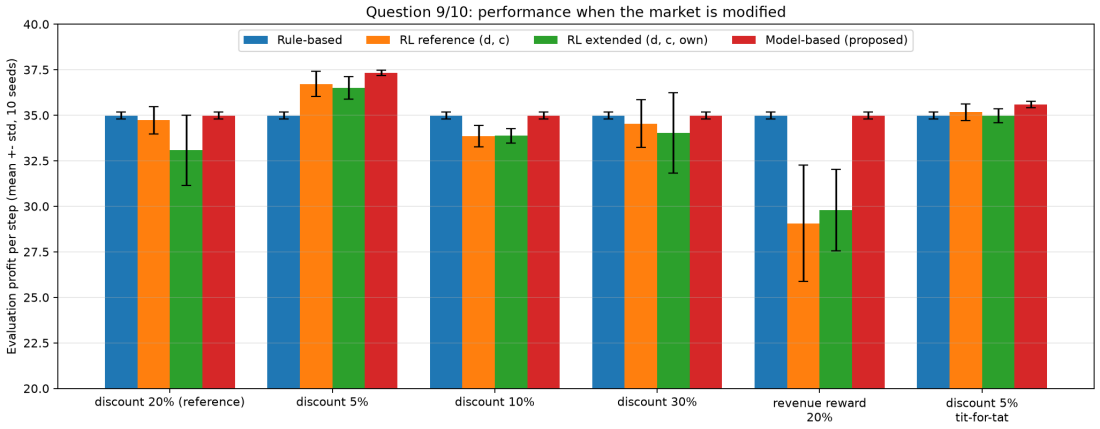

LEARNING_AGENTS = [
    ("RL reference (d, c)", QLearningAgent),
    ("RL extended (d, c, own)", ExtendedQLearningAgent),
    ("Model-based (proposed)", ModelBasedAgent),
]
```

Bảng 6.12: Giá trị kỳ vọng chính xác mỗi bước của các chính sách mốc trong từng kịch bản.

Kịch bản	Không giảm	Luôn giảm	Luật	Tối ưu (d, c)	Chính sách tối ưu
Giảm 20% (tham chiếu)	35,000	24,900	35,000	35,000	0000
Giảm 5%	35,000	37,350	35,000	37,350	1111
Giảm 10%	35,000	33,200	35,000	35,000	0000
Giảm 30%	35,000	16,600	35,000	35,000	0000
Thưởng doanh thu, 20%	70,000	66,400	70,000	70,000	0000
Giảm 5%, đối thủ tit-for-tat	35,000	35,212	35,000	35,675	0011

Bảng 6.13: Lợi nhuận đánh giá mỗi bước (TB $\pm$ SD, 10 seed) và số seed đạt tối ưu. Kịch bản doanh thu báo lợi nhuận thực tế, không phải doanh thu.

Kịch bản	Luật	RL (d, c)	RL (d, c, own)	Dựa trên mô hình
Giảm 20%	$34{,}99 \pm 0{,}18$	$34{,}72 \pm 0{,}75$ (9)	$33{,}07 \pm 1{,}91$ (4)	$34{,}99 \pm 0{,}18$ (10)
Giảm 5%	$34{,}99 \pm 0{,}18$	$36{,}72 \pm 0{,}70$ (2)	$36{,}51 \pm 0{,}62$ (1)	$37{,}33 \pm 0{,}14$ (9)
Giảm 10%	$34{,}99 \pm 0{,}18$	$33{,}85 \pm 0{,}59$ (1)	$33{,}87 \pm 0{,}39$ (1)	$34{,}99 \pm 0{,}18$ (10)
Giảm 30%	$34{,}99 \pm 0{,}18$	$34{,}54 \pm 1{,}32$ (9)	$34{,}03 \pm 2{,}20$ (8)	$34{,}99 \pm 0{,}18$ (10)
Doanh thu, 20%	$34{,}99 \pm 0{,}18$	$29{,}07 \pm 3{,}20$ (1)	$29{,}79 \pm 2{,}24$ (1)	$34{,}99 \pm 0{,}18$ (10)
5%, tit-for-tat	$34{,}99 \pm 0{,}18$	$35{,}16 \pm 0{,}46$ (1)	$34{,}98 \pm 0{,}38$ (0)	$35{,}59 \pm 0{,}18$ (5)



Hình 6.12: Câu 9–10: lợi nhuận đánh giá khi thị trường thay đổi.

Ảnh hưởng của thay đổi.

- **Giảm 5%:** lợi nhuận mỗi đơn khi giảm giá là 4,5. Khi nhu cầu thấp, giảm giá cho $10 \times 0{,}45 \times 4{,}5 = 20{,}25 > 15$; khi nhu cầu cao, $20 \times 0{,}70 \times 4{,}5 = 63 > 55$. Luôn giảm giá trở thành tối ưu (37,35). Agent luật không đổi nên mất 2,35 mỗi bước (6,3%); RL tham chiếu tự chuyển sang giảm giá 75% thời gian và đạt 36,72. Đây là kịch bản cho thấy rõ lợi thế thích ứng của RL.
- **Giảm 10%:** giảm giá có lợi trước mắt (khi nhu cầu thấp 18 so với 15; khi nhu cầu cao 56 so với 55) nhưng kích hoạt phản ứng giảm giá của đối thủ ở bước sau. Tối ưu vẫn là không giảm giá. RL tham chiếu chỉ thấy lợi ích trước mắt, vì hậu quả xảy ra ở trạng thái mà nó không phân biệt được (hạn chế 1). Kết quả là RL giảm giá 50% thời gian và đạt 33,85 – *thua* agent luật.
- **Giảm 30%:** giảm giá thiệt hại rõ (biên lợi nhuận chỉ còn 2), nên RL học đúng 9/10 seed, gần giống kịch bản tham chiếu.
- **Đối thủ tit-for-tat, 5%:** tối ưu là chỉ giảm giá khi nhu cầu cao (35,675). RL tham chiếu đạt 35,16, nhỉnh hơn luật nhưng chỉ 1/10 seed đạt tối ưu, vì khoảng cách giá trị giữa các chính sách rất nhỏ so với nhiễu.

Câu 10. Thiết kế agent tốt hơn

Phân tích ở câu 7–9 chỉ ra hai điểm yếu của agent RL tham chiếu: *trạng thái thiếu thông tin* và *cập nhật TD với α cố định quá nhiễu*. Agent đề xuất (`ModelBasedAgent`) xử lý cả hai.

- **Trạng thái:** $s_i = (d_t,\ a_{-i}(t-1),\ a_i(t-1))$, 8 trạng thái. Thêm hành động trước của chính agent làm trạng thái trở thành Markov khi đối thủ phản ứng theo hành động trước (agent luật, tit-for-tat).
- **Hành động:** $\{0, 1\}$ như đề bài; có thể mở rộng thành nhiều mức giảm giá mà không đổi thuật toán.
- **Hàm thưởng:** lợi nhuận (6.3), đúng mục tiêu kinh doanh (câu 5 cho thấy thưởng doanh thu dẫn tới hành vi sai).
- **Phương pháp học và ra quyết định:** học *mô hình* thay vì học trực tiếp Q. Agent đếm $N(s, a)$, tổng thưởng và tần suất trạng thái kế tiếp để ước lượng $\hat{R}(s, a)$ và $\hat{P}(s' \mid s, a)$. $\hat{P}$ chứa cả quá trình nhu cầu lẫn *mô hình phản ứng của đối thủ*. Cứ 50 episode và khi kết thúc huấn luyện, agent giải $Q(s, a) = \hat{R}(s, a) + \gamma \sum_{s'} \hat{P}(s' \mid s, a) \max_{a'} Q(s', a')$ bằng lặp giá trị với $\gamma = 0{,}98$, gần mục tiêu thưởng trung bình hơn $\gamma = 0{,}9$. Ước lượng bằng trung bình mẫu có phương sai giảm dần theo $1/N$, thay vì giữ mức nhiễu cố định như $\alpha = 0{,}1$. Khám phá dùng cùng lịch ε với baseline để so sánh công bằng.
- **Quy trình đánh giá:** cùng giao thức tách biệt như Thí nghiệm E (10 seed, 500 episode, $\varepsilon = 0$, số ngẫu nhiên chung), cộng giá trị kỳ vọng chính xác và mức tối ưu cho từng kịch bản. Để đo độ bền vững, dùng *regret* = tối ưu − giá trị kỳ vọng, và lấy *regret lớn nhất* trên tập kịch bản đa dạng (mức giảm giá, hàm thưởng, đối thủ). Agent bền vững hơn là agent có regret lớn nhất nhỏ hơn và độ lệch chuẩn giữa các seed nhỏ hơn.

```
class ModelBasedAgent(ExtendedQLearningAgent):
    """
    Question 10: proposed agent.

    State: (d, c, own previous action), as in ExtendedQLearningAgent.
    Learning: instead of a single noisy TD update, the agent counts
        N(s, a), the reward sum and next-state frequencies, giving
        R_hat(s, a) and P_hat(s' | s, a). P_hat contains the demand
        process and a model of the competitor's reaction.
    Planning: value iteration on the learned model every 50 episodes
        and once after training. plan_gamma = 0.98 is closer to the
        average-reward objective used in evaluation.
    Exploration: same epsilon-greedy schedule as the reference agent.
    """

    plan_gamma = 0.98
    plan_every = 50

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        n_states = int(np.prod(self.state_shape))
        self.visits = np.zeros((n_states, 2))
        self.reward_sum = np.zeros((n_states, 2))
        self.transitions = np.zeros((n_states, 2, n_states))

    def index(self, state):
        return int(np.ravel_multi_index(state, self.state_shape))

    def learn(self, state, action, reward, next_state, terminal=False):
        i = self.index(state)
        self.visits[i, action] += 1
        self.reward_sum[i, action] += reward
        self.transitions[i, action, self.index(next_state)] += 1

    def set_episode(self, episode):
        super().set_episode(episode)

        if episode % self.plan_every == 0:
            self.plan()

    def finalize(self):
        self.plan()

    def plan(self):
        visits = np.maximum(self.visits, 1)
        reward = self.reward_sum / visits
        transition = self.transitions / visits[..., None]

        value = np.zeros(reward.shape[0])

        for _ in range(1000):
            q = reward + self.plan_gamma * transition @ value
            new_value = q.max(axis=1)

            if np.max(np.abs(new_value - value)) < 1e-8:
                break

            value = new_value

        self.q = q.reshape(self.state_shape + (2,))

```

Bảng 6.14: Regret = tối ưu − giá trị kỳ vọng chính xác (TB 10 seed). Kịch bản doanh thu tính theo đơn vị doanh thu.

Kịch bản	Luật	RL (d, c)	RL (d, c, own)	Dựa trên mô hình
Giảm 20%	0,000	0,255	1,887	0,000
Giảm 5%	2,350	0,630	0,868	0,020
Giảm 10%	0,000	1,149	1,121	0,000
Giảm 30%	0,000	0,439	0,952	0,000
Doanh thu, 20%	0,000	2,298	2,242	0,000
5%, tit-for-tat	0,675	0,539	0,717	0,085
Regret lớn nhất	2,350	2,298	2,242	**0,085**

Kết quả kiểm tra. Agent dựa trên mô hình đạt tối ưu ở 10/10 seed trong bốn kịch bản, 9/10 ở kịch bản 5% (trung bình 37,33 so với tối ưu 37,35) và 5/10 với đối thủ tit-for-tat (35,59 so với 35,675). Regret lớn nhất chỉ 0,085, so với 2,35 của luật (thua khi giảm 5%) và khoảng 2,3 của RL tham chiếu (thua khi khoảng cách giá trị nhỏ). Chính sách của nó gần như không đổi trong nửa sau huấn luyện (0–2,3 lần, so với hàng trăm lần của Q-learning).

Một kết quả đối chứng quan trọng: *chỉ thêm trạng thái mà vẫn dùng Q-learning* (`ExtendedQLearningAgent`) không cải thiện, thậm chí kém hơn ở kịch bản tham chiếu (33,07, 4/10). Bảng Q có 16 giá trị nên mỗi ô nhận ít dữ liệu hơn, trong khi nhiễu cập nhật không giảm. Như vậy, cải thiện của agent đề xuất đến từ việc *kết hợp* trạng thái đầy đủ với ước lượng ít nhiễu và lập kế hoạch, không đến từ riêng phần trạng thái.

Giới hạn của đề xuất: agent cần không gian trạng thái nhỏ, rời rạc để đếm; nó giả định đối thủ dừng trong lúc học (với đối thủ cũng đang học, mô hình cần quên dữ liệu cũ, ví dụ dùng cửa sổ trượt); và tập kịch bản kiểm tra ở đây vẫn nhỏ. Một kiểm tra mạnh hơn sẽ thêm đối thủ ngẫu nhiên, đối thủ đổi chiến lược giữa chừng và nhu cầu có tính mùa vụ, rồi so sánh regret lớn nhất của cả ba agent trên tập đó.

6.6 Thảo luận, hạn chế và đề xuất

6.6.1 Phân biệt quan sát và giải thích

Bảng 6.15: Quan sát đo được và mức độ chắc chắn của giải thích.

Quan sát	Giải thích	Mức bằng chứng
RL hòa luật ở 9/10 seed (tham chiếu)	Luật trùng chính sách tối ưu; RL tốt nhất chỉ hòa	Chứng minh bằng liệt kê chính xác
Thưởng huấn luyện không tăng sau $\sim$500 episode	Chính sách tham lam dao động giữa tối ưu và kém	Đo trực tiếp (Hình 6.3)
Dao động lớn của Q	Nhiễu nhị thức, α cố định, trạng thái thiếu số bước còn lại	Hợp lý, chưa tách riêng từng nguyên nhân
$\varepsilon = 0$ khóa vào lựa chọn ban đầu	Thưởng luôn dương, Q khởi tạo 0	Suy ra trực tiếp từ công thức cập nhật
$\varepsilon = 0{,}30$ cho chính sách tham lam kém	Điểm hội tụ phụ thuộc phân phối hành vi khi trạng thái không Markov	Giả thuyết, cần thí nghiệm riêng
RL thua luật khi giảm 10%	RL không thấy hậu quả trễ của giảm giá	Phù hợp với tính toán; agent dựa trên mô hình sửa được
Hai agent cùng học kém hơn	Không dừng và vòng phản hồi giảm giá	Quan sát; chưa tách khỏi nhiễu

6.6.2 Các nhận xét chung

- **Agent luật** cho quyết định nhất quán (độ lệch chuẩn giữa các seed chỉ 0,18, hoàn toàn do nhiễu doanh số), nhưng hiệu quả phụ thuộc vào việc luật có khớp với môi trường hay không: tối ưu ở mức giảm 10–30%, mất 6,3% lợi nhuận ở mức 5%.
- **Agent RL** cải thiện được qua phản hồi và thích ứng khi thị trường đổi, nhưng việc học không đảm bảo chính sách tối ưu: kết quả phụ thuộc seed, α, ε và cách biểu diễn trạng thái.
- **Nhiễu ngẫu nhiên** làm thưởng dao động ngay cả khi chính sách không đổi (34,77–35,26 giữa các seed đánh giá của cùng chính sách). Vì vậy giá trị kỳ vọng chính xác và số ngẫu nhiên chung rất hữu ích để so sánh.
- **Hai agent cùng học** ảnh hưởng đến kết quả của nhau (câu 8).
- **Huấn luyện và đánh giá đo hai thứ khác nhau:** thưởng huấn luyện gồm chi phí khám phá và chính sách đang thay đổi; thưởng đánh giá đo chính sách cuối với bảng Q đóng băng.

6.6.3 Hạn chế của chính báo cáo này

- 10 seed đủ để thấy xu hướng nhưng khoảng tin cậy vẫn rộng; các so sánh như $\alpha = 0{,}01$ với $0{,}10$ không tách biệt được về mặt thống kê.
- Agent đề xuất được thiết kế *sau khi* đã thấy điểm yếu của agent tham chiếu trên chính các kịch bản này, nên có nguy cơ thiết kế khớp quá mức với tập kiểm tra. Cần thêm kịch bản mới chưa dùng khi thiết kế để kiểm tra lại.
- Mọi kết luận chỉ đúng trong mô hình mô phỏng đơn giản hóa; mô hình chuyển đổi là minh họa, không được kiểm chứng với dữ liệu thật [18].

6.6.4 Đề xuất cải tiến

1. Đưa hành động trước của chính agent (và nếu cần, số bước còn lại) vào trạng thái; dùng α giảm theo số lần thăm hoặc học dựa trên mô hình.
2. Mở rộng môi trường: nhu cầu có quán tính (chuỗi Markov), nhiều mức giảm giá, tồn kho, khách hàng trung thành để giảm giá có lợi ích dài hạn.
3. Đánh giá theo regret trên một tập kịch bản và đối thủ đa dạng, báo cáo trung bình, độ lệch chuẩn và trường hợp xấu nhất.
4. Với nhiều agent cùng học: học luân phiên, mô hình hóa đối thủ có quên dữ liệu cũ, hoặc các thuật toán học đa agent như joint-action learner.

6.7 Kết luận

Báo cáo đã cài đặt môi trường thương mại điện tử hai người bán, agent Q-learning và agent dựa trên luật, tách riêng giai đoạn huấn luyện và đánh giá, và thực hiện đủ năm thí nghiệm A–E trên 10 seed độc lập cùng ba phần mở rộng cho câu 8–10.

Trong môi trường tham chiếu, kết quả đo được không ủng hộ nhận định RL tốt hơn hay kém hơn luật: luật trùng với chính sách tối ưu (35,00), RL đạt tối ưu ở 9/10 seed và chênh lệch không có ý nghĩa thống kê. Kết quả đổi khi môi trường đổi: ở mức giảm giá 5%, RL thích ứng và vượt luật; ở mức 10%, RL thua luật vì trạng thái không thể hiện hậu quả trễ của việc giảm giá. Tốc độ học và lịch khám phá ảnh hưởng mạnh đến độ ổn định: $\alpha = 0{,}5$ và $\varepsilon = 0{,}30$ cố định cho chính sách kém hơn nhiều; $\varepsilon = 0$ khóa agent vào lựa chọn ngẫu nhiên ban đầu. Khi hai agent cùng học, số seed kết thúc ở chính sách kém tăng từ 1/10 lên 5/10.

Agent đề xuất ở câu 10 – trạng thái có hành động trước của chính agent, học mô hình thị trường và đối thủ bằng đếm, rồi lập kế hoạch – có regret lớn nhất 0,085 trên sáu kịch bản, so với khoảng 2,3 của cả agent luật lẫn agent RL tham chiếu. Bài học chính của thí nghiệm là: chọn agent nào không thể dựa vào một lần chạy với phần thưởng cao nhất; cần một quy trình đánh giá tách biệt, có mốc tối ưu, lặp nhiều seed và thử trên nhiều điều kiện môi trường.

Tài liệu tham khảo

[1] A. M. Turing, "Computing Machinery and Intelligence," *Mind*, 1950. https://academic.oup.com/mind/article/LIX/236/433/986238.

[2] J. McCarthy, M. Minsky, N. Rochester và C. Shannon, "A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence," 1955. https://www-formal.stanford.edu/jmc/history/dartmouth/dartmouth.html.

[3] F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, 1958. https://doi.org/10.1037/h0042519.

[4] D. Rumelhart, G. Hinton và R. Williams, "Learning representations by back-propagating errors," *Nature*, 1986. https://doi.org/10.1038/323533a0.

[5] A. Krizhevsky, I. Sutskever và G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *NeurIPS*, 2012. https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c-Abstract.html.

[6] A. Vaswani và cộng sự, "Attention Is All You Need," *NeurIPS*, 2017. https://arxiv.org/abs/1706.03762.

[7] K. He và cộng sự, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.

[8] Y. Bengio, P. Simard và P. Frasconi, "Learning Long-Term Dependencies with Gradient Descent is Difficult," *IEEE Transactions on Neural Networks*, 1994. https://doi.org/10.1109/72.279181.

[9] A. Krizhevsky, V. Nair và G. Hinton, "CIFAR-10 and CIFAR-100 datasets." https://www.cs.toronto.edu/~kriz/cifar.html.

[10] P. V. Khôi, mã nguồn và báo cáo Assignment 03, 04, 06, 07 của project *PTHTTM*. https://github.com/KhoiF/PTHTTM.

[11] W3C, *RDF 1.1 Concepts and Abstract Syntax*, 2014. https://www.w3.org/TR/rdf-concepts/.

[12] A. Singhal, Google, *Introducing the Knowledge Graph: things, not strings*, 2012. https://blog.google/products-and-platforms/products/search/introducing-knowledge-graph-things-not/.

[13] P. Lewis và cộng sự, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. https://arxiv.org/abs/2005.11401.

[14] Qwen, *Qwen3-0.6B Model Card*. https://huggingface.co/Qwen/Qwen3-0.6B.

[15] D. Edge và cộng sự, *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, 2024. https://arxiv.org/abs/2404.16130.

[16] scikit-learn, *TfidfVectorizer API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[17] Intelligent System Development, *Laboratory Experiment 02: Reinforcement-Learning-Based Agents versus Rule-Based Agents*, năm học 2026–2027, tệp `TAILIEU/ASSIGNMENT-AGENT_02/assignment_Agent02_due_29.10.pdf`.

[18] Intelligent System Development, *AGENT02: Python Implementation – Reinforcement-Learning-Based Agents and Rule-Based Agents*, tệp `TAILIEU/ASSIGNMENT-AGENT_02/agent02_complementary_code.pdf` và mã tham khảo `agent02_experiment.py`.

[19] R. S. Sutton và A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[20] Phan Văn Khôi, *agent02_experiment.py* và kết quả thí nghiệm, https://github.com/KhoiF/PTHTTM, thư mục `src/Agent02/`.